

How to Scale Your Model: Exercise Solutions

Nishant Relan

July 17, 2026

Abstract

Worked solutions to the exercises in *How to Scale Your Model* [1] — the inline pop quizzes and the end-of-chapter worked problems. Each problem restates the book’s prompt, followed by my hand-written solution converted to latex. Chapters 1–8 and 12 are transcribed from my notebook via a scan pipeline; the computational chapters (9–10) are authored by hand.

Contents

1 Chapter 1: All About Rooflines	2
2 Chapter 2: How to Think About TPUs	6
3 Chapter 3: Sharded Matrices and How to Multiply Them	10
4 Chapter 4: All the Transformer Math You Need to Know	20
5 Chapter 5: How to Parallelize a Transformer for Training	26
6 Chapter 6: Training LLaMA 3 on TPUs	28
7 Chapter 7: All About Transformer Inference	32
8 Chapter 8: Serving LLaMA 3-70B on TPUs	37
9 Chapter 9: How to Profile TPU Programs	43
10 Chapter 10: Programming TPUs in JAX	46
11 Chapter 12: How to Think About GPUs	53
A Notebook: Chapter 3, Problem 8 (ICI collective benchmark)	62
B Notebook: Chapter 10 (Programming TPUs in JAX)	70

1 Chapter 1: All About Rooflines

Question 1 [int8 matmul]

Say we want to do the matmul $X[B, D] \cdot_D Y[D, F] \rightarrow Z[B, F]$. Here and throughout we'll use the notation $A \cdot_D B$ to indicate that the multiplication is performing a contraction over the D dimension. This is an abuse of einsum notation. in int8 precision (1 byte per parameter) instead of bfloat16 (2 bytes per parameter) since TPUs/GPUs can do matmuls faster in lower precision.

1. How many bytes need to be loaded from memory? How many need to be written back to memory?
2. How many total OPs are performed?
3. What is the arithmetic intensity?
4. What is a roofline estimate for T_{math} and T_{comms} ? What are reasonable upper and lower bounds for the runtime of the whole operation?

Assume our HBM bandwidth is $8.2\text{e}11$ bytes/s and our int8 peak OPs/s is $3.94\text{e}14$ (about 2x bfloat16).

Solution. (1) Bytes loaded from memory (int8, 1 byte per parameter):

$$BD + DF.$$

Bytes written back to memory:

$$BF.$$

- (2) Total OPs (same as bfloat16, just faster on int8 hardware):

$$2BDF \text{ OPs.}$$

- (3) Arithmetic intensity:

$$I = \frac{2BDF}{BD + DF + BF}.$$

If $B \ll D, F$, then

$$\boxed{I = 2B}.$$

- (4) Roofline estimates:

$$T_{\text{math}} = \frac{2BDF}{3.94\text{e}14}, \quad T_{\text{comms}} = \frac{BD + DF + BF}{8.2\text{e}11}.$$

Since the compute and communication can be overlapped completely, a reasonable lower bound (upper bound on performance) is

$$\max(T_{\text{math}}, T_{\text{comms}}),$$

and an upper bound on runtime is $T_{\text{math}} + T_{\text{comms}}$.

Question 2 [int8 + bf16 matmul]

In practice we often do different weight vs. activation quantization, so we might store our weights in very low precision but keep activations (and compute) in a higher precision. Say we want to quantize our weights in int8 but keep activations (and compute) in bfloat16. At what batch size do we become compute bound? Assume $1.97\text{e}14$ bfloat16 FLOPs/s.

*Hint: this means specifically $\text{bf16}[B, D] * \text{int8}[D, F] \rightarrow \text{bf16}[B, F]$ where B is the “batch size”.*

Solution. We consider $\text{bf16}[B, D] \cdot_D \text{int8}[D, F] \rightarrow \text{bf16}[B, F]$.

FLOPs:

$$2BDF.$$

Memory (activations in bf16 = 2 bytes, weights in int8 = 1 byte):

$$2BD + DF + 2BF.$$

Assuming $B \ll D, F$, the arithmetic intensity is

$$I = \frac{2BDF}{DF} = 2B.$$

We become compute-bound when

$$2B > 240 \Rightarrow \boxed{B > 120}.$$

Question 3

Taking the setup from Question 2, make a roofline plot of peak FLOPs/s vs. B for $F = D = 4096$ and $F = D = 1024$. Use the exact number of bytes loaded, not an approximation.

Solution. The script below computes the achievable FLOP/s as the minimum of the compute roofline and the HBM-bandwidth roofline, using the exact byte count $2BD + DF + 2BF$, and plots it against the batch size B (Figure 1).

```
import numpy as np

PEAK_FLOPS = 1.97e14
HBM = 8.2e11
ACCELERATOR_INTENSITY = PEAK_FLOPS / HBM

import matplotlib.pyplot as plt

def ops(b, d, f):
    return 2 * b * d * f

def memory(b, d, f):
    return 2*b*d + d*f + 2*b*f

def peak_flops_for(bs, d, f):
    ops_values = np.array([ops(b, d, f) for b in bs])
```

```

memory_values = np.array([memory(b, d, f) for b in bs])
arithmetic_intensity = ops_values / memory_values

return np.minimum(arithmetic_intensity * HBM, PEAK_FLOPS)

def main():
    bs = np.linspace(1, 500, 100)

    plt.figure(figsize=(8, 5))
    plt.plot(bs, peak_flops_for(bs, 4096, 4096), label='d = f = 4096', linewidth=2.5)
    plt.plot(bs, peak_flops_for(bs, 1024, 1024), label='d = f = 1024', linewidth=2.5)

    plt.title('Achievable FLOP/s vs. Batch Size')
    plt.xlabel('Batch size')
    plt.ylabel('Achievable FLOP/s')
    plt.grid(True, which='both', linestyle='--', alpha=0.4)
    plt.legend(title='Matrix dimensions')
    plt.tight_layout()
    plt.show()

if __name__ == '__main__':
    main()

```

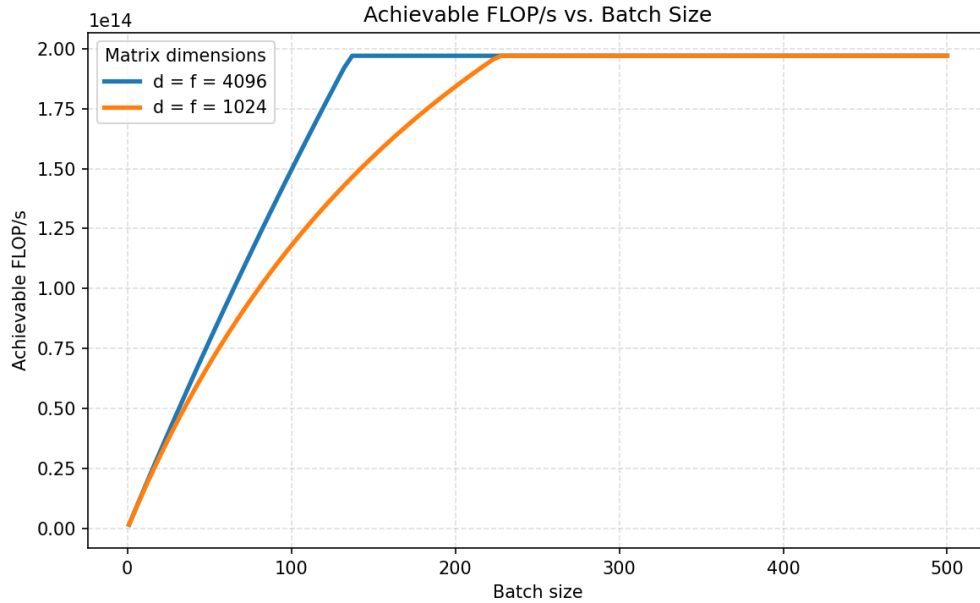


Figure 1: Achievable FLOP/s vs. batch size B for $F = D = 4096$ and $F = D = 1024$.

Question 4

What if we wanted to perform $\text{int8}[B, D] \cdot_D \text{int8}[B, D, F] \rightarrow \text{int8}[B, F]$ where we imagine having a different matrix for each batch element. What is the arithmetic intensity of this operation?

Solution. We consider $\text{int8}[B, D] \cdot_D \text{int8}[B, D, F] \rightarrow \text{int8}[B, F]$ (a separate matrix per batch element).

FLOPs: still

$$2BDF.$$

Memory:

$$BD + BDF + BF.$$

Arithmetic intensity:

$$I \approx \frac{2BDF}{BDF} = 2.$$

Since BDF dominates the denominator, the intensity is essentially constant at 2, so this operation is basically always comms-bound.

Question 5 [Memory Rooflines for GPUs]

Using the spec sheet provided by NVIDIA for the H100 SXM, calculate the batch size at which a bfloat16 matrix multiplication will become compute-bound. *Note that the Tensor Core FLOPs numbers are twice the true value since they're only achievable with structured sparsity.*

Solution. From the spec sheet: FLOPs = 1.98e15, but this is with sparsity. The real value is half that:

$$9.9\text{e}14 \text{ FLOPs/s.}$$

Memory bandwidth $BW = 3.35\text{e}12$ bytes/s. Thus the critical arithmetic intensity is

$$\frac{9.9\text{e}14}{3.35\text{e}12} \approx 296.$$

2 Chapter 2: How to Think About TPUs

Question 1 [bounding LLM latency]

Say you want to sample from a 200B parameter model in bf16 that's split across 32 TPU v4p. How long would it take to load all the parameters from HBM into the systolic array? *Hint: use the numbers above.*

Solution. A 200B parameter model stored in bf16 occupies

$$2 \text{ bytes} \times 200 \times 10^9 = 400 \text{ GB.}$$

Split across 32 chips this is

$$\frac{400 \text{ GB}}{32} = 12.5 \text{ GB/chip} = 1.25 \times 10^{10} \text{ bytes/chip.}$$

The HBM bandwidth for a TPU v4p is about 1.2×10^{12} bytes/s, so the time to load all parameters from HBM is

$$\frac{1.25 \times 10^{10}}{1.2 \times 10^{12}} \approx \frac{1}{100} \text{ s.}$$

$t \approx 1 \times 10^{-2} \text{ s} = 10 \text{ ms}$

Question 2 [TPU details]

Consider a full TPU v5e pod. How many total CPU hosts are there? How many TPU TensorCores? What is the total FLOPs/s for the whole pod? What is the total HBM? Do the same exercise for TPU v5p pod.

Solution. TPU v5e pod: the pod is 16×16 and each host is a 4×2 slice.

Number of hosts:

$$\frac{16 \times 16}{4 \times 2} = \frac{256}{8} = 32 \text{ hosts.}$$

Each v5e chip has one core, so there are 256 TensorCores.

Total FLOPs/s (FLOPs/s in the table is per chip):

$$1.97 \times 10^{14} \times 256 \approx 5 \times 10^{16} \text{ FLOPs/s (bf16).}$$

Total HBM (16 GB/chip):

$$16 \text{ GB} \times 256 = 4096 \text{ GB} = 4 \text{ TB.}$$

$\text{v5e: 32 hosts, 256 cores, } \approx 5 \times 10^{16} \text{ FLOPs/s, 4 TB HBM}$

TPU v5p pod: the pod is $16 \times 20 \times 28$ chips and each host is a $2 \times 2 \times 1$ slice.

Number of hosts:

$$\frac{16 \times 20 \times 28}{2 \times 2} = \frac{8960}{4} = 2240 \text{ hosts.}$$

Each v5p chip has two TensorCores:

$$8960 \times 2 = 17,920 \text{ cores.}$$

Total FLOPs/s:

$$4.59 \times 10^{14} / \text{chip} \times 8960 \text{ chips} \approx 4 \times 10^{18} \text{ FLOPs/s (bf16)}.$$

HBM bandwidth:

$$2.8 \times 10^{12} \times 2240 \approx 6.27 \times 10^{15} \text{ b/s}.$$

Total HBM capacity (96 GB/chip):

$$96 \text{ GB} \times 8960 = 860,160 \text{ GB} \approx 860 \text{ TB}.$$

v5p: 2240 hosts, 17,920 cores, $\approx 4 \times 10^{18}$ FLOPs/s, ≈ 860 TB HBM

Question 3 [PCIe operational intensity]

Imagine we're forced to store a big weight matrix A of type bf16[D, F], and a batch of activations x of type bf16[B, D] in host DRAM and want to do a matrix multiplication on them. This is running on a single host, and we're using a single TPU v6e chip attached to it. You can assume $B \ll D$, and $F = 4D$ (we'll see in future chapters why these are reasonable assumptions). What is the smallest batch size B we need to remain FLOPs bound over PCIe? Assume PCIe bandwidth of 1.6e10 bytes / second.

Solution. We have $A : \text{bf16}[D, F]$ and $x : \text{bf16}[B, D]$, with $F = 4D$.

FLOPs for the matmul $[B, D] \times [D, 4D]$:

$$2 \cdot D \cdot B \cdot 4D = 8BD^2.$$

Bytes of memory to transfer (with $B \ll D$):

$$2BD + 8D^2 \approx 8D^2.$$

Assuming we don't have to transfer the result back to host memory, the arithmetic intensity is

$$\frac{8BD^2}{8D^2} = B.$$

With FLOPs on a v6e chip = 9.2×10^{14} and PCIe bandwidth = 1.6×10^{10} , being FLOPs bound requires

$$B > \frac{9.2 \times 10^{14}}{1.6 \times 10^{10}} \implies \boxed{B > 57,500}.$$

Question 4 [general matmul latency]

Let's say we want to multiply a weight matrix int8[16384, 4096] by an activation matrix of size int8[B, 4096] where B is some unknown batch size. Let's say we're on 1 TPU v5e to start.

1. How long will this multiplication take as a function of B? *Hint: it may help to calculate how long it will take to load the arrays from HBM and how long the multiplication will actually take. Which is bottlenecking you?*
2. What if we wanted to run this operation out of VMEM? How long would it take as a function of B?

Solution. We multiply $\text{int8}[16384, 4096]$ by $\text{int8}[B, 4096]$.

Part 1 (on 1 TPU v5e). HBM bandwidth $= 8.2 \times 10^{11}$, and int8 FLOPs/s $= 1.97 \times 10^{14} \times 2$. FLOPs for this computation:

$$2 \cdot 4096 \cdot 16384 \cdot B = 1.34 \times 10^8 \cdot B \text{ flops,}$$

so

$$T_{\text{math}} = \frac{1.34 \times 10^8 B}{1.97 \times 10^{14} \cdot 2}.$$

HBM read/write bytes:

$$16384 \cdot 4096 + 8 \cdot 4096 + 16384 \cdot B = 6.7 \times 10^7 + 2.049 \times 10^4 B,$$

so

$$T_{\text{comms}} = \frac{6.7 \times 10^7 + 2.049 \times 10^4 B}{8.2 \times 10^{11}} = 8.17 \times 10^{-5} + 2.5 \times 10^{-8} B.$$

Inflection point (setting $T_{\text{math}} = T_{\text{comms}}$):

$$\frac{6.8 \times 10^{-7} B}{2} = 8.17 \times 10^{-5} + 2.5 \times 10^{-8} B \implies \boxed{B \approx 260}.$$

Part 2 (out of VMEM). Using VMEM bandwidth $= 1.6 \times 10^{13}$ b/s instead:

$$\frac{6.8 \times 10^{-7} B}{2} = \frac{6.7 \times 10^7 + 2 \times 10^4 B}{1.6 \times 10^{13}} \implies \boxed{B \approx 11}.$$

Question 5 [ICI bandwidth]

Let's say we have a TPU v5e 4x4 slice. Let's say we want to send an array of type `bf16[8, 128, 8192]` from `TPU{0,0}` to `TPU{3, 3}`. Let's say the per-hop latency for TPU v5e is $1\mu\text{s}$.

1. How soon will the first byte arrive at its destination?
2. How long will the total transfer take?

Solution. We send `bf16[8, 128, 8192]` from `TPU{0,0}` to `TPU{3,3}` on a v5e 4×4 slice. v5e ICI bandwidth $= 4.5 \times 10^{10}$ b/s.

With wrap-around that would be 2 hops, but a 4×4 slice has no wrap-around, so there are 6 hops.

1. First byte. With per-hop latency $1\mu\text{s}$ over 6 hops,

$$\boxed{\text{first byte arrives in } \approx 6\mu\text{s}}.$$

2. Total transfer. Amount of data:

$$2 \cdot 8 \cdot 128 \cdot 8192 = 1.68 \times 10^7 \text{ bytes.}$$

Over a single port,

$$\frac{1.68 \times 10^7}{4.5 \times 10^{10}} = 3.7 \times 10^{-4} \text{ s,}$$

but it can be sent via two ports, so divide by 2:

$$\boxed{\approx 1.86 \times 10^{-4} \text{ s} = 186\mu\text{s transfer time}}.$$

Question 6 [pulling it all together, hard]

Imagine you have a big matrix **A**: `int8[128 * 1024, 128 * 1024]` sharded evenly across a TPU v5e 4x4 slice but offloaded to host DRAM on each chip. Let's say you want to copy the entire array to `TPU{0, 0}` and multiply it by a vector `bf16[8, 128 * 1024]`. How long will this take? *Hint: use the numbers above.*

Solution. Total bytes to transfer (int8):

$$(128 \cdot 1024)^2 = 1.7 \times 10^{10} \approx 17 \text{ GB.}$$

There are two hosts, so each host gets $\approx 8.5 \times 10^9$ bytes.

PCIe transfer to all chips (16 links at 1.6×10^{10} b/s):

$$\frac{8.5 \times 10^9}{1.6 \times 10^{10} \cdot 8 \text{ links}} = 0.0664 \text{ s} = 66 \text{ ms.}$$

ICI time is lower bounded by `TPU{0,0}` receiving $\frac{15}{16} \cdot 1.7 \times 10^{10}$ over its two ICI ports:

$$\frac{\frac{15}{16} \cdot 1.7 \times 10^{10}}{2 \cdot 4.5 \times 10^{10}} \approx 177 \text{ ms.}$$

HBM $\rightarrow$ MXU load time:

$$\frac{\frac{15}{16} \cdot 1.7 \times 10^{10}}{8.2 \times 10^{11}} \approx 20 \text{ ms.}$$

T_{math} (FLOPs):

$$\frac{2(128 \cdot 1024) \cdot 8 \cdot 128 \cdot 1024}{1.97 \times 10^{14}} = \frac{2.75 \times 10^{11}}{1.97 \times 10^{14}} \approx 1.396 \text{ ms.}$$

Overlapping the stages, the total is bottlenecked by the slowest piece:

$$\boxed{\max(\text{times}) \approx 177 \text{ ms}.}$$

Pop Quiz [Calculating VPU throughput]

Using the above information, calculate how many vector FLOPs/s a TPU v5p can perform. A TPU v5p has a clock speed of about 1.75GHz.

Solution. A TPU v5p has a clock speed of $1.75 \text{ GHz} = 1.75 \times 10^9 \text{ cycles/s}$.

Each cycle each core executes 4 vector instructions, and there are $8 \cdot 128 \cdot 4$ ALUs in the VPU. Therefore

$$1.75 \times 10^9 \cdot 8 \cdot 128 \cdot 4 = \boxed{7 \times 10^{12} \text{ FLOPs/s}}.$$

3 Chapter 3: Sharded Matrices and How to Multiply Them

Pop Quiz [2D sharding across 1 axis]

Consider an array `fp32[1024, 4096]` with sharding $A[I_{XY}, J]$ and mesh $\{'X': 8, 'Y': 2\}$. How much data is held by each device? How much time would it take to load this array from HBM on H100s (assuming 3.4×10^{12} memory bandwidth per chip)?

Solution. We have $A[I_{XY}, J]$ with global shape `fp32[1024, 4096]` and mesh $\{X : 8, Y : 2\}$.
Total size of the array:

$$\text{fp32}[1024, 4096] = 4 \cdot 1024 \cdot 4096 \text{ bytes} = 1.67 \times 10^7 \text{ bytes} \approx 16 \text{ MB.}$$

The first dimension is sharded over both X and Y , so we divide by $|X| \cdot |Y| = 16$:

$$\frac{16 \text{ MB}}{16} \approx 1 \text{ MB/device.}$$

Loading this from HBM at 3.4×10^{12} bytes/s:

$$\frac{1 \text{ MB}}{3.4 \times 10^{12}} = \frac{10^6}{3.4 \times 10^{12}} \approx \boxed{294 \text{ ns}}.$$

Local shape (dividing the first dimension by $|X| \cdot |Y| = 16$):

$$\left[\frac{1024}{16}, 4096\right] = \boxed{[64, 4096]}.$$

Pop Quiz

Let **A** be an array with shape `int8[128, 2048]`, sharding $A[I_{XY}, J]$, and mesh `Mesh({'X': 2, 'Y': 8, 'Z': 2})` (so 32 devices total). How much memory does **A** use per device? How much total memory does **A** use across all devices?

Solution. We have $A[I_{XY}, J]$ with global shape `int8[128, 2048]` and mesh $\{X : 2, Y : 8, Z : 2\}$ (32 devices total).

The first dimension is sharded over X and Y , i.e. into $|X| \cdot |Y| = 16$ shards per replica:

$$\frac{128 \cdot 2048}{16} = 8 \cdot 2048 = 16,384 \approx 16 \text{ kB per device.}$$

Since A is replicated over Z (32 devices), the total memory is

$$16 \text{ kB} \cdot 32 = \boxed{512 \text{ kB total}}.$$

Pop Quiz 2 [AllGather time]

Using the numbers from Part 2, how long does it take to perform the `AllGatherY([EY, F]) → [E, F]` on a TPU v5e with a 2D mesh $\{'X': 8, 'Y': 4\}$,

$$E = 2048$$

,

$$F = 8192$$

in bfloat16? What about with

$$E = 256, F = 256$$

?

Solution. We perform $\text{AllGather}_Y([E_Y, F]) \rightarrow [E, F]$ on a TPU v5e with mesh $\{X : 8, Y : 4\}$, in bfloat16.

Part (a): $E = 2048, F = 8192$.

Per-device bytes:

$$2 \cdot \frac{E}{4} \cdot F = 2 \cdot \frac{2048}{4} \cdot 8192 = 2 \cdot 512 \cdot 8192 \approx 8.4 \times 10^6 \text{ bytes/device.}$$

With $W_{\text{ici}}(\text{v5e}) = 9.0 \times 10^{10}$:

$$t = \frac{4 \cdot 8.4 \times 10^6}{9.0 \times 10^{10}} \approx 360 \mu s.$$

This value does not account for the lack of a wraparound connection on this axis. Without wraparound we have to do a maximum of 3 hops instead of 2, so we multiply by $\frac{3}{2}$:

$$t = \frac{3}{2} \cdot 360 \mu s = \boxed{540 \mu s}.$$

Part (b): $E = 256, F = 256$.

$$\text{total bytes} = 2 \cdot 256^2 = 1.3 \times 10^5, \quad \text{bytes/device} = 32768.$$

$$t_{\text{comms}} = \frac{1.3 \times 10^5}{9.0 \times 10^{10}} = 1.44 \times 10^{-6} = 1.44 \mu s.$$

With 3 hops, that is $\approx 3 \mu s$ of latency, so we are latency bound:

$$\boxed{\approx 3 \mu s \text{ total}}.$$

Question 1 [replicated sharding]

An array is sharded $A[I_X, J, K, \dots]$ (i.e., only sharded across X), with a mesh $\text{Mesh}(\{'X': 4, 'Y': 8, 'Z': 2\})$. What is the ratio of the total number of bytes taken up by A across all chips to the size of one copy of the array?

Solution. Let V_A be the number of bytes in array A , sharded $A[I_X, J, K, \dots]$ with mesh $\{X : 4, Y : 8, Z : 2\}$.

Since A is sharded only along X (size 4), each device holds $\frac{V_A}{4}$.

With $\#$ devices = 64, the total bytes across all chips are

$$64 \cdot \frac{V_A}{4} = 16 V_A.$$

Therefore the ratio of total size to one copy is

$$\boxed{\text{ratio} = 16}.$$

Question 2 [AllGather latency]

How long should $\text{AllGather}_X([B_X, D_Y])$ take on a TPU v4p 4x4x4 slice with mesh $\text{Mesh}(\{'X': 4, 'Y': 4, 'Z': 4\})$ if $B = 1024$ and $D = 4096$ in bfloat16? How about

$$\text{AllGather}_{XY}([B_X, D_Y])$$

? How about

$$\text{AllReduce}_Z([B_X, D_Y]\{U_Z\})$$

?

Solution. On a TPU v4p $4 \times 4 \times 4$ slice ($W_{\text{ici}} = 9 \times 10^{10}$ bidirectional), with $B = 1024$, $D = 4096$ in bfloat16. Here $V = 2 \cdot 1024 \cdot 4096$.

Part 1: $\text{AllGather}_X([B_X, D_Y])$. We only gather over X , and the array is sharded over Y , so effectively we gather V/Y :

$$\frac{V}{W_{\text{ici}} \cdot Y} = \frac{2 \cdot 1024 \cdot 4096}{W_{\text{ici}} \cdot 4} \approx \boxed{23 \mu s}.$$

Part 2: $\text{AllGather}_{XY}([B_X, D_Y])$. Twice the bandwidth but gathering the full array:

$$\frac{V}{W_{\text{ici}} \cdot 2} = \frac{2 \cdot 1024 \cdot 4096}{W_{\text{ici}} \cdot 2} \approx \boxed{46 \mu s}.$$

Part 3: $\text{AllReduce}_Z([B_X, D_Y]\{U_Z\})$. An AllReduce costs twice an AllGather; each shard has size $\frac{2BD}{X \cdot Y}$:

$$\frac{4BD}{X Y W} = \frac{2 \cdot 1024 \cdot 4096}{16 \cdot W_{\text{ici}}} \approx \boxed{11.5 \mu s}.$$

Question 3 [latency-bound AllGather]

Let's say we're performing an $\text{AllGather}_X([B_X])$ but B is very small (say 128). How long should this take on a TPU v4p 4x4x4 slice with mesh $\text{Mesh}(\{'X': 4, 'Y': 4, 'Z': 4\})$ in bfloat16? *Hint: you're probably latency bound.*

Solution. $\text{AllGather}_X([B_X])$ with $B = 128$ in bfloat16 on a TPU v4p $4 \times 4 \times 4$ slice.

The array uses only $2B = 256$ bytes total. The bandwidth time is

$$\frac{2B}{W_{\text{ici}}} = \frac{256}{9 \times 10^{10}} \approx 3 \text{ ns} \quad (\text{bandwidth time}).$$

The latency: over an axis of size 4 with a wraparound link, the full gather takes 2 hops $\rightarrow 2 \mu s$. So the total time is

$$\max(3 \text{ ns}, 2 \mu s) = \boxed{2 \mu s}.$$

Question 4 [matmul strategies]

To perform $X[B, D] \cdot_D Y[D_X, F] \rightarrow Z[B, F]$, in this section we tell you to perform $\text{AllGather}_X(Y[D_X, F])$ and multiply the fully replicated matrices (Case 2, *Strategy 1*). Instead, you could multiply the local shards like $X[B, D_X] \cdot_D Y[D_X, F] \rightarrow Z[B, F]$

U_X (Case 3, *Strategy 2*), and then $\text{AllReduce}_X(Z[B, F])$

U_X). How many FLOPs and comms does each of these perform? Which is better and why?

Solution.

Q4 We compare two ways to perform $X[B, D] \cdot_D Y[D_X, F] \rightarrow Z[B, F]$. Let C be FLOPs/s per chip and W ($=W_{\text{ici}}$) the bandwidth per chip.

Strategy A (AllGather then multiply):

$$\text{AllGather}_X(Y[D_X, F]) = \frac{V}{W_{\text{ici}}} = \frac{2DF}{W_{\text{ici}}}.$$

comms = $2DF$, FLOPs = $2DBF$.

$$T_{\text{math}} = \frac{2BDF}{C}, \quad T_{\text{comms}} = \frac{V}{W_{\text{ici}}} = \frac{2DF}{W_{\text{ici}}},$$

$$T_{\text{total(A)}} = \max\left(\frac{2BDF}{C}, \frac{2DF}{W_{\text{ici}}}\right).$$

Strategy B (multiply local shards then AllReduce): $X[B, D_X] \cdot Y[D_X, F] \rightarrow Z[B, F]\{U_X\}$, then AllReduce_X . comms = $4BF$, FLOPs = $2D_X BF = \frac{2DBF}{X}$.

$$T_{\text{math}} = \frac{2 \cdot \frac{D}{X} \cdot BF}{C} = \frac{2DBF}{XC}, \quad T_{\text{comms}} = \frac{2V}{W_{\text{ici}}} = \frac{2BF \cdot 2}{W_{\text{ici}}} = \frac{4BF}{W_{\text{ici}}},$$

$$T_{\text{total(B)}} = \max\left(\frac{2DBF}{XC}, \frac{4BF}{W_{\text{ici}}}\right).$$

Which regime are we in? For Strategy B, comms-bound when

$$\frac{2BDF}{XC} < \frac{4BF}{W_{\text{ici}}} \Rightarrow \frac{2D}{XC} = \frac{4}{W_{\text{ici}}} \Rightarrow D = \frac{2XC}{W_{\text{ici}}} = 2X \cdot \frac{C}{W_{\text{ici}}}.$$

With $\frac{C}{W_{\text{ici}}} \approx 2500$, this gives $D = 5000X$. If $D < 10000$, then $X < 2$; so if $X \geq 2$, D is below this threshold and Strategy B is comms bound.

For Strategy A, comms-bound when

$$\frac{2BDF}{C} < \frac{2DF}{W_{\text{ici}}} \Rightarrow \frac{B}{C} = \frac{1}{W_{\text{ici}}} \Rightarrow B = \frac{C}{W_{\text{ici}}} = 2500.$$

If $B < 2500$ we are comms bound, otherwise compute bound.

Suppose $B < 2500$. Then both strategies are comms bound and we compare

$$\frac{2DF}{W} \text{ vs } \frac{4BF}{W} \Rightarrow D \text{ vs } 2B.$$

Usually $D > 2B$ (and $D > 5000$), so

Strategy B is generally better (matmul then AllReduce), especially for large models with $D > 5000$.

Q4 (continued) This continues the comparison of the two strategies for the large-batch case. Suppose the batch is large, say $B \gtrsim 2500$, so that we are still comms-bound for strategy B but compute-bound for strategy A. Then strategy A is better than strategy B exactly when

$$\frac{2BDF}{C} < \frac{4BF}{W} \iff \frac{D}{C} < \frac{2}{W} \iff D < \frac{2C}{W} \iff \boxed{D < 5000}.$$

So if D is large and B is large, strategy B still wins.

Question 5 [minimum latency]

Let's say I want to do a matmul $A[I, J] \cdot_J B[J, K] \rightarrow C[I, K]$ on a TPU v4p 4x4x4 with the lowest possible latency. Assume the inputs can be sharded arbitrarily but the result should be fully replicated. How should my inputs be sharded? What is the total FLOPs and comms time?

Solution. We want $A[I, J] \cdot_J B[J, K] \rightarrow C[I, K]$ on a TPU v4p $4 \times 4 \times 4$ with the result fully replicated. The candidate shardings are:

Option 1: fully replicate.

Option 2: $A[I, J_{XYZ}] \cdot B[J_{XYZ}, K]$ (local multiply + AllReduce over the contracting axis).

Option 3: $A[I_X, J_Y] \cdot B[J_Y, K_Z]$ (local multiply, AllGather across Y , then AllGather across X, Z).

Option 4: $A[I_X, J] \cdot B[J, K_Y]$ (local multiply, then AllGather across X, Y).

Option 5: $A[I_{XY}, J] \cdot B[J, K_Z]$ (shard the outer dimensions across all available axes).

Analyze Option 1. There are no comms, so

$$T_{\text{math}} = \frac{2IJK}{C}, \quad T_{\text{comms}} = 0, \quad \text{time} = \frac{2IJK}{C}.$$

Analyze Option 4 (sharding I over X and K over Y):

$$T_{\text{math}} = \frac{2 \cdot J \cdot \frac{I}{4} \cdot \frac{K}{4}}{C} = \frac{IJK}{8C}, \quad T_{\text{comms}} = \frac{2IK}{2W} = \frac{IK}{W},$$

$$\text{so } T_{\text{total}} = \max\left(\frac{IJK}{8C}, \frac{IK}{W}\right).$$

Analyze Option 5 (sharding I over XY , i.e. 16 devices, and K over Z):

$$T_{\text{math}} = \frac{2 \cdot \frac{I}{16} \cdot J \cdot \frac{K}{4}}{C} = \frac{IJK}{32C}, \quad T_{\text{comms}} = \frac{2IK}{3W} = \frac{2}{3} \frac{IK}{W}.$$

The big idea is that sharding the outer dimensions across all available axes is the way to go. Comparing Option 5 to Option 1 (Option 5 is always comms-bound here), Option 1 takes more time when

$$\begin{aligned} \frac{2IJK}{C} &> \frac{2}{3} \frac{IK}{W} \\ \frac{J}{C} &> \frac{1}{3W} \\ J &> \frac{C}{3W} \approx 800. \end{aligned}$$

So when $J > 800$, Option 1 takes more time, and

Option 5 (or any sharding of the outer dimensions across all axes) is optimal.

Question 6

Let's say we want to perform $A[I_X, J_Y] \cdot_J B[J_Y, K] \rightarrow C[I_X, K]$ on TPU v5e 4x4. What communication do we perform? How much time is spent on communication vs. computation?

- What about $A[I_X, J] \cdot_J B[J_X, K_Y] \rightarrow C[I_X, K_Y]$? This is the most standard setting for training where we combine data, tensor, and ZeRO sharding.

- What about $A[I_X, J] \cdot_J B[J, K_Y] \rightarrow C[I_X, K_Y]$? This is standard for inference, where we do pure tensor parallelism (+data).

Solution. Q6: TPU v5e 4×4 , with $C = 1.97 \times 10^{14}$ FLOPs/s and $W_{\text{ici}} = 9 \times 10^{10}$ bytes/s.

Part (a): $A[I_X, J_Y] \cdot_J B[J_Y, K] \rightarrow C[I_X, K]$. The contracting axis J is sharded over Y , so the comms is an AllReduce over Y .

$$T_{\text{math}} = \frac{2 \cdot \frac{I}{4} \cdot \frac{J}{4} \cdot K}{C} = \frac{IJK}{8C}.$$

For comms, the local output is $V = 2 \cdot \frac{I}{4} \cdot K$ bytes; an AllReduce costs twice an AllGather, and with the no-wraparound factor $\frac{3}{2}$,

$$T_{\text{comms}} = \frac{V}{W_{\text{ici}}} \cdot 2 \cdot \frac{3}{2} = \frac{2 \cdot \frac{I}{4} \cdot K \cdot 2 \cdot 3}{2W_{\text{ici}}} = \frac{3IK}{2W_{\text{ici}}}.$$

We are compute-bound when

$$\frac{IJK}{8C} > \frac{3IK}{2W_{\text{ici}}} \iff J > \frac{12C}{W_{\text{ici}}} \approx 30\text{K}.$$

Since J won't be 30K, we are comms bound at $T_{\text{comms}} = \frac{3IK}{2W_{\text{ici}}}$.

Part (b): $A[I_X, J] \cdot_J B[J_X, K_Y] \rightarrow C[I_X, K_Y]$. Here we AllGather B over X (to un-shard J_X), then multiply locally.

$$T_{\text{math}} = \frac{2 \cdot \frac{I}{4} \cdot J \cdot \frac{K}{4}}{C} = \frac{IJK}{8C}.$$

The bytes gathered are $V = 2 \cdot J \cdot \frac{K}{4}$, and with the $\frac{3}{2}$ no-wraparound factor,

$$T_{\text{comms}} = \frac{V}{W_{\text{ici}}} \cdot \frac{3}{2} = \frac{3JK}{4W_{\text{ici}}}.$$

We are compute-bound when

$$\frac{IJK}{8C} > \frac{3JK}{4W_{\text{ici}}} \iff I > \frac{6C}{W_{\text{ici}}} \approx 12\text{K}.$$

So we are compute bound when $I > 12\text{K}$.

c): This is the case $A[I_X, J] \cdot_J B[J, K_Y] \rightarrow C[I_X, K_Y]$. The contracting dimension J is unsharded on both operands, so no communication is required:

$$A[I_X, J] \cdot B[J, K_Y] \rightarrow C[I_X, K_Y] \quad (\text{no comms!})$$

Compute time (mesh $\{X : 4, Y : 4\}$, so the local shard is $I/4 \times K/4$):

$$T_{\text{math}} = \frac{2 \cdot \frac{I}{4} \cdot J \cdot \frac{K}{4}}{C} = \frac{IJK}{8C}.$$

Since there are no comms, this operation is purely compute bound.

Question 7

A typical Transformer block has two matrices $W_{\text{in}}[D, F]$ and $W_{\text{out}}[F, D]$ where $F \gg D$. Say we have a batch size B . Then the full block is $\text{In}[B, D] \cdot W_{\text{in}}[D, F] \cdot W_{\text{out}}[F, D]$. Let's pick $D = 8192$, $F = 32768$, and $B = 128$ and assume everything is in bfloat16. Assume we're running on a TPU v5e 2x2 slice but let's pretend each TPU only has 300MB of free memory. How should In , W_{in} , W_{out} , and Out be sharded to stay below the memory limit while minimizing overall time? How much time is spent on comms and FLOPs? *Hint: the final output doesn't need to be fully replicated, but it should be sharded the same as the input so the "layer" can be repeated.*

Solution. We have $\text{In}[B, D] \cdot W_{\text{in}}[D, F] \cdot W_{\text{out}}[F, D]$ with $B = 128$, $D = 8192$, $F = 32768$ in bfloat16 on a TPU v5e 2×2 slice, with only 300MB free per device.

Memory of the pieces:

$$\begin{aligned}\text{sizeof}([B, D]) &= 2BD = 2\text{MB}, \\ \text{sizeof}([D, F]) &= \text{sizeof}([F, D]) = 2FD = 537\text{MB}.\end{aligned}$$

The two weight matrices are far too big to replicate, so we shard the matmuls. We don't want to communicate over $W_{\text{in}}/W_{\text{out}}$, so we keep those local and do communication on the small $[B, D]$ activations:

$$\begin{aligned}[B, D] \cdot [D, F_{xy}] &\rightarrow [B, F_{xy}] \quad (\text{local}), \\ [B, F_{xy}] \cdot [F_{xy}, D] &\rightarrow [B, D]\{U_{xy}\} \quad (\text{local}).\end{aligned}$$

We could AllReduce here, or instead shard the input as $[B, D_{xy}]$, do an AllGather up front, and a ReduceScatter at the end. Since an AllReduce is roughly $2\times$ as expensive as an AllGather + ReduceScatter, the two are roughly equivalent comms-wise, but sharding the input is more scalable, so we go with that.

Per-device memory check:

$$2\text{MB} + \frac{537}{2}\text{MB} \approx 270\text{MB} < 300\text{MB} \quad \checkmark$$

Final algorithm (with $C = 1.97 \times 10^{14}$, $W_{\text{ici}} = 9.0 \times 10^{10}$):

$$\text{AllGather}_{xy}[B, D_{xy}], \text{ two local matmuls, ReduceScatter } [B, D]\{U_{xy}\} \rightarrow [B, D_{xy}].$$

Comms (AllGather then ReduceScatter, each moving $\approx 2BD$):

$$T_{\text{comms}} = \frac{4BD}{W_{\text{ici}}} \approx \frac{46.6 \mu s}{2} \approx 23 \mu s.$$

Compute (two local matmuls, each $2BDF/(4C)$):

$$T_{\text{math}} = \frac{2BDF}{4C} + \frac{2BDF}{4C} = \frac{BDF}{C} \approx 174 \mu s.$$

Overall,

$$\max(T_{\text{math}}, T_{\text{comms}}) \leq T_{\text{total}} \leq T_{\text{math}} + T_{\text{comms}},$$

$$\boxed{174 \mu s \leq T_{\text{total}} \leq 197 \mu s.}$$

Question 9 [another strategy for sharded matmuls?]

Above we claimed that when only one input to a matmul is sharded along its contracting dimension, we should AllGather the sharded matrix and perform the resulting contraction locally. Another strategy you might think of is to perform the sharded matmul and then AllReduce the result (as if both inputs were sharded along the contracting dimension), i.e. $A[I, J_X] *_J B[J, K] \rightarrow C[I, K]$ by way of

1. $C[I, K]$
 $U_X = A[I, J_X] \cdot B[J_X, K]$
2. $C[I, K] = \text{AllReduce}(C[I, K]$
 $U_X)$

Answer the following:

1. Explicitly write out this algorithm for matrices $A[N, M]$ and $B[M, K]$, using indices to show exactly what computation is done on what device. Assume A is sharded as $A[I, J_X]$ across ND devices, and you want your output to be replicated across all devices.
2. Now suppose you are ok with the final result not being replicated on each device, but instead sharded (across either the N or K dimension). How would the algorithm above change?
3. Looking purely at the communication cost of the strategy above (in part 2, not 1), how does this communication cost compare to the communication cost of the algorithm in which we first AllGather A and then do the matmul?

Solution. Part 1. For $A[N, M]$ and $B[M, K]$, let d be the number of devices, so the number of M -indices per device is $M/d = \ell$. For device p , $0 \leq p \leq d$, we compute the partial sum

$$O_{ik} = \sum_{j=p\ell+1}^{(p+1)\ell} A_{ij} B_{jk},$$

i.e. each device sums over only the contracting indices j that it stores. Then AllReduce the result to form the full $C[I, K]$.

Part 2. If the output only needs to be sharded (not replicated), we can replace the AllReduce with a cheaper ReduceScatter:

$$\text{ReduceScatter}(C[I, K]\{U_X\}) = C[I_X, K].$$

Part 3. Comparing communication costs:

$$\begin{aligned} \text{Strategy 1 (AllGather } A): \quad & \frac{V}{W}, \quad V = NM, \\ \text{Strategy 2 (ReduceScatter } O): \quad & \frac{V}{W}, \quad V = NK. \end{aligned}$$

So the cost ratio of Strategy 1 to Strategy 2 is

$$\boxed{\frac{M}{K}}.$$

Question 10: Fun with AllToAll

In the table above, it was noted that the time to perform an AllToAll is a factor of 4 lower than the time to perform an AllGather or ReduceScatter (in the regime where we are throughput-bound). In this problem we will see where that factor of 4 comes from, and also see how this factor would change if we only had single-direction ICI links, rather than bidirectional ICI links.

1. Let's start with the single-direction case first. Imagine we have D devices in a ring topology and want to do either an AllGather or a ReduceScatter on an $N \times N$ matrix $A[I_X, J]$ (say D divides N for simplicity). Describe the comms involved in these two collectives, and calculate the total number of scalars (floats or ints) which are transferred across **a single** ICI link during the entirety of this algorithm.
2. Now let's think about an AllToAll, still in the single-directional ICI case. How is the algorithm different in this case than the all-gather case? Calculate the number of scalars that are transferred across a single ICI link in this algorithm.
3. You should have found that the ratio between your answers to part (a) and part (b) is a nice number. Explain where this factor comes from in simple terms.
4. Now let's add bidirectional communication. How does this affect the total time needed in the all-gather case?
5. How does adding bidirectional communication affect the total time needed in the AllToAll case?
6. Now simply explain the ratio between AllGather time and AllToAll time in a bidirectional ring.

Solution. This problem investigates where the factor-of-4 speedup of an AllToAll (relative to an AllGather/ReduceScatter) comes from, and how it changes between unidirectional and bidirectional ICI rings.

Part 1 (unidirectional AllGather / ReduceScatter). Each device holds a strip of the matrix of size $(\frac{N}{D}, N)$, so the number of scalars in one strip is $\frac{N^2}{D}$. At each step a device passes this strip on to its neighbor, and this happens for $D - 1$ hops (each shard must reach every device except the one it starts on). Therefore the total number of scalars transferred across a single ICI link is

$$\frac{N^2(D-1)}{D}.$$

Part 2 (unidirectional AllToAll). At the first step each device transfers a block of size $(\frac{N}{D}, N - \frac{N}{D})$, then $(\frac{N}{D}, N - \frac{2N}{D}), \dots, (\frac{N}{D}, N - \frac{D-1}{D}N)$, since progressively less data needs to be forwarded. Simplifying the sum of the widths:

$$\begin{aligned} \sum_{i=1}^D \left(N - \frac{iN}{D} \right) &= DN - \frac{N}{D} \sum_{i=1}^D i \\ &= DN - \frac{N}{D} \cdot \frac{D(D+1)}{2} \\ &= DN - \frac{N(D+1)}{2} \\ &= N \left(\frac{2D - D - 1}{2} \right) = N \left(\frac{D-1}{2} \right). \end{aligned}$$

So the total number of scalars is

$$\text{total} = \frac{N}{D} \cdot \frac{N(D-1)}{2} = \boxed{\frac{N^2(D-1)}{2D}} \text{ scalars.}$$

Part 3. The ratio between the answers to part 1 and part 2 is 2. The idea is that rather than transferring a constant C scalars each hop, we transfer C , then $C-1, \dots, 1$, and $1+\dots+C = \frac{C(C+1)}{2}$. That is where the factor of 2 comes from.

Part 4 (bidirectional AllGather). We send the same number of scalars, $\frac{N^2}{D}$, across each link every hop, but each device sends across 2 links simultaneously, so we do this for half the hops. Thus each link sees $\frac{1}{2}$ the data, and the total time gets cut in half.

Part 5 (bidirectional AllToAll). We send $\sim \frac{D}{2}$ chunks in one direction and $\frac{D}{2}$ chunks in the other direction on the first step, then $\frac{D}{2} - 2$ in each direction, etc. So rather than $\sum_{i=1}^D i$ we have

$$\frac{\sum_{i=1}^{D/2} i}{2} = \frac{\frac{D}{2} \left(\frac{D}{2} + 1 \right)}{2} = \frac{D \left(\frac{D}{2} + 1 \right)}{4}.$$

Each chunk is $\frac{N^2}{D}$, so we have

$$\frac{N^2}{D} \cdot \frac{D \left(\frac{D}{2} + 1 \right)}{4} = \frac{\sum_{i=1}^D i}{4}.$$

Thus each link only sends $\frac{1}{4}$ of the data compared to the unidirectional case.

Part 6. In the unidirectional case, AllToAll is $2\times$ as fast. In the bidirectional case, AllGather gets a $2\times$ speedup and AllToAll gets a $4\times$ speedup. So AllToAll gets an additional $2\times$ relative speedup compared to AllGather when moving to bidirectional. Thus we get the factor of 4.

4 Chapter 4: All the Transformer Math You Need to Know

Question 1

How many parameters does a model with $D = 4096$, $F = 4 \cdot D$, $V = 32,000$, and $L = 64$ have? What fraction of these are attention parameters? How large are our KV caches per token? *You can assume $N \cdot H = D$ and multi-head attention with int8 KV's.*

Solution. Part (a) – total parameter count. Using the per-layer counts (per the book):

$$\text{Attention: } 2D(N + K)H, \quad \text{FFN: } 3DF,$$

plus the embedding/unembedding and layer-norm terms $2DV + D$.

With multi-head attention $N = K$ and $NH = D$, the attention term becomes

$$2D(2N)H = 4DNH = 4D^2.$$

So the total is

$$(4D^2 + 3DF + D)L + 2DV.$$

Plugging in $D = 4096$, $F = 4D$, $L = 64$, $V = 32,000$:

$$\approx 1.7 \times 10^{10} \approx \boxed{17\text{B parameters}}.$$

Part (b) – fraction that are attention parameters:

$$\frac{4D^2 L}{17B} \approx 0.25 \approx \boxed{\frac{1}{4}}.$$

Part (c) – KV cache per token (int8, so 1 byte each):

$$2 L K H = 2 N L H = 2 \cdot 64 \cdot 4096 \approx \boxed{524 \text{ kB/token}}.$$

Question 2

How many total FLOPs are required to perform $A[BX, DY] * D W[DY, F]$ on $\{'X': 4, 'Y': 8, 'Z': 4\}$. How many FLOPs are performed by each TPU?

Solution. We compute $A[B_X, D_Y] * D W[D_Y, F]$ on the mesh $\{X=4, Y=8, Z=4\}$.

Per-device FLOPs:

$$2 \cdot \frac{B}{X} \cdot \frac{D}{Y} \cdot F = \frac{2BDF}{32} = \frac{BDF}{16}.$$

Required (theoretical) FLOPs, summing over the sharded X, Y dimensions:

$$\frac{BDF}{16} \cdot X \cdot Y = 2BDF.$$

Actual FLOPs, since the work is replicated across Z :

$$\frac{BDF}{16} \cdot X \cdot Y \cdot Z = 2BDF \cdot Z = \boxed{8BDF}.$$

Question 3

How many FLOPs are involved in performing $A[I, J, K, L] * B[I, J, M, N, O] \rightarrow C[K, L, M, N, O]$?

Solution. Here I, J are contracting dimensions and K, L, M, N, O are non-contracting, with no batching axes, so

$$\boxed{2 \cdot I \cdot J \cdot K \cdot L \cdot M \cdot N \cdot O} \text{ FLOPs.}$$

Question 4

What is the arithmetic intensity of self-attention (ignoring the Q/K/V/O projections)? *Give the answer as a function of the Q and KV lengths T and S.* At what context length is attention FLOPs-bound? Given the HBM bandwidth of our TPUs, plot the effective relative cost of attention to the FFW block as the context length grows.

Solution. Self-attention operations (ignoring Q/K/V/O projections):

$$Q[B, T, K, G, H] \cdot K[B, S, K, H] \rightarrow O[B, T, S, K, G],$$

$$U[B, T, S, K, G] \cdot V[B, S, K, H] \rightarrow X[B, T, K, G, H].$$

FLOPs:

$$2BTSKGH + 2BTSKGH = 4BTSKGH.$$

Comms (bf16 bytes):

$$2BTKGH + 2BSKH + 2BSKH + 2BTKGH = 4BTKGH + 4BSKH.$$

Arithmetic intensity:

$$\frac{4BTSKGH}{4BTKGH + 4BSKH} = \frac{TSKGH}{TKGH + SKH} = \boxed{\frac{TSG}{TG + S}}.$$

Prefill ($T = S$):

$$\frac{T^2G}{TG + T} = \frac{TG}{G + 1} \approx T \quad (\text{if } G = 1 \Rightarrow \frac{T}{2}).$$

So during prefill attention is compute-bound at

$$\boxed{T \approx 240 \quad (480 \text{ when } G = 1)}.$$

Decode ($T = 1$):

$$\text{intensity} = \frac{SG}{G + S} \approx G \quad (\text{since } S \gg G),$$

so we are virtually always memory-bandwidth bound since $G \ll 240$.

For the relative-cost plot, the FFW block:

$$B[T, D] \cdot [D, F] (\times 2), \quad B[T, F] \cdot [F, D] \text{ (ignoring the pointwise multiply).}$$

FLOPs:

$$(2BTDF) \cdot 2 + 2BTDF = 6BTDF.$$

Comms:

$$2BTD + 4DF + 2DF + 2BTD = 4BTD + 6DF.$$

Intensity:

$$\frac{6BTD}{4BTD + 6DF} = \boxed{\frac{6BT}{4BT + 6F}}.$$

The following script plots the roofline latency model for attention and the FFW block across sequence/context length; the result is shown in Figure 2.

```
import numpy as np
import matplotlib.pyplot as plt

D = 4096
N = 32
H = D // N
F = 4*D

PEAK_FLOPS = 1.97e14
HBM = 8.2e11
B = 1
G = 1

RIDGE = PEAK_FLOPS / HBM

def time_for(flops, intensity):
    achievable = np.minimum(PEAK_FLOPS, intensity * HBM)
    return flops / achievable

def attention_prefill(T):
    # S = T
    intensity = T / 2
    flops = 4 * B * T * T * N * H
    return time_for(flops, intensity)

def ffn_prefill(T):
    intensity = (6 * B * T * F) / (4 * B * T + 6 * F)
    flops = 6 * B * T * D * F
    return time_for(flops, intensity)

def attention_decode(S):
    # T = 1
    intensity = (S * G) / (S + G)
    flops = 4 * B * S * N * H
    return time_for(flops, intensity)

def ffn_decode(S):
    # T = 1 and independent of context length S
    intensity = (6 * B * F) / (4 * B + 6 * F)
    flops = 6 * B * D * F
    return time_for(flops, intensity) * np.ones_like(S)
```

```

def main():
    S = np.logspace(0, np.log10(32768), 400)

    fig, (ax_prefill, ax_decode) = plt.subplots(1, 2, figsize=(14, 5.5), sharey=True)

    ax_prefill.plot(S, attention_prefill(S), label='attention', linewidth=2.5)
    ax_prefill.plot(S, ffn_prefill(S), label='ffn', linewidth=2.5)
    ax_prefill.set_title('Prefill (S = T)')
    ax_prefill.set_xlabel('sequence length T')
    ax_prefill.set_ylabel('time (s)')

    attn_cross = 2 * RIDGE # I = T/2 = ridge
    ffn_cross = 6 * F * RIDGE / (6 * F - 4 * RIDGE) # 6TF/(4T+6F) = ridge
    ax_prefill.axvline(attn_cross, color='C0', linestyle=':', alpha=0.6,
                      label=f'attention crossover (T={attn_cross:.0f})')
    ax_prefill.axvline(ffn_cross, color='C1', linestyle=':', alpha=0.6,
                      label=f'ffn crossover (T={ffn_cross:.0f})')

    ax_decode.plot(S, attention_decode(S), label='attention', linewidth=2.5)
    ax_decode.plot(S, ffn_decode(S), label='ffn', linewidth=2.5)
    ax_decode.set_title('Decode (T = 1) - both memory-bound, no crossover')
    ax_decode.set_xlabel('context length S')

    for ax in (ax_prefill, ax_decode):
        ax.set_xscale('log')
        ax.set_yscale('log')
        ax.grid(True, which='both', linestyle='--', alpha=0.4)
        ax.legend()

    fig.suptitle('Roofline latency model vs. sequence/context length')
    fig.tight_layout()
    plt.show()

if __name__ == '__main__':
    main()

```

Question 5

At what sequence length are self-attention FLOPs equal to the QKVO projection FLOPs?

Solution. Self-attention FLOPs (from Q4): $4BTSKGH$.

Projection FLOPs:

$$Q : B[T, D] \cdot [D, NH] \rightarrow 2BTDNH, \quad K : B[S, D] \cdot [D, KH] \rightarrow 2BSDKH,$$

$$V : 2BSDKH, \quad O : B[T, NH] \cdot [NH, D] \rightarrow 2BTDNH.$$

For prefill/training take $T = S$, and assume $K = N$ (MHA). Then

$$\text{self-attention: } 4BT^2NH,$$

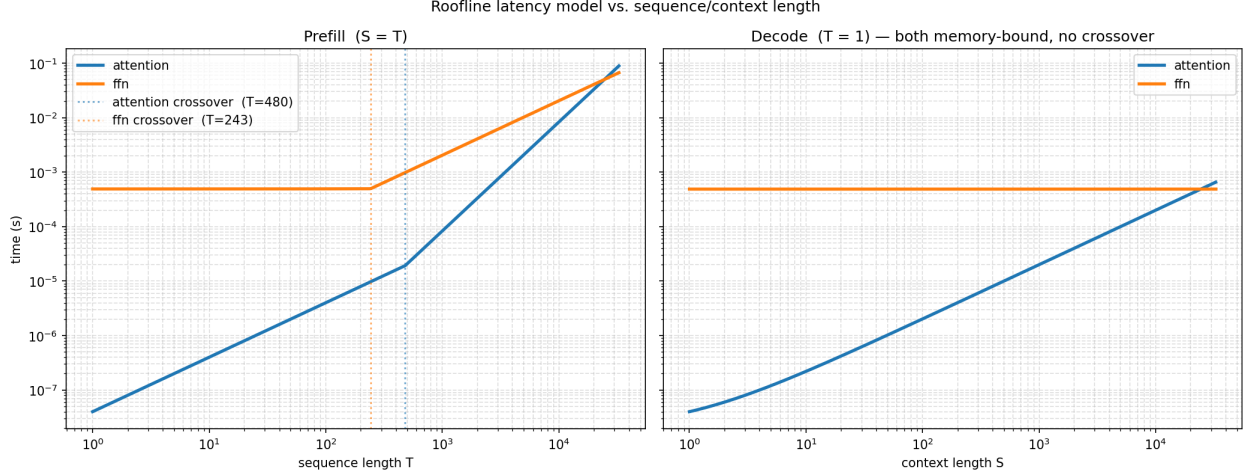


Figure 2: Roofline latency model: prefill ($S = T$) with per-curve memory→compute crossovers, and decode ($T = 1$), where both attention and the FFW block are memory-bound.

$$\text{QKVO: } 4BTD(N + K)H = 8BTDNH.$$

Setting them equal:

$$4BT^2NH = 8BTDNH \Rightarrow \boxed{T = 2D}.$$

Question 6

Say we only save the output of each of the 7 main matmuls in a Transformer layer during our forward pass (Q, K, V, O + the three FFW matrices). How many extra FLOPs do we need to “rematerialize” during the backwards pass?

Solution. The main rematerialization is the attention matrices. From Q4 the attention recompute costs

$$4BTSKGH = 4BTSNH \text{ FLOPs,}$$

plus $O(BTD)$ FLOPs for the layer-norm recompute (and the second LN).

For training $T = S$, so this is

$$\boxed{4BT^2NH}.$$

Question 7

DeepSeek v3 says it was trained for 2.79M H800 hours on 14.8T tokens (source). Given that it has 37B activated parameters, roughly what hardware utilization did they achieve? *Hint: note that they used FP8 FLOPs without structured sparsity.*

Solution. H800 peak: 1979e12 FLOPs/s ($\approx 1.98\text{e}15$), and $\approx 1.5\text{e}15$ FLOPs/s without structured sparsity.

Tokens: $14.8T = 14.8\text{e}12$.

Achieved FLOPs/s = total FLOPs / total time, with total FLOPs $\approx 6ND$:

$$\frac{6ND}{\text{total time}} = \frac{6 \cdot 37\text{e}9 \cdot 14.8\text{e}12}{2.79\text{e}6 \cdot 3600} = 3.2712\text{e}14 \text{ FLOPs/s.}$$

Utilization:

$$\frac{3.2712\text{e}14}{1.5\text{e}15} \approx \boxed{21.8\%},$$

(using the with-sparsity peak 1.979e15 instead gives $\approx 16.5\%$).

Question 8

Mixture of Experts (MoE) models have E copies of a standard dense MLP block, and each token activates k of these experts. What batch size in tokens is required to be compute-bound for an MoE with weights in int8 on TPU v5e? For DeepSeek, which has 256 (routed) experts and $k = 8$, what is this number?

Solution. An MoE model has E copies of a standard dense MLP block, and each token activates k of them.

FLOPs. Each MLP block (three matmuls) costs $6BTDF$ FLOPs, so with k activated experts the total is

$$\text{FLOPs} = 6kBTDF.$$

Bytes loaded (int8 weights, E copies plus activations):

$$\text{Comms} = 4BTD + 6DFE.$$

Arithmetic intensity:

$$\text{Intensity} = \frac{6kBTDF}{4BTD + 6DFE} = \frac{6kBT F}{2BT + 3FE}.$$

Assuming $FE \gg BT$,

$$\text{Intensity} \approx \frac{6kBT F}{3FE} = \frac{2kBT}{E}.$$

To be compute-bound we set this equal to the TPU v5e intensity of 240 FLOPs/byte:

$$\frac{2kBT}{E} = 240.$$

At decode time $T = 1$, so

$$\frac{2kB}{E} = 240.$$

For DeepSeek v3 with $E = 256$ and $k = 8$:

$$\frac{2 \cdot 8 \cdot B}{256} = 240 \implies \frac{16B}{256} = 240 \implies \boxed{B = 3840}.$$

This is a remarkably large batch size to reach at generation time.

5 Chapter 5: How to Parallelize a Transformer for Training

Question 1

How many parameters does LLaMA-2 13B have (I know that’s silly but do the math)? *Note that, as in Transformer Math, LLaMA-3 has 3 big FFW matrices, two up-projection and one down-projection. We ignored the two “gating” einsum matrices in this section, but they behave the same as Win in this section.*

Solution. Counting parameters per component:

$$\begin{aligned}\text{attention: } & 2D(N+k)H \\ \text{FFN: } & 3DF \\ \text{LN: } & 2D \\ \text{embed/unembed: } & 2DV\end{aligned}$$

So the total parameter count is

$$\text{params} = L(3DF + 2D(N+k)H + 2D) + 2DV.$$

Evaluating numerically,

$\text{params} \approx 13\text{e}9 = 13\text{B}$

Question 2

Let’s assume we’re training with BS=16M tokens and using Adam. Ignoring parallelism for a moment, how much total memory is used by the model’s parameters, optimizer state, and activations? *Assume we store the parameters in bf16 and the optimizer state in fp32 and checkpoint activations three times per layer (after the three big matmuls).*

Solution. Parameters and optimizer state (Adam): storing the parameters in bf16 (2 bytes) together with the two fp32 optimizer moments (the first and second moment accumulators, 4 bytes each) gives

$$(2 + 4 + 4) \cdot 13\text{B} = 10 \cdot 13\text{B} = 130\text{GB}.$$

Activations (checkpointed after the three big matmuls; two of shape BF and one of shape BD per layer, in bf16):

$$\begin{aligned}\text{activations} &= \left(\frac{BTF \cdot 2}{2} + \frac{BTD}{2} \right) \cdot 2 \cdot L = 2(BTF \cdot 2 + BTD)L \\ &= 2 \cdot BT(2F + D)L, \quad BT = 16\text{M} \\ &= 2 \cdot 16\text{M}(2F + D)L \\ &= 4.19\text{e}13 = 41.9\text{e}12 = 41.9\text{T (in activations)}.\end{aligned}$$

$\text{params} + \text{opt state} = 130\text{GB}, \quad \text{activations} \approx 41.9\text{TB}$

Question 3

Assume we want to train with 32k sequence length and a total batch size of 3M tokens on a TPUv5p 16x16x16 slice. Assume we want to use bfloat16 weights and a float32 optimizer, as above.

1. Can we use pure data parallelism? Why or why not?

2. Can we use pure FSDP? Why or why not? With pure FSDP, how much memory will be used per device (assume we do gradient checkpointing only after the 3 big FFW matrices).
3. Can we use mixed FSDP + tensor parallelism? Why or why not? If so, what should X and Y be? How much memory will be stored per device? Using only roofline FLOPs estimates and ignoring attention, how long will each training step take at 40% MFU?

Solution. Setup: $\text{In}[B_x, D] \cdot W_{in}[D, F] \cdot W_{out}[F, D_x] \rightarrow \text{out}[B_x, D]$.

Part 1 (pure data parallelism): We can't use pure DP. From Q2, the model params plus optimizer state is $\sim 130\text{GB}$, which won't fit on a single TPU.

Part 2 (pure FSDP): With FSDP we split both params and activations evenly over $16 \times 16 \times 16 = 4096$ devices.

$$\begin{aligned} \text{per device} &= \frac{42\text{T}}{4096} \approx 1.03\text{e}10 \approx 10\text{e}9 \text{ activations} \quad (\text{old BS}) \\ \text{activations} &= \frac{2.3\text{M}(2F + D)L}{4096} \approx 1.92\text{e}9 = 1.9\text{GB} \\ \text{params} &= \frac{130\text{GB}}{4096} = 32\text{MB} \end{aligned}$$

So from a memory standpoint we can use FSDP. Now check comms vs. compute bound. We are compute bound when

$$\frac{B}{X} > \frac{C}{w \cdot 3} \Rightarrow \frac{B}{X} \gtrsim 850 \Rightarrow B \geq 4096 \cdot 850 \Rightarrow B \geq 3.4\text{M}.$$

Thus we are comms bound, since $B = 3\text{M} < 3.4\text{M}$. So it's tough to do FSDP alone: we can't do it efficiently.

Part 3 (mixed FSDP + tensor parallelism): We need $\frac{B}{N} > \frac{\alpha^2}{M_X M_Y F}$ to be compute bound. With $\alpha = 2550$, $M_X M_Y = 2$, $N = 4096$, $F = 13824$:

$$B > 963333 \approx 1\text{M} < 3\text{M}.$$

Thus we can be compute bound, so we can use FSDP + TP. For the split,

$$\begin{aligned} X_{opt} &= \sqrt{\frac{B}{F} \cdot \frac{M_X}{M_Y} \cdot N}, \quad \frac{M_X}{M_Y} = 2, \\ \Rightarrow X_{opt} &\approx 1333.3 \quad \left(\text{could have } \frac{M_X}{M_Y} = 0.5 \right). \end{aligned}$$

So roughly FSDP over 1024 and TP over 4. Memory per device is

$$1.9\text{GB} + 32\text{MB} \approx 2\text{GB} \quad (\text{from step 2}).$$

Step time: each device does $6ND$ FLOPs with $D = 3\text{M}$, $N = 13\text{B}$, and each device does $\frac{1}{4096}$ of that:

$$\frac{6 \cdot 3\text{M} \cdot 13\text{B}}{4096} = 5.7\text{e}13 \text{ FLOPs}.$$

With TPU v5p peak $4.59\text{e}14$ FLOP/s at 40% MFU:

$$\frac{5.7\text{e}13}{4.59\text{e}14} \cdot \frac{1}{0.4} \approx \boxed{310\text{ms} / \text{training step}}.$$

6 Chapter 6: Training LLaMA 3 on TPUs

Question

How many FLOPs does LLaMA-3 perform per token per training step? *This helps us determine how expensive the whole training process will be.*

Solution. We do roughly $6 \cdot (\text{param count})$ FLOPs per token per training step, so

$$6 \cdot N = 6 \cdot 70\text{B} = \boxed{420\text{B FLOPs / token}}$$

That is about half a TFLOP per token per step.

Question

LLaMA 3 was trained for about 15 trillion tokens. How many FLOPs is that total?

Solution. Training for 15T tokens gives a total of

$$420\text{B} \cdot 15\text{T} = 4.2e11 \cdot 15e12 = 4.2 \cdot 15 e23 = 60e23 \approx \boxed{6e24 \text{ FLOPs}}$$

Question

Let's say we wanted to train on a full TPU v5p pod with $16 \times 20 \times 28 = 8960$ chips. How long would this take to train at 40% MFU in bfloat16, assuming we are compute-bound?

Solution. Each TPU v5p performs $4.59e14$ FLOPs/s. Training on the full pod of 8960 chips at 40% MFU:

$$T = \frac{6.3e24}{0.4 \cdot 4.59e14 \cdot 8960} \approx 3.8\text{M seconds} \approx \boxed{44 \text{ days}}$$

Question

LLaMA 3-70B was pretrained with a batch size of about 4M tokens. How many TPUs do we need at minimum to train with this batch size? *You can assume bfloat16 parameters and float32 optimizer state, and that you checkpoint gradients 4 times per layer.*

Solution. With batch size $BS = 4\text{M}$ tokens, the only binding constraint is memory.

Parameters + optimizer state (bf16 params, fp32 optimizer state):

$$(2 + 8) \cdot 70\text{B} = 10 \cdot 70\text{B} = 700\text{GB}$$

Activations (gradient checkpoints): each activation is $\sim B \cdot D$, with 4 checkpoints per layer, 80 layers, and $\times 2$ for bf16:

$$4 \cdot 4\text{M} \cdot 8192 \cdot 80 \cdot 2 \approx 2.1e13 = 21e12 = 21\text{TB}$$

So the total is about 21.7TB. Each TPU v5p has 96GB of HBM, so

$$\min \text{TPUs} = \frac{21.7\text{TB}}{96\text{GB}} = 225.75 \Rightarrow \boxed{226 \text{ TPUs}}$$

Question

Under the same assumptions as the question above, if we use 8960 TPU v5p chips, how much memory will we use per-chip?

Solution. Total memory is still about 21.7TB, so on 8960 chips:

$$\frac{21.7\text{TB}}{8960} = 2.42e9 \approx \boxed{2.4 \text{ GB / chip}}$$

which is basically nothing.

Question

Under the assumptions above, can we train our model with FSDP alone? To start, let's say we can't do any sequence/context parallelism. *This should be the first idea you have, since it's simple and will introduce no extra communication if it works.*

Solution. Technically no, because we only have 1024 sequences, so a maximum of 1024 shards for pure data parallelism / FSDP over sequences. We could do DP over the rest, but we would need to check whether we'd be comms-bound doing so.

Question

Let's relax the requirement of not doing any sequence sharding. If we allow ourselves to do FSDP over both the batch *and* sequence axes, can we train LLaMA 3-70B with only FSDP on 8960 chips?

Solution. Doing 8960-way FSDP over batch and sequence:

$$\frac{4\text{M}}{8960 \text{ chips}} \approx 446 \text{ tokens / chip}$$

Memory fits (as we saw in Q5). But are we comms-bound? Using all 3 axes of the pod, we need the per-device batch to satisfy $B/X > 850$. Since $446 < 850$,

yes, we are comms-bound.

Question

Now let's look at mixed tensor parallelism and FSDP. Does there exist some combination that lets us remain compute-bound? What amount of FSDP and tensor parallelism should we do if so?

Solution. We are comms-bound if the per-chip batch is below $\frac{2550^2}{2F} = 113$. Checking:

$$450 > \frac{u^2}{2F} = 113 \checkmark$$

Since the per-chip batch $\approx 450 > 113$, we are back in the compute-bound regime. The optimal amount of FSDP:

$$X_{opt} = \sqrt{\frac{B}{P}} \cdot 2N = 1597.45$$

Rounding to reasonable powers of 2:

1024 FSDP, 8 TP or 2048 FSDP, 4 TP

Question 1 [Scaling LLaMA 70B to more chips]

say we want to train LLaMA 3-70B on 4 pods with the same batch size. What parallelism scheme would we use? Would we be compute or communication bound? Roughly how long would it take to train? *Make sure to use the correct roofline bound.*

Solution. We have 4 pods. Say we do DP across pods. Then the per-device batch size is

$$\frac{450}{4} \approx 112.5 \approx 113 \text{ tokens / device,}$$

so we are just about compute-bound within a pod. Across pods we also need to check. The DCN threshold is 73,440, so we need

$$\frac{B}{4} > 73,440 \Rightarrow B \gtrsim 292\text{k} \quad (B = \text{per-pod batch size}).$$

Since each pod is doing $4M/4 \approx 1M$ tokens, we are still compute-bound.

Thus we can compute training time as total FLOPs divided by FLOPs/s across all devices, then divide by the 40% MFU. From Q2 the total is $6.3e24$ FLOPs, and

$$\text{FLOPs/s} = 4.59e14 \cdot 8960 \cdot 4 \text{ pods.}$$

So

$$\text{training time} = \frac{6.3e24}{4.59e14 \cdot 8960 \cdot 4 \cdot 0.4} = 957\text{k seconds} \approx \boxed{11 \text{ days to train}}$$

(We can use the same parallelism scheme as in Q8.)

Question 2 [LLaMA 405B]

- (a) Using the LLaMA 3-405B config, write a table with all the key hyperparameters as above. How many total parameters does this model have? How many FLOPs per training step? How many FLOPs do we perform if we train for 15T tokens?
- (b) Assume we want to train on 8 TPU v5p pods. What parallelism scheme would we use? How long would training take? Would we be compute or comms bound?

Solution. Part (a). Reading the key hyperparameters for the LLaMA 3-405B config:

L (layers)	126
D (d_{model})	16384
F (d_{ff})	53248
N (n_{heads})	128
K (n_{kv})	8
$H = D/N$	128
V (vocab)	128256

This gives a total of roughly 405B parameters, so the per-token cost is about $6 \cdot 405e9$ FLOPs, and training over 15T tokens is

$$6 \cdot N \cdot D = 6 \cdot 405e9 \cdot 15e12 \approx 3.6e25 \text{ FLOPs.}$$

Part (b). We want to train on 8 TPU v5p pods, i.e. $8 \times 8960 = 71680$ chips, at the same $\sim 4M$ token batch.

Memory: this is still fine within a pod – even scaling everything up by $10\times$ we would only be at $\sim 20\text{GB}/\text{device}$.

Per-device batch: 8 pods gives roughly $\frac{113}{2} \approx 56$ tokens per device. The minimum per-device batch to stay compute-bound under FSDP is

$$\frac{\alpha^2}{2F} = \frac{2550^2}{2 \cdot 53248} \approx 61.05.$$

Since $56 < 61$, we are comms-bound with pure FSDP, and there is no way to avoid this. So we must use mixed FSDP + tensor parallelism.

To pick the split we use

$$X_{\text{opt}} = \sqrt{\frac{2BN}{F}}.$$

Evaluating gives $X_{\text{opt}} \approx 420$, which is why we round to sensible powers of two and take the chosen scheme

$$\boxed{512\text{-way FSDP, } 16\text{-way TP}}$$

(with 4096 FSDP / 2 TP as an alternative).

Time: within a pod we are comms-bound, but between pods we are still compute-bound, since we have $\sim 500\text{k}$ tokens per pod $> 292\text{k}$ required. Writing the ratio with $z = \sqrt{B/N}$,

$$\frac{T_{\text{math}}}{T_{\text{comms}}} = z \cdot c.$$

Here $z < z_{\text{min}}$, so $T_{\text{comms}} > T_{\text{math}}$; in particular

$$T_{\text{comms}} = \frac{T_{\text{math}}}{z \cdot c} \approx 1.04 T_{\text{math}}.$$

The math (compute) time is

$$T_{\text{math}} = \frac{6ND}{8960 \cdot 8 \cdot C \cdot 0.4} = \frac{6 \cdot 405\text{e}9 \cdot 15\text{e}12}{71680 \cdot 4.59\text{e}14 \cdot 0.4} \approx 2.7\text{e}6 \text{ s},$$

so training takes about

$$\boxed{\approx 32 \text{ days}}$$

7 Chapter 7: All About Transformer Inference

Pop Quiz

Assume we want to take a generate step with a batch size of 4 tokens from a 30B parameter dense model on TPU v5e 4x4 slice in int8 with bf16 FLOPs, 8192 context and 100 kB / token KV caches. What is a reasonable lower bound on the latency of this operation? What if we wanted to sample a batch of 256 tokens?

Solution. We estimate a lower bound on the generate-step latency. Given data: parameters use 30 GB (int8), batch size 4, KV cache 100 kB/token, and (taking roughly halfway through the 8192 context, ≈ 4096 tokens) each KV cache is $100 \text{ kB} \cdot 4096 \approx 400 \text{ MB}$. We have 16 chips, each with HBM bandwidth $8.2\text{e}11 \text{ B/s}$ and $1.97\text{e}14 \text{ bf16 FLOPs/s}$.

Small batch (memory bound). The batch's KV caches use $4 \cdot 400 \text{ MB} = 1.6 \text{ GB}$, so

$$t \approx \frac{30 \text{ GB} + 1.6 \text{ GB}}{8.2\text{e}11 \cdot 16} \approx \boxed{2.4 \text{ ms}}.$$

Batch of 256 (compute bound for the MLP blocks). Here we add the memory time for the KV caches and the compute time:

$$\begin{aligned} t_{\text{mem}} &= \frac{256 \cdot 100 \text{ kB} \cdot 4096}{8.2\text{e}11 \cdot 16} \approx 8 \text{ ms}, \\ t_{\text{comp}} &= \frac{2ND}{\cdot} = \frac{2 \cdot 30 \text{ GB} \cdot 256}{1.97\text{e}14 \cdot 16} \approx 5 \text{ ms}. \\ t &\approx 8 \text{ ms} + 5 \text{ ms} = \boxed{13 \text{ ms}}. \end{aligned}$$

Question 1

How many parameters does the above model have? How large are its KV caches per token in int8? *You can assume we share the input and output projection matrices.*

Solution. Total parameter count (sharing input/output projections):

$$(2D(N + K)H + 3DF)L + DV = \boxed{18.4\text{B params}}.$$

KV cache per token in int8:

$$2 \cdot K \cdot H \cdot L = 262144 = \boxed{262 \text{ kB / token}}.$$

Question 2

Say we want to serve this model on a TPUv5e 4x4 slice and can fully shard our KV cache over this topology. What's the largest batch size we can fit, assuming we use int8 for everything and want to support 128k sequences? What if we dropped the number of KV heads to 1?

Solution. Each device has 16 GB of HBM, so total memory is

$$16 \cdot 16 = 256 \text{ GB}.$$

The KV cache is $128\text{k} \cdot 262 \text{ kB} = 33.5 \text{ GB}$ per sequence. Subtracting the 18.4 GB of parameters, the largest batch size is

$$\text{largest bs} = \frac{256 - 18.4}{33.5} = 7.07 \Rightarrow \boxed{\text{bs} = 7}.$$

With a single KV head ($K = 1$): $2 \cdot K \cdot H \cdot L = 2 \cdot H \cdot L = 32 \text{ kB}$ / token, so

$$128k \cdot 32768 = 4.19 \text{ GB} / \text{seq},$$

$$\frac{256 - 18.4}{4.19} \approx 56 \Rightarrow \boxed{\text{largest bs} = 56}.$$

Question 3

How long does it take to load all the parameters into the MXU from HBM assuming they're fully sharded on a TPU v5e 4x4 slice? Assume int8 parameters. *This is a good lower bound on the per-step latency.*

Solution. With v5e HBM bandwidth $8.2e11 \text{ B/s}$ per chip and total parameters $18.4e9$ bytes (int8) fully sharded over 16 chips:

$$\frac{18.4e9}{16 \cdot 8.2e11} = 1.402 \text{ ms} \approx \boxed{1.4 \text{ ms}}.$$

Question 4

Let's say we want to serve this model on a TPUv5e 4x4 slice using int8 FLOPs and parameters/activations. How would we shard it for both prefill and decode? *Hint: maybe answer these questions first:*

1. What does ICI look like on a 4x4?
2. What's the roofline bound on tensor parallelism?
3. How can we shard the KV caches?

For this sharding, what is the rough per-step latency for generation?

Solution. ICI on a 4x4: a 4×4 2D mesh with no wraparound, two dimensions each of size 4.

Tensor-parallel roofline bound (prefill): $Y \cdot F/2200$, but with no wraparound it is closer to $F/2200$. Since

$$F/2200 = 16384/2200 \approx 7,$$

a feasible setup is 4-way TP with 4-way sequence sharding for prefill.

Generation (ICI bound): we need

$$Y > \frac{F}{4B} = \frac{2048}{B}.$$

For $B < 128$, $Y > 16$, so we can do 16-way TP; then we shard the KV cache over the head dimension (x) and the batch dimension (y).

Take 16-way TP with the KV cache sharded over head (x) and batch (y), and use the latency equation

$$\frac{B \cdot 262k \cdot 128k}{16 \cdot 8.2e11} + \max\left(\frac{2B \cdot 18.4e9}{16 \cdot 3.97e14}, 1.4 \text{ ms}\right).$$

For $B = 1$:

$$\frac{262k \cdot 128k}{16 \cdot 8.2e11} + \max\left(\frac{2 \cdot 128k \cdot 18.4e9}{16 \cdot 3.97e14}, 1.4 \text{ ms}\right),$$

where the compute term is $< 1.4 \text{ ms}$ at batch size 1 and the first term is 2.556 ms . Hence

$$\text{theoretical step time} \approx \boxed{4 \text{ ms}}.$$

Question 5

Let's pretend the above model is actually an MoE. An MoE model is effectively a dense model with E copies of the FFW block. Each token passes through k of the FFW blocks and these k are averaged to produce the output. Let's use $E=16$ and $k=2$ with the above settings.

1. How many total and activated parameters does it have? *Activated means used by any given token.*
2. What batch size is needed to become FLOPs bound on TPU v5e?
3. How large are its KV caches per token?
4. How many FLOPs are involved in a forward pass with T tokens?

Solution. Treating the model as an MoE with $E = 16$, $k = 2$.

1. Activated and total parameters:

$$\begin{aligned}\text{active} &= (2D(N + K)H + 3kDF)L + DV = 3.13\text{e}10 = \boxed{31.3\text{B params}}, \\ \text{total} &= (2D(N + K)H + 3EDF)L + DV = 2.12\text{e}11 = \boxed{212\text{B params}}.\end{aligned}$$

2. The FFN block loads $E/k = 8\times$ more parameters than it uses, so the FLOPs-bound batch size is

$$240 \cdot 8 = \boxed{1920 \text{ tokens}}.$$

3. KV cache per token: same as before, 262 kB / token.
4. FLOPs in a forward pass over T tokens:

$$\boxed{2 \cdot N_{\text{active}} \cdot T}.$$

Question 6

With MoEs, we can do “expert sharding”, where we split our experts across one axis of our mesh. In our standard notation, our first FFW weight has shape $[E, D, F]$ and we shard it as $[EZ, DX, FY]$ where X is only used during training as our FSDP dimension. Let's say we want to do inference on a TPU v5e:

1. What's the HBM weight loading time for the above model on a TPU v5e 8x16 slice with $Y=8$, $Z=16$? How much free HBM is available per TPU?
2. What is the smallest slice we could fit our model on?

Solution.

Q6 Total parameters = 212B. With the sharding $[E_Z, D_X, F_Y]$ and $Y = 8$, $Z = 16$ ($X = 1$ during inference), each device holds

$$\frac{E}{16} \cdot D \cdot \frac{F}{8}$$

of the FFN weights. Out of the 212B total, the FFN weights are

$$3 \cdot E \cdot D \cdot F \cdot L = 2.06\text{e}11 = 206\text{B},$$

so to approximate we only consider the FFN weights. Each device loads

$$\frac{3EDFL}{128} \approx 1.61 \text{ GB},$$

and the loading time is

$$\frac{1.61\text{e}9}{8.2\text{e}11} = 1.96 \text{ ms} \approx \boxed{2 \text{ ms}}.$$

This leaves about $\boxed{14 \text{ GB}}$ of free HBM per device.

(smallest slice, cont. of Q6) Smallest slice we could fit the model on. The MoE has about 212B parameters, i.e. 212e9 bytes in int8, and each TPU v5e chip has 16GB of HBM:

$$\frac{212\text{B}}{16\text{GB}} \approx 13.25.$$

Thus 14 devices could fit the model, so

$$\boxed{\text{a } 4 \times 4 \text{ slice could serve this model.}}$$

Question 7 [2D model sharding]

Here we'll work through the math of what the ESTI paper calls 2D weight-stationary sharding. We describe this briefly in Appendix B, but try doing this problem first to see if you can work out the math. The basic idea of 2D weight stationary sharding is to shard our weights along both the D and F axes so that each chunk is roughly square. This reduces the comms load and allows us to scale slightly farther.

Here's the algorithm for 2D weight stationary:

Solution. We work out T_{math} and T_{comms} for 2D weight-stationary sharding.

Since all FLOPs are fully sharded,

$$T_{\text{math}} = \frac{2BDF}{XYZ} + \frac{2BDF}{XYZ} = \frac{4BDF}{XYZ}.$$

For communications, accounting for the number of axes over which each collective runs (and that the AllReduce is twice as expensive), with $W = W_{\text{ici}}$:

$$\begin{aligned} \text{AG: } \frac{2BD}{2X \cdot W} &= \frac{BD}{XW}, \\ \text{AR: } \frac{2BF \cdot 2}{YZ \cdot W} &= \frac{4BF}{YZW}, \\ \text{RS: } \frac{2BD}{2X \cdot W} &= \frac{BD}{XW}. \end{aligned}$$

Adding these,

$$T_{\text{comms}} = \frac{2BD}{XW} + \frac{4BF}{YZW}.$$

To find the optimal sharding we minimize the comms. Let $F = 4D$ and $YZ = N/X$. Then

$$T_{\text{comms}} = \frac{2BD}{XW} + \frac{16BDX}{NW}.$$

We take the argmin over X :

$$\frac{dT_{\text{comms}}}{dX} = -\frac{2BD}{X^2W} + \frac{16BD}{NW} = 0 \implies \frac{2BD}{X^2W} = \frac{16BD}{NW}.$$

Hence

$$\frac{2}{X^2} = \frac{16}{N} \implies X^2 = \frac{N}{8} \implies \boxed{X = \sqrt{\frac{N}{8}}},$$

and

$$YZ = \frac{N}{X} = \frac{N}{\sqrt{N/8}} = \boxed{\sqrt{8N}}.$$

Plugging back into T_{comms} (using $1/X = YZ/N$):

$$\begin{aligned} T_{\text{comms}} &= \frac{2BD \cdot YZ}{NW} + \frac{16BDX}{NW} = \frac{2BD\sqrt{8N}}{NW} + \frac{16\sqrt{N/8}BD}{NW} \\ &= \frac{2BD(\sqrt{8N} + \sqrt{8N})}{NW} = \boxed{\frac{8\sqrt{2}BD}{\sqrt{N}W}}. \end{aligned}$$

For normal (1D) tensor parallelism, the comms are, over 3 axes,

$$T_{\text{TP comms}} = \frac{4BD}{W} \cdot \frac{1}{3} = \frac{4BD}{3W}.$$

The new scheme is better when

$$T_{\text{TP comms}} > T_{\text{2D comms}} \iff \frac{4BD}{3W} > \frac{8\sqrt{2}BD}{\sqrt{N}W} \iff \frac{4}{3} > \frac{8\sqrt{2}}{\sqrt{N}} \iff \sqrt{N} > \frac{8\sqrt{2} \cdot 3}{4}.$$

Squaring,

$$N > \frac{64 \cdot 2 \cdot 9}{16} = 4 \cdot 2 \cdot 9 = \boxed{72}.$$

So when the total number of chips exceeds 72, this new scheme beats 1D tensor parallelism. Note that for inference we only consider comms because we are already comms bound, so we just care about minimizing comms.

8 Chapter 8: Serving LLaMA 3-70B on TPUs

Question

How large are LLaMA 3-70B's KV caches per token? *You can assume we store them in int8. This determines how large our batch size can be on a given topology.*

Solution. The KV cache size per token (stored in int8) is

$$2 \cdot K \cdot H \cdot L = 2 \cdot 8 \cdot 128 \cdot 80 = \boxed{163 \text{ kB/token}}$$

using $K = 8$ KV heads, head dimension $H = 128$, and $L = 80$ layers.

Question

Let's say we want to serve L3 70B at batch size 32 and 8192 sequence length with everything (params and KV) in int8. How much total memory will this use? What's the smallest slice we could serve this on?

Solution. Total params: 70 GB. KV cache/token: 163 kB. Batch size: 32. Sequence length: 8192.

$$70\text{e}9 + 8192 \cdot 163\text{e}3 \cdot 32 = \boxed{112.7 \text{ GB}}$$

This could serve on a 4×2 slice (128 GB HBM) barely, maybe.

Question

At this batch size and quantization on a TPU v5e 4x2, roughly what latency would we expect per decode step? What throughput (tokens / sec / chip). What about a 4x4? *Assume we perform our FLOPs in bfloat16 and everything is fully sharded.*

Solution. Compute bound? The critical batch size = 120 with int8 params and bf16 FLOPs. Since $B = 32 < 120$, latency is memory bound.

$$\begin{aligned} \text{step time} &= \frac{\text{total memory}}{\text{total bw}} = \frac{112.7 \text{ GB}}{8 \cdot 8.2\text{e}11} = 17.18 \text{ ms} \\ \frac{1}{0.01718} &\approx 58.2, \quad 58.2 \cdot 32 = \boxed{1862 \text{ tok/s}} \end{aligned}$$

On a 4×4 : still memory bound, so latency goes down by a factor of 2 and total throughput $\uparrow \times 2$.

For a 4×4 (or 4×2), the ICI/compute roofline is

$$\frac{2 \cdot F}{2200} = \frac{28672 \cdot 2}{2200} = 26,$$

so we're safe.

Question

On TPU v5e, using bfloat16 weights and activations, how large do our batch sizes need to be for us to be compute-bound in our matmuls? What if we do int8 weights but perform our FLOPs in bfloat16? What about int8 weights with int8 FLOPs?

Solution. Accelerator intensity for v5e in bf16: 240.

bf16 weights: 240

int8 weights: $240/2 = 120$ (half the time for loading)

int8 everything: $120 \cdot 2 = 240$ (half the time for FLOPs, since we can do int8 FLOPs)

Question

What is the smallest TPU v5e topology we could serve LLaMA 3-70B on using bfloat16, int8, and int4 (both KVs and parameters) with 8k context? *You can think of KV caches as negligibly small for this one.*

Solution. Number of params: 70B. KV cache entries per sequence: $2 \cdot 8 \cdot 128 \cdot 80 \cdot 8192 = 1.34\text{B}$.
bfloat16:

$$\text{param size} = 140 \text{ GB}, \quad \text{num devices} = 140/16 = 8.75 \Rightarrow \text{smallest slice } 4 \times 4.$$

$$\text{num sequences} = \frac{256 \text{ GB} - 140 \text{ GB}}{2 \cdot 1.34 \text{ GB}} = 43.2 \approx 43.$$

int8:

$$\text{param size} = 70 \text{ GB}, \quad \text{num devices} = 70/16 = 4.375 \Rightarrow \text{min slice } 4 \times 2.$$

$$\text{num seq} = \frac{8 \cdot 16 \text{ GB} - 70 \text{ GB}}{1.34} = 43.2 \approx 43.$$

int4:

$$\text{param size} = 35 \text{ GB}, \quad \text{num devices} = \frac{4.375}{2} = 2.19 \Rightarrow \text{min slice } 2 \times 2.$$

$$\text{max seq} = \frac{4 \cdot 16 \text{ GB} - 35 \text{ GB}}{1.34/2} = 43.2 \approx 43.$$

Question

Assume we use the largest batch size that fits on these topologies, what latency could we expect for each generate step?

Solution. Since $43 < 120$, we are bandwidth bound, not compute bound. Also we aren't ICI bound since that only happens when $Y > 26$. So it's just HBM capacity/bw:

$$\frac{16 \text{ GB}}{8.2\text{e}11} \approx \boxed{19.5 \text{ ms}}$$

Question

For each of these, what throughput per chip does this give us (in terms of queries / chip)? *You can assume our median decode length is 512 tokens.*

Solution. At batch size 43:

$$\text{total throughput} = \frac{43}{512 \cdot 19.5 \text{ ms}} = 4.31 \text{ qps}.$$

Per chip:

$$\begin{aligned} \text{bf16: } \frac{4.31}{16} &= 0.27 \text{ qps/chip} \\ \text{int8: } \frac{4.31}{8} &= 0.54 \text{ qps/chip} \\ \text{int4: } \frac{4.31}{4} &= 1.08 \text{ qps/chip} \end{aligned}$$

Question

How would our peak throughput change if we doubled our topology for each of the above examples?

Solution. bf16, 4×8 slice:

$$\frac{32 \cdot 16 \text{ GB} - 140 \text{ GB}}{2 \cdot 1.34 \text{ GB/seq}} = 138 \text{ seqs} \Rightarrow \text{total throughput} = \frac{138}{512 \cdot 19.5 \text{ ms}} = 13.82 \text{ qps} \Rightarrow \frac{13.82}{32} = 0.43 \text{ qps/chip.}$$

int8, 4×4 slice:

$$\frac{16 \cdot 16 \text{ GB} - 70 \text{ GB}}{1.34 \text{ GB/seq}} = 138 \text{ seqs} \Rightarrow 13.82 \text{ qps} \Rightarrow 0.86 \text{ qps/chip.}$$

int4, 4×2 slice:

$$\frac{8 \cdot 16 \text{ GB} - 35 \text{ GB}}{\frac{1}{2} \cdot 1.34 \text{ GB/seq}} = 138 \text{ seqs} \Rightarrow 13.82 \text{ qps} \Rightarrow 1.72 \text{ qps/chip.}$$

Question

Now let's dig into the question of sharding. Let's say we wanted to serve in bfloat16 on a TPU v5e 4x8. What sharding would we use for our model on a TPU v5e 4x8 during generation? Can we avoid being communication bound?

Solution. We've seen previously that when $Y > 26$ (2 axes) we go from FLOPs bound to ICI bound. Thus we cannot avoid being comms bound on a 4×8 slice.

But we can still be HBM bound vs ICI. We only become ICI bound vs HBM when

$$Y > \frac{F}{8B} \Rightarrow Y > \frac{28672}{8B}.$$

If $Y = 32$, then this only happens when

$$B > \frac{28672}{8 \cdot 32} = 112.$$

Below that we are HBM bound, and so our latency still goes down compared to a 4×4 slice.

Question

Assume we achieve a 40% FLOPs utilization during prefill. How long will a prefill of length 8192 take on 16 TPU v5e chips?

Solution.

$$\text{FLOPs: } 2ND = \frac{2 \cdot 70B \cdot 8192}{16 \cdot 1.97\text{e}14 \cdot 0.4} = \boxed{909 \text{ ms}}$$

Question

Assume we have a median prefill length of 8192 tokens and a median decode length of 4096 tokens. Say we have a generate batch size of 32. On average how many sequences finish decoding per step? On average how many tokens are evicted from our KV cache each step?

Solution. After 4096 steps we evict 32 seqs, so $32/4096$ seqs per step are evicted. Each evicted sequence has $8192 + 4096$ KV cache tokens. Thus

$$\frac{32 \cdot (8192 + 4096)}{4096} = \boxed{96}$$

tokens are evicted from the KV cache each step.

Question

Assume we do disaggregated serving with a median prefill length of 8192 and a median decode length of 512. Assume the prefill and generate latencies calculated above in bfloat16. What ratio of prefill:generate servers will you need to keep both fully saturated.

Solution. Prefill latency: 909 ms/sequence on a 4×4 slice. Decode latency: 19.5 ms (at bs 43) on a 4×4 slice.

For 512 steps that is $19.5 \cdot 512 \approx 10$ s to decode 43 seqs, so decode generation servers can go through 4.3 seqs/second.

Prefill can go through $\frac{1}{0.909} = 1.10$ seqs/s.

$$\text{ratio} \approx \frac{4}{1} \text{ prefill : decode} \Rightarrow \boxed{4 \text{ prefill servers for every decode server}}$$

Question 1

How many FLOPs does each forward pass for LLaMA 3-405B use per-token? Assuming we're FLOPs bound, what is a lower bound on a single forward pass on N chips on TPU v5e? What if we're comms bound? *Ignore the fact that the model does not fit on a single chip.*

Solution. LLaMA 3-405B has $N = 405 \times 10^9$ parameters, so each forward pass uses per token

$$2N = 2 \cdot 405 \times 10^9 = 810 \text{ GFLOPs/token.}$$

If we are FLOPs bound, then on N_{chips} TPU v5e chips (each 1.97×10^{14} bf16 FLOP/s) a lower bound on a single forward pass per token is

$$\frac{2 \cdot 405 \times 10^9}{1.97 \times 10^{14} \cdot N_{\text{chips}}} = \boxed{\frac{4.11 \text{ ms}}{N_{\text{chips}}}}.$$

If instead we are comms (HBM) bound, the cost is dominated by loading the parameters. Taking the weights in bf16, $2 \cdot 405 = 810$ GB, and HBM bandwidth 8.2×10^{11} B/s per chip:

$$\frac{810 \times 10^9}{8.2 \times 10^{11} \cdot N_{\text{chips}}} = \boxed{\frac{987.9 \text{ ms}}{N_{\text{chips}}}}.$$

Question 2

Assume we want to serve LLaMA 3-8B with BS240 using int8 weights and int8 KV caches. How many bytes are used by (a) model parameters (b) KV caches and (c) peak working activations (roughly)? What's the smallest topology we can run this on?

Solution. LLaMA 3-8B: $D = 4096$, $F = 14336$, $L = 32$, $K = 8$, $H = 128$, $V = 128256$. Serving at batch size 240 with int8 weights and int8 KV caches.

(a) Model parameters: with 8×10^9 parameters at 1 byte each,

$$\boxed{\approx 8 \text{ GB}}.$$

(b) KV caches (assuming 8192 sequence length). Per token,

$$2KHL = 2 \cdot 8 \cdot 128 \cdot 32 = 65536 \text{ bytes} = 64 \text{ kB},$$

so per sequence $2KHL \cdot 8192 = 536.9$ MB, and at $B = 240$,

$$240 \cdot 536.9 \text{ MB} = \boxed{128.9 \text{ GB}}.$$

(c) Peak working activations (roughly). Attention scores dominate:

$$B \cdot (K \cdot G) \cdot S = 240 \cdot 8 \cdot 4 \cdot 8192 = 62.9 \text{ MB},$$

and the projection matrices are

$$2BTF = 2 \cdot 240 \cdot 1 \cdot 14336 = 7 \text{ MB},$$

so peak activations add negligible memory overhead.

Total memory:

$$8 \text{ GB} + 128.9 \text{ GB} \approx 137 \text{ GB}.$$

Since $\frac{137 \text{ GB}}{16 \text{ GB}} \approx 8.5$, the smallest slice is

$$\boxed{4 \times 4 \text{ (16 chips)}}.$$

Question 3

How would you serve LLaMA 3-405B on TPU v5e? Assume int8 weights and bfloat16 FLOPs. Let's say we have a firm limit of 15ms / token, what's the highest throughput configuration we could achieve? What is the theoretical minimum step time?

Solution. LLaMA 3-405B ($D = 16384$, $K = 8$, $H = 128$, $L = 126$, $F = 53248$), int8 weights, bf16 FLOPs, 8192 sequence length. Parameters are 405 GB.

KV cache per sequence:

$$2KHL \cdot 8192 = 2 \cdot 8 \cdot 128 \cdot 126 \cdot 8192 = 2.114 \text{ GB/seq}.$$

Just to fit the parameters we need $\frac{405}{16} \approx 25$ chips minimum.

Try a 4×8 slice (32 chips). Parameter loading time alone is

$$\frac{405 \text{ GB}}{32 \cdot 8.2 \times 10^{11}} = 15.43 \text{ ms},$$

which is a strict lower bound on latency and already exceeds our budget, so we need more chips. Move to an 8×8 slice.

On 8×8 (64 chips), HBM = $64 \cdot 16 = 1024$ GB. After parameters, $1024 - 405 = 619$ GB remain for KV cache, giving

$$\frac{619 \text{ GB}}{2.114 \text{ GB/seq}} = 292.8 \text{ sequences},$$

so we can be compute-bound for the FFN (critical batch size ≈ 120 for int8 weights with bf16 FLOPs); let's aim for $B = 240$.

Assuming compute bound, the latency is

$$\underbrace{\frac{B \cdot 2.114 \times 10^9}{64 \cdot 8.2 \times 10^{11}}}_{\text{attention}} + \max \left(\frac{2B \cdot 405 \times 10^9}{64 \cdot 1.97 \times 10^{14}}, \frac{405 \times 10^9}{64 \cdot 8.2 \times 10^{11}} \right).$$

Requiring ≤ 15 ms:

$$\frac{B \cdot 2.114 \times 10^9}{64 \cdot 8.2 \times 10^{11}} + \frac{2B \cdot 405 \times 10^9}{64 \cdot 1.97 \times 10^{14}} \leq 15 \times 10^{-3},$$

$$4.02 \times 10^{-5} B + 6.42 \times 10^{-5} B \leq 15 \times 10^{-3} \Rightarrow B \leq 143.5 \Rightarrow B \leq 143$$

to stay below the latency limit. Then throughput per chip is

$$\approx \frac{143}{0.015 \cdot 64} = 148.95 \text{ tok/s/chip}.$$

However, we are actually ICI bound! For 405B we need

$$Y \leq \frac{F}{8B} = \frac{53248}{8B} \Rightarrow 64 \leq \frac{53248}{8B} \Rightarrow B \leq 104.$$

So to not be ICI bound at 8×8 tensor parallelism we need $B \leq 104$.

Checking the single-matmul times with $B = 100$:

$$\begin{aligned} T_{\text{ici}} &= \frac{2 \cdot 100 \cdot 16384}{9 \times 10^{10}} = 36.4 \mu s, \\ T_{\text{hbm}} &= \frac{2 \cdot 16384 \cdot 53248}{64 \cdot 8.2 \times 10^{11}} = 33.25 \mu s, \\ T_{\text{math}} &= \frac{2 \cdot 100 \cdot 16384 \cdot 53248}{64 \cdot 1.97 \times 10^{14}} = 13.84 \mu s, \end{aligned}$$

so we are closer to the HBM bound (not exactly, since with LLaMA 405B $F \leq 4D$).

Taking $B = 100$ (MLP now memory bound since $B < 120$):

$$\text{latency} = \frac{100 \cdot 2.114 \times 10^9}{64 \cdot 8.2 \times 10^{11}} + \frac{405 \times 10^9}{64 \cdot 8.2 \times 10^{11}} = 4.02 \times 10^{-3} + 7.717 \times 10^{-3} = \boxed{11.74 \text{ ms}} \text{ (min step time)}.$$

Throughput per chip:

$$\frac{100}{64 \cdot 0.01174} = \boxed{133.1 \text{ tok/s}} \text{ (not much worse than 149).}$$

Scaling further gives better latency but the same/worse throughput per chip.

9 Chapter 9: How to Profile TPU Programs

Question 1

When taking a look at the profile, the telling operation is:

```
%all-reduce = bf16[1,8192]{1,0:T(2,128)(2,1)} all-reduce(bf16[1,8192]{1,0:T(2,128)(2,1)S(1)}) %reshape.8), channel_id=1, replica_groups=[1,8]<=[8], use_global_device_ids=true, to_apply=%region_1.21.clone
```

In particular, we can infer that the second matrix was sharded along its contracting dimension (4096). Since the all-reduce happened over all 8 devices, the true size of this contracting dimension is $4096 * 8 = 32768$. Thus, the shape of the full computation is:

$$\text{In}[1, 8192] * W_{\text{up}}[8192, 32768] * W_{\text{down}}[32768, 8192]$$

In terms of what looks suspicious, the profile shows that we are highly memory bound (mainly due to the batch size being 1). Note that each device is doing $2 * 4096 * 8192 \approx 6.7e7$ FLOPs per matrix multiplication, which should take roughly 0.34us, but is loading $2 * 4096 * 8192$ bytes from HBM, which takes 82us. And we can see from the profile that the first matmul takes 90us, so we're close to the HBM roofline, but we are bound by weight movement (it looks like the second matmul was prefetched from HBM into VMEM, which is why it took less time).

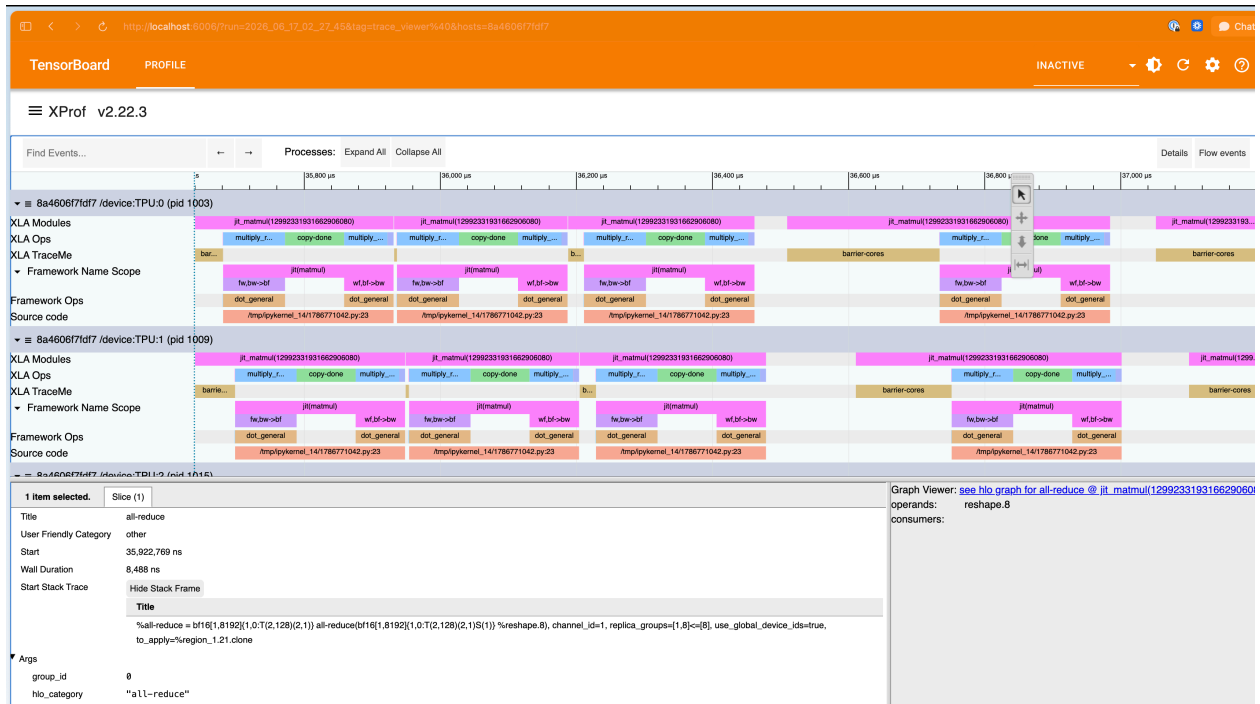


Figure 3: Screenshot of the profile.

Part	Measured	Roofline
<i>per layer</i> ($\times 8$)		
Q/K/V projections ($\times 3$)	1.14 ms	1.05 ms
QK^T + softmax + AV	0.12 ms	0.09 ms
attn_proj	0.36 ms	0.35 ms
attn all-reduce (f32)	1.59 ms	1.49 ms
FFW up + down	6.01 ms	5.58 ms
FFW all-reduce (bf16)	0.78 ms	0.75 ms
RMSNorms	~ 0.20 ms	~ 0
per-layer subtotal	~ 10.2 ms	~ 9.3 ms
<i>per step</i> ($\times 1$)		
Embedding (vd,btv->btd)	5.67 ms	5.47 ms
Unembedding / logits	5.70 ms	5.47 ms
Whole step	99.2 ms	67.5 ms

Table 1: Unoptimized version: measured vs. roofline time. Whole-step MXU utilization $\approx 68\%$.

Part	Measured	Roofline
<i>per layer</i> ($\times 8$)		
all-gather (un-shard d_{model} , QKV)	0.37 ms	0.37 ms
Q/K/V projections ($\times 3$, incl. V transpose)	1.27 ms	1.05 ms
QK^T + softmax + AV	0.27 ms	0.09 ms
attn_proj	0.36 ms	0.35 ms
attn_proj reduce-scatter (f32)	0.86 ms	0.75 ms
all-gather (un-shard d_{model} , FFW)	0.37 ms	0.37 ms
FFW up + down	5.90 ms	5.58 ms
FFW reduce-scatter (bf16)	0.39 ms	0.37 ms
RMSNorms (+ tiny stat AR)	~ 0.04 ms	~ 0
per-layer subtotal	~ 9.8 ms	~ 8.9 ms
<i>per step</i> ($\times 1$)		
Embedding (vd,btv->btd)	2.86 ms	2.73 ms
Unembedding / logits (+ all-gather)	5.70 + 0.37 ms	5.47 + 0.37 ms
Whole step	89.1 ms	64.8 ms

Table 2: Optimized version (shard d_{model}): measured vs. roofline time. Whole-step MXU utilization $\approx 73\%$.

Question 2

Table 1 shows a breakdown of how long (for the unoptimized version) each step takes, and how long it should take (roofline). It should be noted that trying to run the given notebook out of the box did not work. It seems as if Jax has become stricter about specifying sharding constraints. I had to explicitly replicate d_{model} to get the code to run.

The natural next step to optimize this was to shard along d_{model} . This, in theory, makes it easier for comms and compute to be overlapped. In particular, we can split every AllReduce into an AllGather at the beginning followed by a ReduceScatter at the end. While the same amount of bytes are being moved, this gives a chance for the compiler to overlap comms more efficiently. Doing this, we get the profile in Table 2.

From Table 2 (derived directly from the profile) we indeed see gains, but they come from some unexpected places. In particular: Firstly, each device is doing half the work for the embedding step in the unoptimized version compared to the optimized version. That saves ~ 2.8 ms. It should also be noted that no collectives were actually overlapped. Everything ran in sequence. However, we still saved ~ 0.4 ms per layer in collectives. This is because we split up the fp32 AllReduce in the attention block into an fp32 ReduceScatter and a bf16 AllGather. So we moved 25% less data. We also get 1ms of savings total from reduced computation from RMSNorms.

The last couple of ms look like a measurement artifact – I profiled the unoptimized version first, and there seems to be a synchronization step that takes ~ 2.5 ms before any computation happens, that isn't present in the optimized profile. Overall, simply sharding d_{model} saved ~ 7 ms during the forward pass.

Directly from the optimized profile we can answer the following:

1. We are doing DP 4, TP 2. We can tell from this op:

```
%all-gather.109 = bf16[2,1024,8192]{2,1,0:T(8,128)(2,1)S(1)} all-gather(bf16
  [2,1024,4096]{2,1,0:T(8,128)(2,1)S(1)} %multiply_multiply_fusion.23), channel_id
=2, replica_groups=[4,2]<=[8], dimensions={2}, use_global_device_ids=true,
frontend_attributes={async_collective_name="all-gather-start"}
```

which shows an all gather over two devices before doing the FFN computation.

2. The batch size is 8 (2×4), and the sequence length T is 1024. Since we AllGather the entire activations at each step, we can read d_{model} right off the AllGather: 8192.

From the up projection HLO:

```
%fusion.53 = bf16[2,1024,16384]{2,1,0:T(8,128)(2,1)} fusion(bf16[2,1024,8192]{2,1,0:T(8,128)(2,1)S(1)} %all-gather.110, bf16[8192,16384]{1,0:T(8,128)(2,1)} %param
.8), kind=kOutput, calls=%fused_computation.40
```

we can see that the output shape is $[B, T, d_{\text{ff}}/2]$. Thus, d_{ff} is 32768, or $4 \cdot d_{\text{model}}$.

3. Not including comms (AR + RS), we can see from Table 2 that $\sim 1.9\text{ms}$ is spent on attention per layer, and $\sim 5.9\text{ms}$ is spent on the MLP block per layer. So roughly 3x more time is spent on the MLP block compared to attention.
4. At the roofline, we see attention per layer should cost 1.49ms and the MLP block should cost 5.58ms. This is a factor of 3.7x.

10 Chapter 10: Programming TPUs in JAX

For readability, I only include written answers to the questions here. The Appendix contains the full Kaggle notebook associated with these problems.

I used a (2, 4) mesh for all problems.

Question 1

Part 1

Both implementations took roughly 12us to complete. No comms were added in the `jit` case (and obviously not in the manual `shard_map` implementation either). The difference in total time seems to be entirely due to the difference in kernel launch overheads.

Part 2

See notebook for implementation.

Question 2

Part 1

See notebook in appendix for implementation. I used `jax.lax.ragged_dot`, same as I used in my implementation of the MoE layer for the coding part of this project.

Part 2

We see a huge AllGather! All activations are AllGathered and replicated on each device. Refer to the screenshots in the profile (Figures 4 and 5). 97% of the execution time is taken up by the AllGather. The total execution time is 1.5ms.

Part 3

As recommended I used AllToAll collectives inside of a while loop, processing one chunk at a time, where a chunk was the next C tokens for each expert (with padding if an expert was already exhausted).

This explicit implementation was around 8x faster (3.5ms vs 450us). See the screenshot of the HLO ops (Figure 6). Much less time is spent on comms.

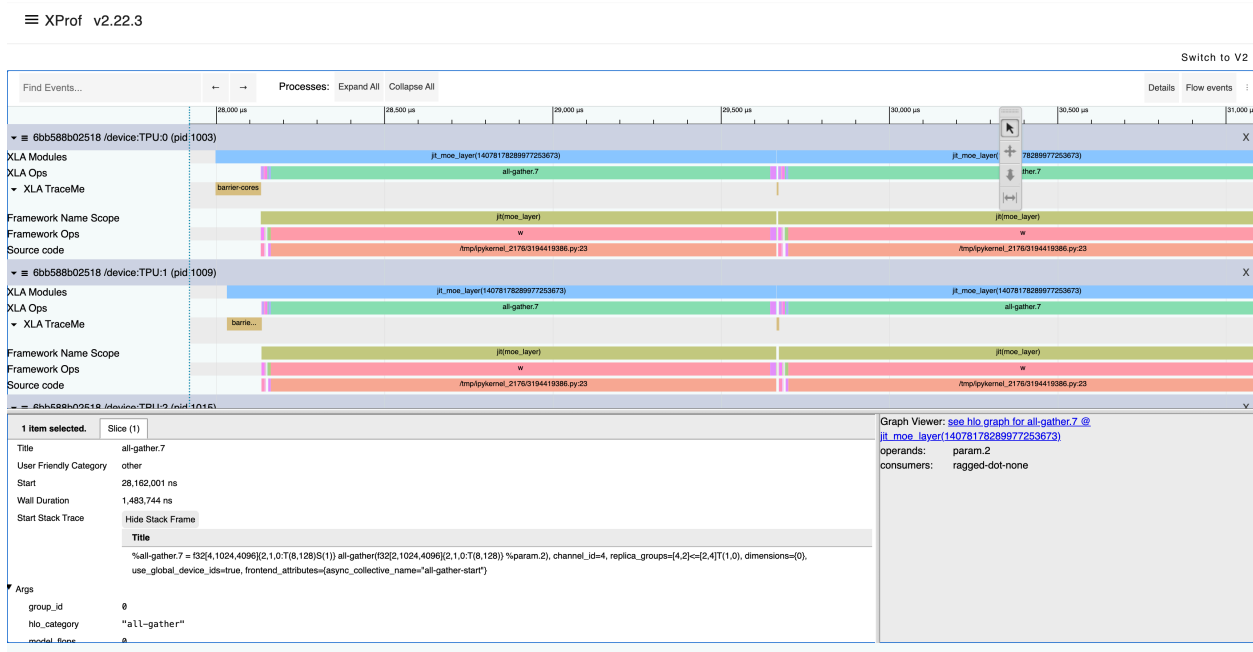


Figure 4: Trace view of the ragged_dot MoE layer: a single AllGather dominates the timeline.

Part 4

See notebook for implementation.

Question 3

Part 1

The idea here was to do a sequence of smaller matmuls and AllReduces over the Y axis. Since $\text{matmul}(i+1)$ is not dependent on $\text{AllReduce}(i)$, these operations can be overlapped and the communication can be (at least partially) hidden.

Part 2

The way we overlap comms and compute this time is by having each device compute their contribution of the partial sum that they receive each step, and add it to the accumulator, and then send it on (`ppermute`). Same as before, each device can compute their next contribution while the previous accumulators are being permuted, so in theory these can be overlapped.

Part 3

Viewing the profiles, we see that the implementation using the handwritten collectives performs slightly better (1.98ms vs 2.23ms) but it's not due to overlapping comms and compute. In fact, even in the handwritten version, no comms or compute were overlapped at all.

Overall TPU FLOPS utilization is **0.02%**
Memory bandwidth utilizations: Hbm **2.69%**

Modifying your model's architecture, data dimensions, and improving the efficiency of CPU operations may help reach the TPU's FLOPS potential.

"IDLE" represents the portion of the total execution time on device that is idle.

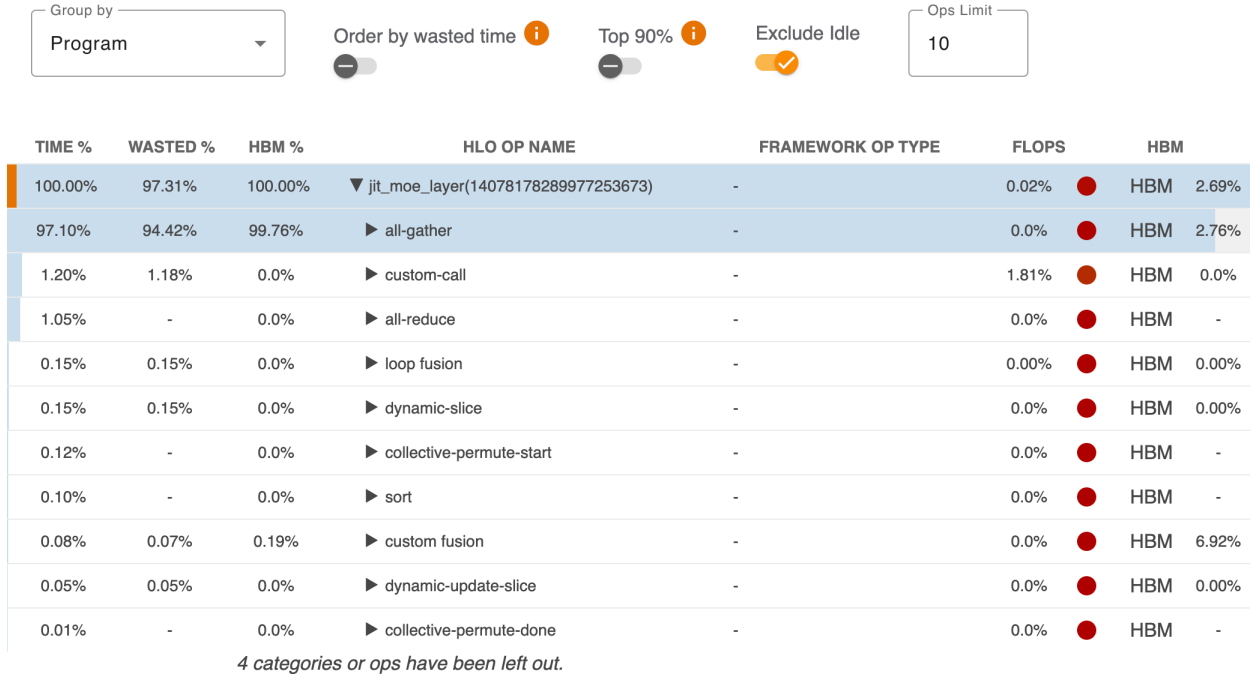


Figure 5: HLO ops for the same run; the AllGather accounts for roughly 97% of the execution time.

It turns out that in the jitted version, XLA also lowers the implementation to an AllGather followed by ReduceScatter. These comms in total took ~1.5ms. In the handwritten implementation, we instead see a sequence of matrix multiplications and **ppermutes**, and the **ppermutes** in total took ~1.1ms.

So even though in both cases everything happened in sequence, the win in my handrolled implementation came from the comms side. This is curious, since both implementations should have moved the same number of bytes. I guess it's possible that for Y=4, and no wraparound on a v5e-8 slice, XLA didn't see a big enough speedup in the overlap.

See the Figures 7 and 8 for the two profiles.

Question 4

I compare the unidirectional and bidirectional reduce scatter implementations. In terms of total runtime, we see a very modest speedup: (1.3ms vs 1.2ms). If we break it down, we see that compute took the same amount of time, and the bidi comms were 100us faster (900us vs 800us). Looking at the profile, we can see that the first 2 permutations were in fact overlapped in the bidi case, but the total comms time then took about double (500us vs 250us) implying that these comms shared bandwidth. Again, this is probably an artifact of the mesh.

Overall TPU FLOPS utilization is **3.45%**
Memory bandwidth utilizations: Hbm **28.70%**

Modifying your model's architecture, data dimensions, and improving the efficiency of CPU operations may help reach the TPU's FLOPS potential.

"IDLE" represents the portion of the total execution time on device that is idle.

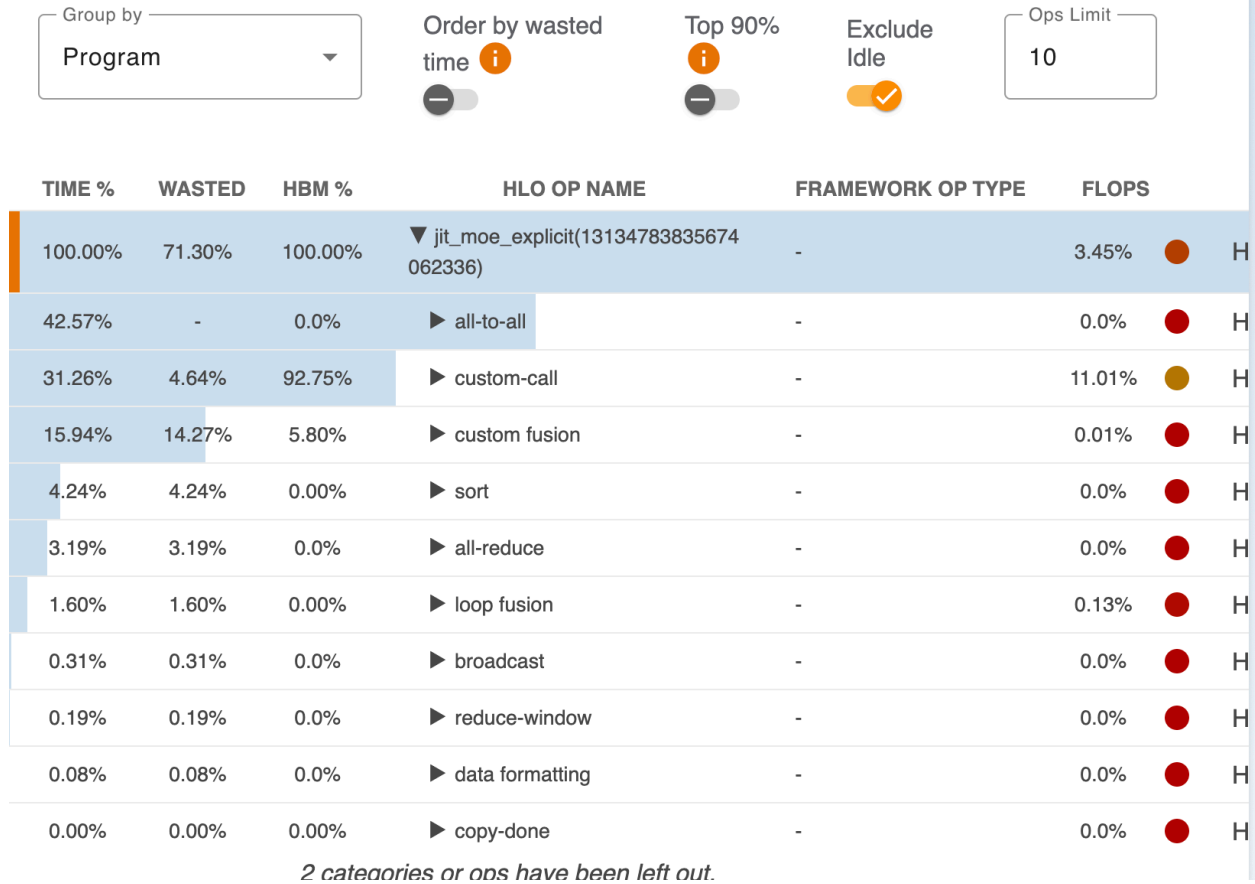


Figure 6: HLO ops for the explicit AllToAll implementation; far less time is spent on communication.

See Figures 9 and 10 for the two profiles.

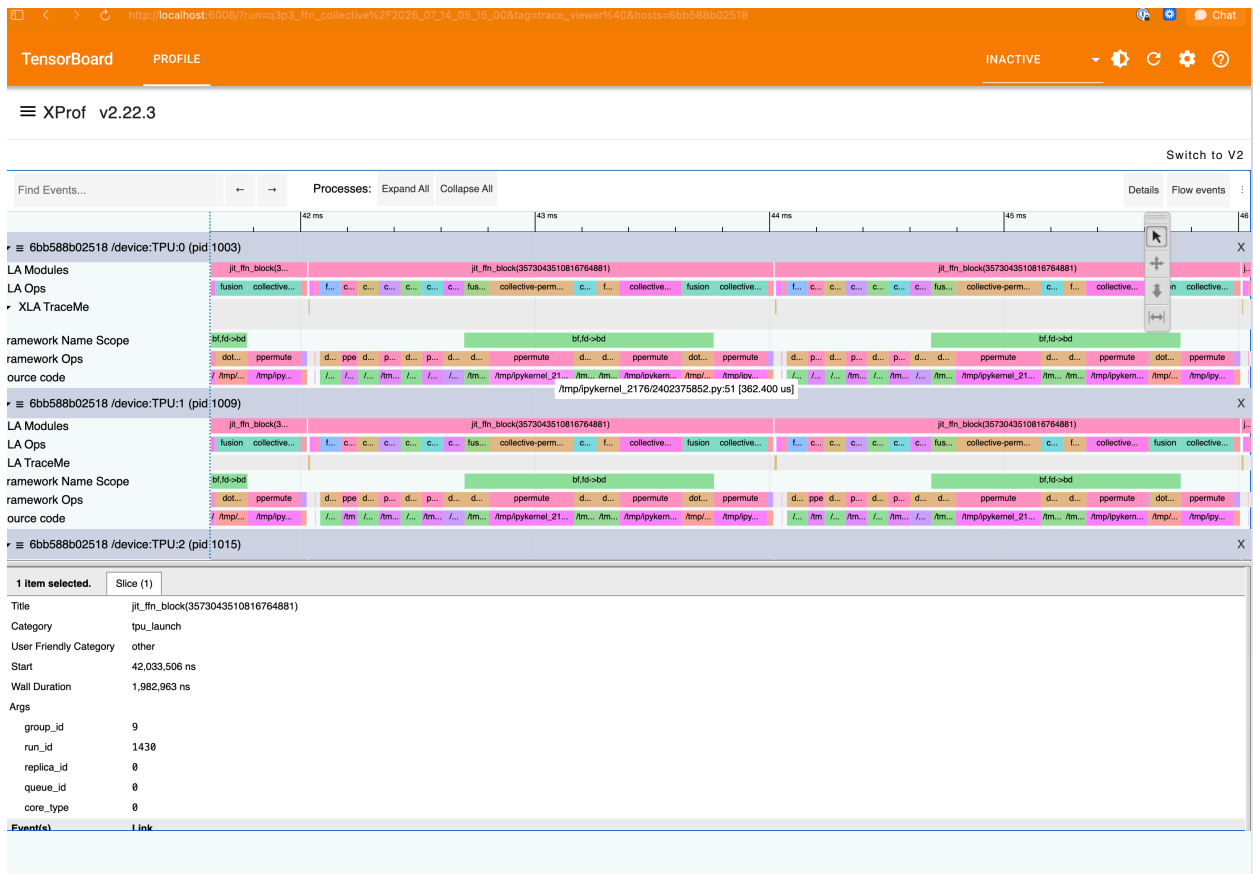


Figure 7: Hand-written collective implementation (matmuls + ppermutes): 1.98ms total.

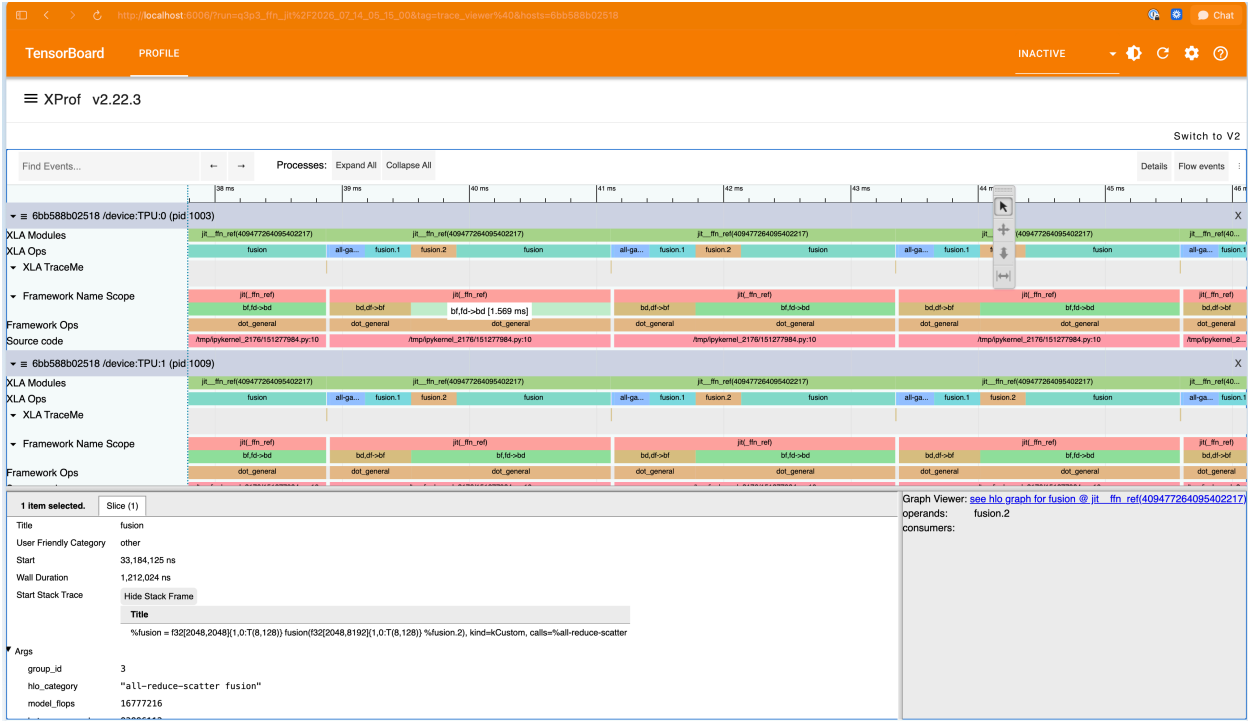


Figure 8: Jitted implementation, which XLA lowers to AllGather + ReduceScatter: 2.23ms total.

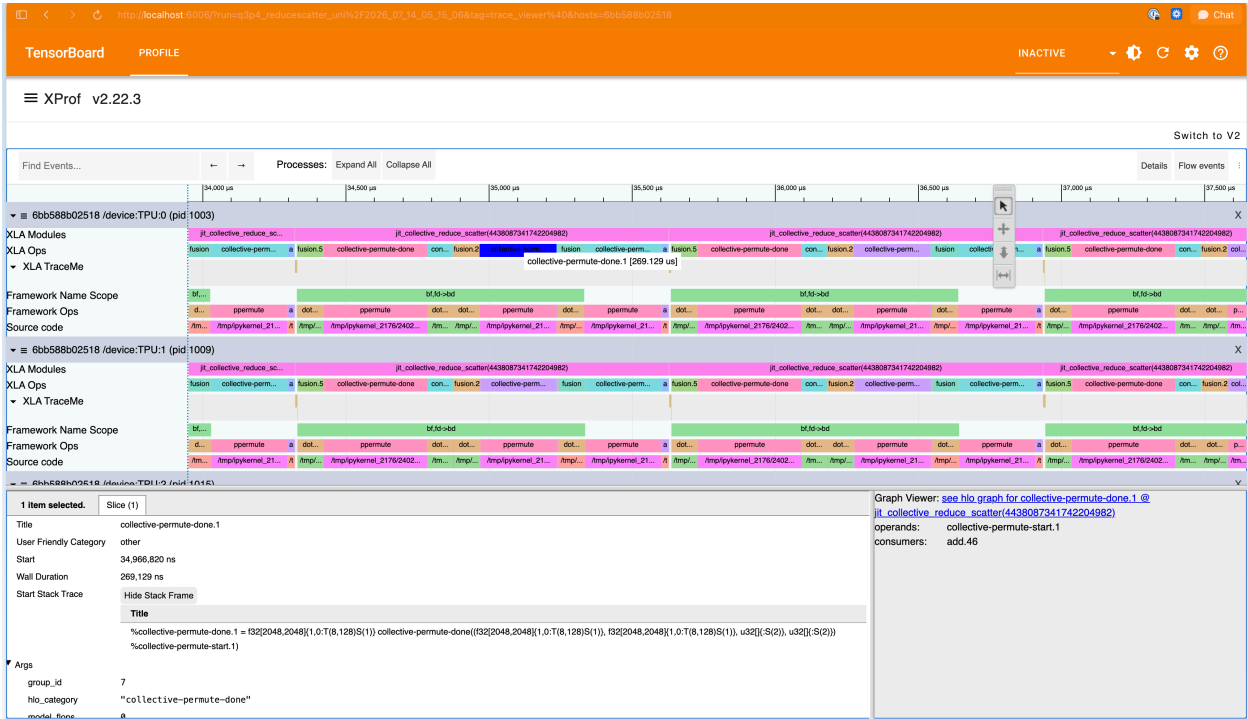


Figure 9: Unidirectional reduce-scatter.

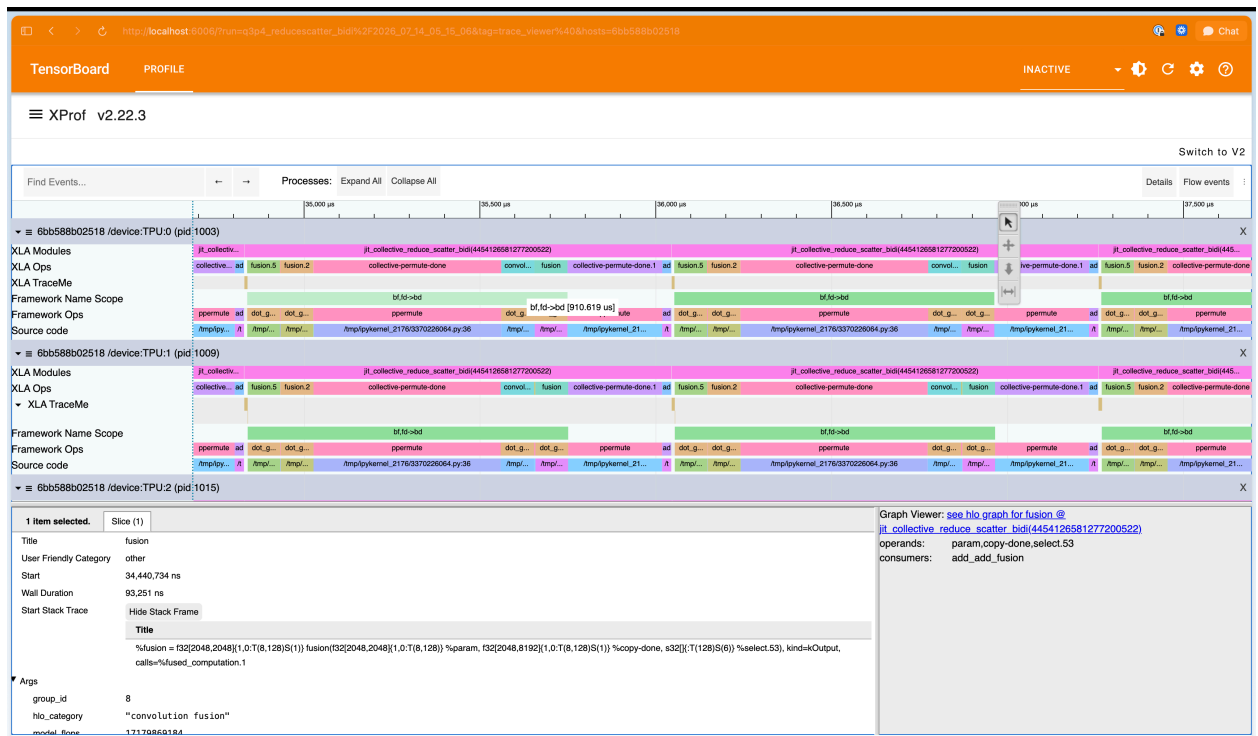


Figure 10: Bidirectional reduce-scatter; the first two permutes overlap but share bandwidth.

11 Chapter 12: How to Think About GPUs

Question 1 [CUDA cores]

How many fp32 CUDA cores (ALUs) does an H100 have? B200? How does this compare to the number of independent ALUs in a TPU v5p?

Solution. For the H100 there are 132 SMs, each with 4 subpartitions containing 32 fp32 CUDA cores:

$$132 \cdot 4 \cdot 32 = \boxed{16896 \text{ CUDA cores}}$$

For the B200 there are 148 SMs/chip with the same cores per subpartition (4 subpartitions each):

$$148 \cdot 4 \cdot 32 = \boxed{18944 \text{ CUDA cores}}$$

From Chapter 2 we know a TPU v5p has 8192 ALU cores, roughly half of the H100's count (running at roughly the same frequency).

Question 2 [Vector FLOPs calculation]

A single H100 has 132 SMs and runs at a clock speed of 1.59GHz (up to 1.98GHz boost). Assume it can do one vector op per cycle per ALU. How many vector fp32 FLOPs can be done per second? With boost? How does this compare to matmul FLOPs?

Solution. Assuming one vector op per cycle per ALU, at the base clock:

$$16896 \cdot 1.59\text{e}9 = 2.68\text{e}13 \text{ FLOPs/s}$$

With boost:

$$16896 \cdot 1.98\text{e}9 = 3.35\text{e}13 \text{ FLOPs/s}$$

Compared to matmul FLOPs:

$$\text{matmul: } 9.9\text{e}14 \text{ FLOPs/s}$$

so the Tensor Cores deliver far more FLOPs/s than the vector units.

Question 3 [GPU matmul intensity]

What is the peak fp16 matmul intensity on an H100? A B200? What about fp8? *By intensity we mean the ratio of matmul FLOPs/s to memory bandwidth.*

Solution. The intensity is the ratio of matmul FLOPs/s to memory bandwidth.

H100:

$$\frac{9.9\text{e}14}{3.4\text{e}12} = \boxed{291}$$

B200:

$$\frac{2.3\text{e}15}{8.0\text{e}12} = \boxed{288}$$

For fp8 the intensity is the same, since the FLOPs get multiplied by 2 but the weight load also does.

Question 4 [Matmul runtime]

Using the answer to Question 3, how long would you expect an `fp16[64, 4096] * fp16[4096, 8192]` matmul to take on a single B200? How about `fp16[512, 4096] * fp16[4096, 8192]`?

Solution. We consider `fp16[64, 4096] · fp16[4096, 8192]` and `fp16[512, 4096] · fp16[4096, 8192]` on a single B200.

Part (a): `[64, 4096] · [4096, 8192]`.

Compute time:

$$\frac{2 \cdot 64 \cdot 4096 \cdot 8192}{2.3e15} = 1.88 \mu s$$

Comms/memory time ($2BD + 2DF + 2BF$ bytes at 8.0e12 B/s):

$$\frac{2 \cdot 64 \cdot 4096 + 2 \cdot 4096 \cdot 8192 + 2 \cdot 64 \cdot 8192}{8.0e12} = 8.59 \mu s$$

$$\Rightarrow \boxed{8.59 \mu s} \quad (\text{max of above})$$

Part (b): `[512, 4096] · [4096, 8192]`.

Compute time:

$$\frac{2 \cdot 512 \cdot 4096 \cdot 8192}{2.3e15} = 1.88 \cdot 9 \approx 15.04 \mu s$$

Comms/memory time:

$$\frac{2 \cdot 512 \cdot 4096 + 2 \cdot 4096 \cdot 8192 + 2 \cdot 512 \cdot 8192}{8.0e12} = 9.97 \mu s$$

$$\Rightarrow \boxed{\text{max} = 15.04 \mu s}$$

Question 5 [L1 cache capacity]

What is the total L1/SMEM capacity for an H100? What about register memory? How does this compare to TPU VMEM capacity?

Solution. H100: 132 SMs/chip with 256 kB/SM (SMEM/L1).

$$132 \cdot 256 \text{ kB} = \boxed{33.8 \text{ MB SMEM capacity}}$$

vs TPU VMEM: 128 MiB.

Registers: 256 kB/SM as well, so also 33.8 MB for the H100.

TPU register memory: 256 kB total — much less.

Question 6 [Calculating B200 clock frequency]

NVIDIA reports here that a B200 can perform 80TFLOPs/s of vector fp32 compute. Given that each CUDA core can perform 2 FLOPs/cycle in a FMA (fused multiply add) op, estimate the peak clock cycle.

Solution. With 2 FLOPs/cycle per CUDA core (FMA), 80 TFLOPs/s corresponds to

$$\frac{80e12}{2} = 40e12 \text{ cycles/s (times CUDA cores)}$$

Using 18944 CUDA cores:

$$\frac{40e12}{18944} = 2.1e9 \Rightarrow \boxed{2.1 \text{ GHz}}$$

Question 7 [Estimating H100 add runtime]

Using the figures above, calculate how long it ought to take to add two `fp32[N]` vectors together on a single H100. Calculate both T_{math} and T_{comms} . What is the arithmetic intensity of this operation? If you can get access, try running this operation in PyTorch or JAX as well for $N = 1024$ and $N=1024 * 1024 * 1024$. How does this compare?

Solution. Adding two `fp32[N]` vectors performs N FLOPs and moves $12N$ bytes ($2 \cdot 4N$ loaded, $4N$ written).

Compute time:

$$T_{\text{math}} = \frac{N \text{ FLOPs}}{26.9\text{e}12 \text{ FLOPs/s}} = \frac{N}{26.9\text{e}12}$$

Comms time:

$$T_{\text{comms}} = \frac{12N}{3.4\text{e}12} = \frac{N}{2.83\text{e}11}$$

Arithmetic intensity:

$$I = \frac{N}{12N} = \boxed{\frac{1}{12}}$$

Question 1 [Total bandwidth for H100 node]

How much total bandwidth do we have per node in an 8xH100 node with 4 switches? *Hint:* consider both the NVLink and NVSwitch bandwidth.

Solution. From the chapter, NVLink bandwidth is 450 GB/s (each GPU), and an NVSwitch can handle 6.4 TB/s.

Since each GPU can egress at most 450 GB/s, over 8 GPUs/node:

$$450 \text{ GB/s} \cdot 8 = 3.6 \text{ TB/s} < 6.4 \text{ TB/s}$$

$$\Rightarrow \boxed{3.6 \text{ TB/s total BW/node}}$$

Question 2 [Bisection bandwidth]

Bisection bandwidth is defined as the smallest bandwidth available between any even partition of a network. In other words, if we split a network into two equal halves, how much bandwidth crosses between the two halves? Can you calculate the bisection bandwidth of an 8x H100 node? *Hint:* bisection bandwidth typically includes flow in both directions.

Solution. Any even partition puts 4 GPUs in each half, each able to egress $4 \cdot 450 \text{ GB/s} = 1.8 \text{ TB/s}$. Counting flow in both directions (full duplex):

$$\Rightarrow \boxed{3.6 \text{ TB/s total (bisection) BW}}$$

Question 3 [AllGather cost]

Given an array of B bytes, how long would a (throughput-bound) AllGather take on an 8xH100 node? Do the math for `bf16[DX, F]` where $D=4096$, $F=65,536$. *It's worth reading the TPU collectives section before answering this. Think this through here but we'll talk much more about collectives next.*

Solution. Each GPU holds $B/8$ bytes and needs to send all $B/8$ bytes to each of the other 7 GPUs, i.e. $\frac{7B}{8}$ egress at 450 GB/s:

$$\frac{\frac{7B}{8}}{450 \text{ GB/s}} = \frac{B}{514e9} \text{ s}$$

For $D = 4096$, $F = 65,536$ (bf16, so $B = 2 \cdot 4096 \cdot 65536$):

$$\frac{2 \cdot 4096 \cdot 65536}{514e9} \approx \boxed{1.04 \text{ ms}}$$

Question 1 [Fat tree topology]

Using the DGX H100 diagram above, calculate the bisection bandwidth of the entire 1024 GPU pod at the node level. Show that the bandwidth of each link is chosen to ensure full bisection bandwidth. *Hint: make sure to calculate both the link bandwidth and switch bandwidth.*

Solution. The DGX H100 pod has 1024 GPUs = 128 nodes (8 GPUs per node), grouped into 4 scalable units (SUs) of 32 nodes each. Each 400 Gbps NDR InfiniBand link carries $400/8 = 50$ GB/s. We check the bandwidth available at each level of the fat tree and show that every level supplies exactly 400 GB/s per node, so the fabric is fully non-blocking (full bisection bandwidth).

Node to leaf (link bandwidth): each node has 8 links of 400 Gbps to the leaf switches, so

$$8 \times 50 \text{ GB/s} = 400 \text{ GB/s per node.}$$

Leaf level (switch bandwidth): each leaf switch is a 64-port switch with $64 \times 50 \text{ GB/s} = 3.2 \text{ TB/s}$ of capacity, of which 32 ports face down toward the nodes. An SU has 8 leaf switches serving its 32 nodes, so the down-bandwidth per node is

$$\frac{8 \times (32 \times 50 \text{ GB/s})}{32 \text{ nodes}} = \frac{8 \times 1.6 \text{ TB/s}}{32} = 400 \text{ GB/s per node.}$$

Spine level: each SU connects up to the 16 spine switches with $8 \times 16 \times 2$ links of 400 Gbps (2 links from each of the 8 leaf switches to each of the 16 spine switches), giving

$$\frac{8 \times 16 \times 2 \times 50 \text{ GB/s}}{32 \text{ nodes}} = \frac{12.8 \text{ TB/s}}{32} = 400 \text{ GB/s per node.}$$

Equivalently, the 16 spine switches supply $16 \times 3.2 \text{ TB/s} = 51.2 \text{ TB/s}$, which spread over all 128 nodes is again $51.2 \text{ TB/s}/128 = 400 \text{ GB/s per node}$.

Every level — the node uplinks, the leaf switches, and the spine switches — is provisioned at exactly 400 GB/s per node, so bisecting the pod anywhere leaves each node its full bandwidth across the cut:

$$\boxed{400 \text{ GB/s per node at every level} \Rightarrow \text{full (non-blocking) bisection bandwidth.}}$$

Each link's capacity is chosen precisely so that no level bottlenecks another — this is what makes it a full fat tree.

Question 2 [Scaling to a larger DGX pod]

Say we wanted to train on 2048 GPUs instead of 1024. What would be the simplest/best way to modify the above DGX topology to handle this? What about 4096? *Hint: there's no single correct answer, but try to keep costs down. Keep link capacity in mind. This documentation may be helpful.*

Solution. Part (a): scaling to 2048 GPUs.

Instead of using 2 links from each leaf switch to each spine switch, reduce to 1 link. This frees up leaf ports, so we can double the number of spine switches while keeping the same total bandwidth per node (16 spine switches per 4 SUs in the original configuration).

Part (b): scaling to 4096 GPUs.

Here we add one more layer to the tree (a top / "core" level). With 4096 GPUs we have

$$4096 \text{ GPUs} \Rightarrow 16 \text{ SUs} \Rightarrow 128 \text{ leaf switches.}$$

The upward-facing bandwidth needs

$$64 \text{ spines} \times 2 = 128 \quad (\text{upward-facing links}).$$

For the downward-facing side, we need 400 GB/s per node for 512 nodes:

$$512 \times 400 \text{ GB/s} = 204.8 \text{ TB/s at this level.}$$

With 50 GB/s per cable,

$$\frac{204.8 \text{ TB/s}}{50 \text{ GB/s per cable}} = 4096 \text{ cables,} \quad \frac{4096}{64} \approx 64 \text{ switches.}$$

$128 \text{ spines,} \quad 64 \text{ higher-level (core) switches}$

Pop Quiz 1 [AllGather time]

Using an 8xH100 node with 450 GB/s full-duplex bandwidth, how long does AllGather(bf16[BX, F]) take? Let $B = 1024$, $F = 16,384$.

Solution. With $B = 1024$, $F = 16,384$ and 450 GB/s bandwidth,

$$T_{\text{comms}} \approx \frac{2 \cdot 1024 \cdot 16384}{450 \times 10^9} \approx \boxed{75 \mu\text{s}}.$$

Pop Quiz 2 [AllToAll time]

Using an 8xH100 node with 450 GB/s unidirectional bandwidth, how long does AllToAllX->N(bf16[BX, N]) take? What if we know only 4 of 8 entries will be non-zero?

Solution. Dense case. The total volume moved is $V = 2BN$ bytes, so

$$T \approx \frac{V}{WN}.$$

Sparse case (only half the entries non-zero, i.e. $k/N = \frac{1}{2}$):

$$\frac{N-1}{N} \cdot \frac{k}{N} \cdot \frac{V}{WN} \approx \frac{kV}{WN^2} = \boxed{\frac{1}{2} \cdot \frac{V}{WN}},$$

roughly half the dense cost.

Pop Quiz 3 [Sharding along 2 axes]

Say we want to perform $\text{AllGather}_X(\text{bf16}[D_X, F_Y])$ where Y is the inner axis over a single SU (256 chips). How long will this take as a function of D , F , and Y ?

Solution. Case $Y \leq 8$ (32 nodes, still bounded by the leaf switch):

$$T_{\text{comms}} = \frac{2DF}{32 \cdot 400\text{e9}}$$

Case $Y > 8$:

$$T_{\text{comms}} = \frac{2DF}{12.8\text{e12}} \cdot \frac{256}{Y} = \frac{DF}{Y \cdot 2.5\text{e10}}$$

Question 1 [SU AllGather]

Consider only a single SU with M nodes and N GPUs per node. Precisely how many bytes are ingressed and egressed by the node level switch during an AllGather? What about the top-level switch?

Solution. Node-level switch.

Each GPU sends $\frac{B}{MN}$ bytes $\Rightarrow$ total ingress $\frac{B}{M}$;

this propagates up to the spine;

we get $\frac{B(M-1)}{M}$ back from the spine switch;

we egress $N \left(B - \frac{B}{MN} \right) = NB - \frac{B}{M}$ bytes.

So for the node-level switch:

$$NB \text{ total bytes egress, } B \text{ bytes ingress}$$

Spine switch. We ingress $\frac{B}{M}$ a total of M times (B bytes), and egress $\frac{B(M-1)}{M}$ a total of M times:

$$B(M-1) \text{ egress}$$

Question 2 [Single-node SHARP AR]

Consider a single node with N GPUs per node. Precisely how many bytes are ingressed and egressed by the switch during an AllReduce using SHARP (in-network reductions)?

Solution. Each GPU sends $\frac{B(N-1)}{N}$ to the switch, giving $B(N-1)$ total ingress.

The switch computes, then returns

$$N \cdot \frac{B}{N} = B \text{ bytes } \left(\frac{B}{N} \text{ to each GPU} \right) \text{ egress.}$$

Each GPU sends $\frac{B}{N}$ back to the switch, giving B total ingress.

The switch then sends $\frac{B(N-1)}{N}$ to each GPU (egress).

Totals:

$$\text{ingress : } B(N-1) + B = BN, \quad \text{egress : } B(N-1) + B = BN$$

Question 3 [Cross-node SHARP AR]

Consider an array $\text{bf16}[\text{DX}, \text{FY}]$ sharded over a single node of N GPUs. How long does $\text{AllReduce}(\text{bf16}[\text{D}, \text{FY}] \{ \text{UX} \})$ take? You can assume we do in-network reductions. Explain how this differs if we have more than a single node?

Solution. Total bytes = $\frac{NB}{Y}$ with $B = 2DF$, and the link bandwidth is $N \cdot W$. So the total time is

$$\frac{NB}{Y} \cdot \frac{1}{NW} = \boxed{\frac{2DF}{YW}}.$$

Question 4 [Spine level AR cost]

Consider the same setting as above, but with $Y = 256$ (so the AR happens at the spine level). How long does the AllReduce take? Again, feel free to assume in-network reductions.

Solution. The spine has 51.2 TB/s, so

$$W_{\text{SU}} = \frac{51.2}{4} \text{ TB/s} = 12.8 \text{ TB/s}.$$

Hence

$$\boxed{\text{time} = \frac{2DF}{12.8 \text{ TB/s}}}.$$

Question 5 [2-way AllGather cost]

Calculate the precise cost of an AllGather of B bytes over exactly 2 nodes. *Make sure to calculate the precise cost and not the approximation, and consider both the intra-node and cross-node cost.*

Solution. Intra-node cost:

$$\frac{7B}{8 \cdot 450\text{e}9}.$$

Leaf (cross-node) cost:

$$\frac{B}{2 \cdot 400\text{e}9}.$$

The overall cost is the max, which is the intra-node term:

$$\boxed{\frac{7B}{8 \cdot 450\text{e}9} = \frac{B}{514\text{e}9}}.$$

Question 1 [B200 rooflines]

A B200 DGX SuperPod (**not GB200 NVL72**) has 2x the bandwidth within a node (900GB/s egress) but the same amount of bandwidth in the scale-out network (400GB/s) (source). The total FLOPs are reported above. How does this change the model and data parallel rooflines?

Solution. We use the following rooflines, where C is the matmul FLOPs/s and W_{col} the relevant bandwidth:

$$\text{DP: } \frac{B}{X} > \frac{C}{W_{\text{col}}}, \quad \text{TP: } Y < \frac{F \cdot W_{\text{col}}}{C}.$$

For a B200 we have $C = 2250\text{e}12$ FLOPs/s.

Intra-node ($W_{col} = 900\text{e}9$):

$$\frac{B}{X} > \frac{C}{W_{col}} = \frac{2250\text{e}12}{900\text{e}9} = 2500,$$

so to be compute-bound the per-GPU batch must be ≥ 2500 , and for tensor parallelism

$$Y < \frac{F}{2500}.$$

Inter-node ($W_{col} = 400\text{e}9$):

$$\text{DP: } \frac{2250\text{e}12}{400\text{e}9} = 5625,$$

so we need $B \geq 5625$, and

$$Y < \frac{F}{5625}.$$

The unchanged scale-out bandwidth makes it harder to be compute-bound: the critical data-parallel batch size rises to 5625.

Question 2 [How to shard LLaMA-3 70B]

Consider LLaMA-3 70B, training in bfloat16 with fp32 optimizer state with Adam.

1. At a minimum, how many H100s would we need simply to store the weights and optimizer?
2. Say we want to train on 4096 H100 GPUs for 15T tokens. Say we achieved 45% MFU (Model FLOPs Utilization). How long would it take to train?
3. LLaMA-3 70B has $F = 28,672$ and was trained with a batch size of about 4M tokens. What is the most model parallelism we could do without being comms-bound? With this plus pure DP, could we train LLaMA-3 while staying compute-bound on 4k chips? What about ZeRO-3? What about with 8-way pipelining? *Note: consider both the communication cost and GPU memory usage.*

Solution. Part 1. The weights plus Adam optimizer state need $2 + 8 = 10$ bytes per parameter, so $70\text{B} \cdot 10 = 700$ GB total. Each GPU has 80 GB, so we need a total of 9 GPUs, i.e.

$$\boxed{2 \text{ H100 nodes.}}$$

Part 2. Training requires $6ND$ FLOPs:

$$T = \frac{6 \cdot 70\text{e}9 \cdot 15\text{e}12}{4096 \cdot 990\text{e}12 \cdot 0.45} = 3.45\text{e}6 \text{ seconds} \approx \boxed{40 \text{ days.}}$$

Part 3.

(a) TP roofline: within a node $W_{col} = 450$ GB/s and $C = 990\text{e}12$, giving a critical intensity

$$\frac{990\text{e}12}{450\text{e}9} \approx 2200.$$

Thus $Y < \frac{28672}{2200} \approx 13$, so the maximum is $\boxed{8\text{-way TP}}$ (staying within one node).

(b) Pure DP would then require 512-way DP. Each device would need to hold $\frac{700 \text{ GB}}{8} = 87.5 \text{ GB} > 80 \text{ GB}$. We can't do this — not enough GPU memory.

(c) With ZeRO-3, the 512-way memory is not an issue. But the DP roofline is $\frac{C}{W} \approx 2475$ since $W = 400 \text{ GB/s}$, and the per-device batch is

$$\frac{4\text{e6}}{4096} \approx 977 < 2475,$$

so we will be communications bound.

(d) With 8-way pipelining we have $8\times$ the bandwidth effectively, so the critical batch size drops to $\frac{2475}{8} < 977$. Thus we can be compute bound.

Question 3 [Megatron-LM hyperparams]

Consider this figure from the Megatron-LM repository highlighting their high MFU numbers.

Note that their sequence length is 4096 everywhere. For the 16B, 70B, and 314B models, what is the per-GPU token batch size? Assuming data parallelism is the outermost axis and assuming bfloat16 reductions, determine whether each of these is theoretically compute-bound or communication-bound, and whether there is a more optimal configuration available?

Solution. Per-GPU token batch size (sequence length 4096):

16B: $BS = 192$, GPUs = 192, so

$$\text{toks/GPU} = \frac{192 \cdot 4096}{192} = 4096.$$

70B: $BS = 384$, GPUs = 768, so

$$\text{toks/GPU} = \frac{384 \cdot 4096}{768} = 2048.$$

314B: $BS = 1536$, GPUs = 3072, so

$$\text{toks/GPU} = \frac{1536 \cdot 4096}{3072} = 2048.$$

Compute- vs. comms-bound (critical intensity ≈ 2475):

16B: $4096 > 2475$	$\Rightarrow$ compute bound,
70B: $2048 > \frac{2475}{2}$	$\Rightarrow$ compute bound (2-way PP),
314B: $2048 > \frac{2475}{8}$	$\Rightarrow$ compute bound (8-way PP).

A Notebook: Chapter 3, Problem 8 (ICI collective benchmark)

tpu-comms-c3-p8

July 16, 2026

```
[2]: import numpy as np
import pandas as pd

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
[3]: !pip uninstall -y jax jaxlib
!pip install -U jax[tpu] -f https://storage.googleapis.com/jax-releases/
↳ libtpu_releases.html
```

Found existing installation: jax 0.10.1

Uninstalling jax-0.10.1:

Successfully uninstalled jax-0.10.1

Found existing installation: jaxlib 0.10.1

Uninstalling jaxlib-0.10.1:

Successfully uninstalled jaxlib-0.10.1

WARNING: Running pip as the 'root' user can result in broken permissions

and conflicting behaviour with the system package manager, possibly rendering

your system unusable. It is recommended to use a virtual environment instead:

<https://pip.pypa.io/warnings/venv>. Use the --root-user-action option if you know what you are doing and want to suppress this warning.

Looking in links: https://storage.googleapis.com/jax-releases/libtpu_releases.html

Collecting jax[tpu]

Downloading jax-0.10.1-py3-none-any.whl.metadata (13 kB)

Collecting jaxlib<=0.10.1,>=0.10.1 (from jax[tpu])

Downloading jaxlib-0.10.1-cp312-cp312-manylinux_2_27_x86_64.whl.metadata (1.3 kB)

Requirement already satisfied: ml_dtypes>=0.5.0 in

/usr/local/lib/python3.12/site-packages (from jax[tpu]) (0.5.4)

```

Requirement already satisfied: numpy>=2.0 in /usr/local/lib/python3.12/site-
packages (from jax[tpu]) (2.4.6)
Requirement already satisfied: opt_einsum in /usr/local/lib/python3.12/site-
packages (from jax[tpu]) (3.4.0)
Requirement already satisfied: scipy>=1.14 in /usr/local/lib/python3.12/site-
packages (from jax[tpu]) (1.17.1)
Collecting libtpu==0.0.41.* (from jax[tpu])
  Downloading libtpu-0.0.41-cp312-cp312-manylinux_2_31_x86_64.whl.metadata (1.2
kB)
Requirement already satisfied: requests in /usr/local/lib/python3.12/site-
packages (from jax[tpu]) (2.34.2)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/site-packages (from requests->jax[tpu]) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/site-
packages (from requests->jax[tpu]) (3.16)
Requirement already satisfied: urllib3<3,>=1.26 in
/usr/local/lib/python3.12/site-packages (from requests->jax[tpu]) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in
/usr/local/lib/python3.12/site-packages (from requests->jax[tpu]) (2026.5.20)
Downloading libtpu-0.0.41-cp312-cp312-manylinux_2_31_x86_64.whl (211.8 MB)
    211.8/211.8 MB
76.2 MB/s eta 0:00:0000:0100:01
Downloading jaxlib-0.10.1-cp312-cp312-manylinux_2_27_x86_64.whl (85.8 MB)
    85.8/85.8 MB
20.6 MB/s eta 0:00:00:0100:01
Downloading jax-0.10.1-py3-none-any.whl (3.2 MB)
    3.2/3.2 MB
15.8 MB/s eta 0:00:00
Installing collected packages: libtpu, jaxlib, jax
  Attempting uninstall: libtpu
    Found existing installation: libtpu 0.0.17
    Uninstalling libtpu-0.0.17:
      Successfully uninstalled libtpu-0.0.17
Successfully installed jax-0.10.1 jaxlib-0.10.1 libtpu-0.0.41
WARNING: Running pip as the 'root' user can result in broken permissions
and conflicting behaviour with the system package manager, possibly rendering
your system unusable. It is recommended to use a virtual environment instead:
https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know
what you are doing and want to suppress this warning.

[notice] A new release of pip is
available: 25.0.1 -> 26.1.2
[notice] To update, run:
pip install --upgrade pip

```



```
[4]: import os
# os.environ["XLA_FLAGS"] = '--xla_force_host_platform_device_count=8' # Use 8
# CPU devices

from functools import partial
import jax
import jax.numpy as jnp
from jax.sharding import Mesh, PartitionSpec as P
Explicit = jax.sharding.AxisType.Explicit
Auto = jax.sharding.AxisType.Auto

jax.__version__
```

```
/usr/local/lib/python3.12/site-packages/jax/_src/cloud_tpu_init.py:88:
UserWarning: Transparent hugepages are not enabled. TPU runtime startup and
shutdown time should be significantly improved on TPU v5e and newer. If not
already set, you may need to enable transparent hugepages in your VM image (sudo
sh -c "echo always > /sys/kernel/mm/transparent_hugepage/enabled")
warnings.warn(
```

```
[4]: '0.10.1'
```

```
[5]: devs = jax.devices()
print(devs[0].platform, "|", devs[0].device_kind, "| n =", jax.device_count())
print(devs)
assert devs[0].platform == "tpu", "Not on TPU - these timings are meaningless"
```

```
WARNING: Logging before InitGoogle() is written to STDERR
E0000 00:00:1780595543.167705      14 common_lib.cc:646] Could not set metric
server port: INVALID_ARGUMENT: Could not find SliceBuilder port 8471 in any of
the 0 ports provided in `tpu_process_addresses`="local"
=== Source Location Trace: ===
learning/45eac/tfrc/runtime/common_lib.cc:228
```

```
tpu | TPU v5 lite | n = 8
[TpuDevice(id=0, process_index=0, coords=(0,0,0), core_on_chip=0),
TpuDevice(id=1, process_index=0, coords=(1,0,0), core_on_chip=0),
TpuDevice(id=2, process_index=0, coords=(0,1,0), core_on_chip=0),
TpuDevice(id=3, process_index=0, coords=(1,1,0), core_on_chip=0),
TpuDevice(id=4, process_index=0, coords=(0,2,0), core_on_chip=0),
TpuDevice(id=5, process_index=0, coords=(1,2,0), core_on_chip=0),
TpuDevice(id=6, process_index=0, coords=(0,3,0), core_on_chip=0),
TpuDevice(id=7, process_index=0, coords=(1,3,0), core_on_chip=0)]
```

```
[6]: mesh = jax.make_mesh((8,), ('x',))
jax.set_mesh(mesh)

@jax.jit
```

```

@jax.shard_map(in_specs=(P('x'),), out_specs=P(None))
def all_gather(a_block):
    b = jax.lax.all_gather(a_block, axis_name='x', axis=0, tiled=True,
        ↪to="invariant")
    return b

@jax.jit
@jax.shard_map(in_specs=(P(None),), out_specs=P(None))
def all_reduce(a_block):
    a_block = a_block + jax.lax.axis_index('x').astype(a_block.dtype)
    b = jax.lax.psum(a_block, axis_name='x')
    return b

@jax.jit
@jax.shard_map(in_specs=(P(None),), out_specs=P('x'))
def reduce_scatter(a_block):
    a_block = a_block + jax.lax.axis_index('x').astype(a_block.dtype)
    b = jax.lax.psum_scatter(a_block, axis_name='x', scatter_dimension=0,
        ↪tiled=True)
    return b

@jax.jit
@jax.shard_map(in_specs=(P('x'),), out_specs=P(None, 'x'))
def all_to_all(a_block):
    b = jax.lax.all_to_all(a_block, axis_name='x', split_axis=1, concat_axis=0,
        ↪tiled=True)
    return b

def all_gather_test():
    A = jax.device_put(jnp.arange(8 * 16.).reshape(8, 16), P('x'))

    a = all_gather(A)
    return a

def all_reduce_test():
    A = jax.device_put(jnp.ones((8,8)), P(None))
    a = all_reduce(A)
    return a

def reduce_scatter_test():
    A = jax.device_put(jnp.broadcast_to(jnp.arange(8).reshape(8,1), (8,8)),
        ↪P(None))
    print(A)
    a = reduce_scatter(A)
    return a

def all_to_all_test():

```

```

    A = jax.device_put(jnp.broadcast_to(jnp.arange(8).reshape(8,1), (8,8)),  

↪P('x'))
    a = all_to_all(A)
    print(A.addressable_shards)
    return a

```

```

[7]: import time, statistics

def time_op(fn, *args, trials=5):
    for _ in range(10):
        out = fn(*args)
        jax.block_until_ready(out)
        times = []
    for _ in range(trials):
        start = time.perf_counter()
        for _ in range(10):
            out = fn(*args)
            jax.block_until_ready(out)
            times.append((time.perf_counter() - start) / 10)
    return statistics.median(times)

```

```

[8]: sizes = [2**i for i in range(3, 16)]
key = jax.random.key(42)
bf16 = jax.dtypes.bfloat16
total_bytes = []
all_gather_times = []
all_reduce_times = []
reduce_scatter_times = []
all_to_all_times = []

for size in sizes:
    total_bytes.append(2*size*size)

    # all gather
    V = jax.device_put(jax.random.normal(key, shape=(int(size), int(size)),  

↪dtype=bf16), P('x'))
    all_gather_times.append(time_op(all_gather, V))

    # all reduce
    V = jax.device_put(jax.random.normal(key, shape=(int(size), int(size)),  

↪dtype=bf16), P(None))
    all_reduce_times.append(time_op(all_reduce, V))

    # reduce scatter
    V = jax.device_put(jax.random.normal(key, shape=(int(size), int(size)),  

↪dtype=bf16), P(None))
    reduce_scatter_times.append(time_op(reduce_scatter, V))

```

```

# all to all
V = jax.device_put(jax.random.normal(key, shape=(int(size), int(size)),
dtype=bf16), P('x'))
all_to_all_times.append(time_op(all_to_all, V))

```

```

[9]: import numpy as np, matplotlib.pyplot as plt

V = np.array(total_bytes, dtype=float)
ag = np.array(all_gather_times)
ar = np.array(all_reduce_times)
rs = np.array(reduce_scatter_times)
a2a = np.array(all_to_all_times)

def bw(t, factor):
    return factor * V / t / 1e9

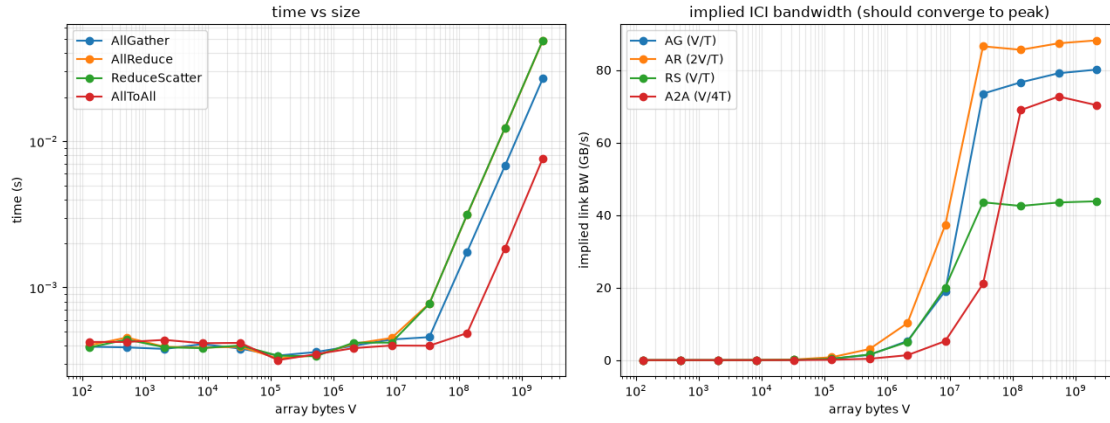
fig, (axT, axBW) = plt.subplots(1, 2, figsize=(13, 5))

for t, lab in [(ag, 'AllGather'), (ar, 'AllReduce'), (rs, 'ReduceScatter'),
               (a2a, 'AllToAll')]:
    axT.loglog(V, t, 'o-', label=lab)
axT.set(xlabel='array bytes V', ylabel='time (s)', title='time vs size')
axT.legend(); axT.grid(True, which='both', alpha=.3)

for t, f, lab in [(ag, 1, 'AG (V/T)'), (ar, 2, 'AR (2V/T)'), (rs, 1, 'RS (V/T)'),
                  (a2a, 0.25, 'A2A (V/4T)')]:
    axBW.semilogx(V, bw(t, f), 'o-', label=lab)
axBW.set(xlabel='array bytes V', ylabel='implied link BW (GB/s)',
         title='implied ICI bandwidth (should converge to peak)')
axBW.legend(); axBW.grid(True, which='both', alpha=.3)
plt.tight_layout(); plt.show()

print(f"at V={V[-1]/1e6:.0f} MB: "
      f"AR/AG={ar[-1]/ag[-1]:.2f} (expected ~2) "
      f"RS/AG={rs[-1]/ag[-1]:.2f} (expected ~1) "
      f"A2A/AG={a2a[-1]/ag[-1]:.2f} (expected ~0.25)")

```



at $V=2147$ MB: $AR/AG=1.82$ (expected ~ 2) $RS/AG=1.83$ (expected ~ 1) $A2A/AG=0.28$ (expected ~ 0.25)

```
[36]: total_bytes[-1] / ag[-1] / 1e9
```

```
[36]: np.float64(80.01507774991205)
```

B Notebook: Chapter 10 (Programming TPUs in JAX)

c10q123-new

July 16, 2026

1 Chapter 10 — Kaggle TPU template

```
[1]: SIMULATE_CPU = False
N_DEVICES = 8

import os
if SIMULATE_CPU:
    os.environ["JAX_PLATFORMS"] = "cpu"
    os.environ["XLA_FLAGS"] = (
        os.environ.get("XLA_FLAGS", "")
        + f" --xla_force_host_platform_device_count={N_DEVICES}"
    )

import jax
import jax.numpy as jnp

print("backend :", jax.default_backend())
print("devices :", jax.device_count())
for d in jax.devices():
    print("    ", d)
assert jax.device_count() == N_DEVICES, (
    f"expected {N_DEVICES} devices, got {jax.device_count()}. "
    "If you just toggled SIMULATE_CPU, restart the kernel. "
    "If profiling, check the TPU v5e-8 accelerator is attached."
)
```

/usr/local/lib/python3.12/site-packages/jax/_src/cloud_tpu_init.py:88:

UserWarning: Transparent hugepages are not enabled. TPU runtime startup and shutdown time should be significantly improved on TPU v5e and newer. If not already set, you may need to enable transparent hugepages in your VM image (sudo sh -c "echo always > /sys/kernel/mm/transparent_hugepage/enabled")

warnings.warn(

WARNING: Logging before InitGoogle() is written to STDERR

E0000 00:00:1784005966.220777 2176 common_lib.cc:648] Could not set metric

server port: INVALID_ARGUMENT: Could not find SliceBuilder port 8471 in any of the 0 ports provided in `tpu\_process\_addresses`="local"

=== Source Location Trace: ===

learning/45eac/tfrc/runtime/common_lib.cc:238

```

backend : tpu
devices : 8
    TPU_0(process=0,(0,0,0,0))
    TPU_1(process=0,(1,0,0,0))
    TPU_2(process=0,(0,1,0,0))
    TPU_3(process=0,(1,1,0,0))
    TPU_4(process=0,(0,2,0,0))
    TPU_5(process=0,(1,2,0,0))
    TPU_6(process=0,(0,3,0,0))
    TPU_7(process=0,(1,3,0,0))

```

```

[2]: # ---- imports + partitioning helpers -----
import time, shutil, functools
from functools import partial
import numpy as np

import jax.sharding as shd
from jax.sharding import Mesh, NamedSharding, PartitionSpec as P
from jax.experimental import mesh_utils

# shard_map moved around across jax versions; try the common locations.
try:
    from jax.experimental.shard_map import shard_map
except ImportError:
    shard_map = jax.shard_map          # newer jax exposes it top-level

def make_mesh(shape=(N_DEVICES,), axis_names=("X",)):
    """Mesh over all devices, e.g. make_mesh((8,), ('X',)) or make_mesh((2,4),
    ↪ ('X', 'Y'))."""
    devices = mesh_utils.create_device_mesh(shape)
    return Mesh(devices, axis_names)

mesh = jax.make_mesh((2, 4), ('x', 'y'), (shd.AxisType.Explicit, shd.AxisType.
    ↪Explicit))
jax.set_mesh(mesh)

```

```

/tmp/ipykernel_2176/4193610154.py:12: DeprecationWarning:
jax.experimental.shard_map is deprecated in v0.8.0. Used jax.shard_map instead.
    from jax.experimental.shard_map import shard_map

```

```

[2]: <jax._src.sharding_impls.set_mesh at 0x7a5426fa70a0>

```

```

[3]: # ---- timing helper -----
def benchmark(fn, *args, n_iters=50, warmup=3, name=None):
    """Time a callable with correct blocking. Returns ms/iter."""
    for _ in range(warmup):
        out = fn(*args)

```



```

jax.block_until_ready(out)
t0 = time.perf_counter()
for _ in range(n_iters):
    out = fn(*args)
jax.block_until_ready(out)
ms = (time.perf_counter() - t0) / n_iters * 1e3
print(f"{{(name or getattr(fn, '__name__', 'fn')):>24}}: {{ms:8.3f}} ms/iter _
↳({n_iters} iters)")
return ms

```

```

[4]: # ---- profile capture + save (downloadable from Kaggle 'Output') -----
from IPython.display import FileLink, display

WORKDIR      = "/kaggle/working" if os.path.isdir("/kaggle/working") else "."
PROFILE_ROOT = os.path.join(WORKDIR, "profiles")

def profile_and_save(fn, *args, name="trace", n_iters=20, warmup=3):
    """Run fn under the JAX profiler; save + zip the trace to /kaggle/working/
    ↳profiles/<name>.zip.
    Download it from the Kaggle Output panel, then open in TensorBoard / trace_
    ↳viewer."""
    logdir = os.path.join(PROFILE_ROOT, name)
    if os.path.exists(logdir):
        shutil.rmtree(logdir)
    os.makedirs(logdir, exist_ok=True)

    for _ in range(warmup):
        out = fn(*args)
        jax.block_until_ready(out)

    with jax.profiler.trace(logdir):
        for _ in range(n_iters):
            out = fn(*args)
            jax.block_until_ready(out)

    archive = shutil.make_archive(logdir, "zip", logdir)
    print("trace dir :", logdir)
    print("zip saved :", archive)
    print("-> grab it from the Kaggle 'Output' panel (right side), then open_
    ↳locally.")
    try:
        display(FileLink(os.path.relpath(archive, WORKDIR)))
    except Exception:
        pass
    return archive

```

1.1 Q1

1.1.1 Part 1

```
[5]: W = jnp.arange(2048*8192, dtype=jnp.float32).reshape(2048, 8192)
W = jax.device_put(W, jax.NamedSharding(mesh, jax.P('x', 'y')))

def avg(x):
    X, Y = mesh.axis_sizes
    x = x.reshape(X, x.shape[0] // X, Y, x.shape[1] // Y,
                  out_sharding=jax.NamedSharding(mesh, jax.P('x', None, 'y', None)))
    return jnp.mean(x, axis=(1, 3))

jit_avg = jax.jit(avg, out_shardings=jax.NamedSharding(mesh, jax.P('x', 'y')))

@jax.shard_map(in_specs=jax.P('x', 'y'), out_specs=jax.P('x', 'y'))
def shard_avg(x):
    return jnp.mean(x, keepdims=True)

shard_avg = jax.jit(shard_avg)
y = shard_avg(W)
z = jit_avg(W)

# profile_and_save(jit_avg, W, name='jit_avg')

# profile_and_save(shard_avg, W, name='shard_avg')
```

```
[6]: q1_W = jax.device_put(jnp.arange(2048 * 8192, dtype=jnp.float32).reshape(2048, 8192),
                             NamedSharding(mesh, P('x', 'y'))))

benchmark(jit_avg, q1_W, name='jit_avg')
benchmark(shard_avg, q1_W, name='shard_avg')

profile_and_save(jit_avg, q1_W, name='q1p1_jit_avg')
profile_and_save(shard_avg, q1_W, name='q1p1_shard_avg')
```

```
jit_avg: 0.297 ms/iter (50 iters)
shard_avg: 0.286 ms/iter (50 iters)
trace dir : /kaggle/working/profiles/q1p1_jit_avg
zip saved : /kaggle/working/profiles/q1p1_jit_avg.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q1p1_jit_avg.zip

trace dir : /kaggle/working/profiles/q1p1_shard_avg
zip saved : /kaggle/working/profiles/q1p1_shard_avg.zip
```

-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q1p1_shard_avg.zip

```
[6]: '/kaggle/working/profiles/q1p1_shard_avg.zip'
```

1.1.2 Part 2

```
[7]: @jax.shard_map(in_specs=(P('x', 'y'),), out_specs=P('x', 'y'))
def local_roll(x, shift):
    return jnp.roll(x, shift, axis=0)
```

1.2 Q2

1.2.1 Part 1

```
[8]: S = 8
D = 1024
F = 4096
E = 4

@jax.sharding.auto_axes(out_sharding=P())
def ragged_dot(x, w, g):
    return jax.lax.ragged_dot(x, w, g)

def moe_layer(w, a, b):
    s, d = a.shape
    e, _, f = w.shape
    indices = jnp.argsort(b)
    group_sizes = jnp.bincount(b, length=e)
    x = a[indices]

    out = ragged_dot(
        x, w, group_sizes
    )

    unperm = jnp.argsort(indices)
    out = out[unperm]
    return out

W = jnp.zeros((E, D, F))
A = jnp.zeros((S, D))
B = jnp.zeros((S,)).astype(jnp.int32)

out = moe_layer(W, A, B)
```

1.3 Part 2

```
[9]: A_shard = jax.NamedSharding(mesh, P('x', None))
B_shard = jax.NamedSharding(mesh, P('x'))
W_shard = jax.NamedSharding(mesh, P('x', None, None))

A = jax.device_put(A, A_shard)
B = jax.device_put(B, B_shard)
W = jax.device_put(W, W_shard)

moe_auto = jax.sharding.auto_axes(moe_layer, out_sharding=P('x', None))

jitted_moe = jax.jit(
    moe_auto,
    in_shardings=(W_shard, A_shard, B_shard),
    out_shardings=(A_shard), # output should be (S_X, D)
)

out = jitted_moe(W, A, B)

import re, collections

hlo = jitted_moe.lower(W, A, B).compile().as_text()
print(collections.Counter(re.
    ↪findall(r'\b(all-gather|all-reduce|reduce-scatter|collective-permute|all-to-all)\b',
    ↪hlo)))

# profile_and_save(shard_avg, W, A, B, name='jitted_moe')
```

```
Counter({'all-reduce': 10, 'collective-permute': 6, 'all-gather': 4})
```

```
[10]: import re

pat = re.compile(
    r'%[\w.\-]+ = (\S+?) ' # output shape
    r'(all-gather|all-reduce|reduce-scatter|collective-permute|all-to-all)\(')
for line in hlo.splitlines():
    s = line.strip()
    m = pat.match(s)
    if not m:
        continue
    shape = m.group(1)
    op = m.group(2)
    opname = re.search(r'op_name="([^\"]*)"', s)
    srcline = re.search(r'source_line=(\d+)', s)
    print(f"{op:18} out={shape:22} line={srcline.group(1) if srcline else '?':
    ↪>3} "
        f"{opname.group(1) if opname else '?'}")
```

```

all-reduce          out=s32[2]{0:T(128)S(1)}   line= 15  jit(moe_layer)/scatter-
add
all-reduce          out=f32[8,1024]{1,0:T(8,128)S(1)} line= 16
jit(moe_layer)/gather
all-gather          out=f32[4,1024,4096]{2,1,0:T(8,128)S(1)} line=  ?  w

```

```

[11]: benchmark(jitted_moe, W, A, B, name='jitted_moe')
      profile_and_save(jitted_moe, W, A, B, name='q2p2_jitted_moe')

```

```

          jitted_moe:    1.551 ms/iter  (50 iters)
trace dir : /kaggle/working/profiles/q2p2_jitted_moe
zip saved : /kaggle/working/profiles/q2p2_jitted_moe.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q2p2_jitted_moe.zip

```

```

[11]: '/kaggle/working/profiles/q2p2_jitted_moe.zip'

```

1.3.1 Part 3

```

[12]: # reimplement with shardmap
      S = 32
      D = 1024
      F = 4096
      E = 8

      W = jnp.zeros((E, D, F))
      A = jnp.zeros((S,D))
      B = jnp.arange(S) % E

      A_shard = jax.NamedSharding(mesh, P('x', None))
      B_shard = jax.NamedSharding(mesh, P('x'))
      W_shard = jax.NamedSharding(mesh, P('x', None, None))

      A = jax.device_put(A, A_shard)
      B = jax.device_put(B, B_shard)
      W = jax.device_put(W, W_shard)

      @jax.shard_map(in_specs=(P('x', None, None), P('x', None), P('x')),
                    out_specs=P('x', None))
      def moe_explicit(w, a, b):
          d = a.shape[1]
          f = w.shape[-1]
          num_local_experts = w.shape[0]
          device_num = jax.lax.axis_index('x')
          num_devices = jax.lax.axis_size('x')
          num_total_experts = num_devices*num_local_experts

```

```

expert_allocation = b
device_allocation = b // num_local_experts

perm = jnp.argsort(expert_allocation)
a_sorted = a[perm]

expert_allocation_sorted = b[perm]
counts = jnp.bincount(device_allocation, length=num_devices)
offsets = jnp.cumsum(counts) - counts
progress = jnp.zeros(num_devices).astype(jnp.int32)
batch_size = a.shape[0] // num_devices
out_acc = jnp.zeros((a.shape[0], f))
out_acc = jax.lax.pcast(out_acc, ('x',), to='varying')

def cond(val):
    out_acc, prog = val
    local_left = jnp.any(prog < counts)
    global_left = jax.lax.psum(local_left.astype(jnp.int32), axis_name='x')
    return global_left > 0

def body(val):
    out_acc, prog = val

    j = jnp.arange(batch_size)
    idx = offsets[:, None] + prog[:, None] + j[None, :]
    valid = (prog[:, None] + j[None, :]) < counts[:, None]
    safe_idx = jnp.where(valid, idx, 0)
    X = jnp.where(valid[:, :, None], a_sorted[safe_idx], 0.0)
    X_labels = jnp.where(valid, expert_allocation_sorted[safe_idx], -1)
    recv_X = jax.lax.all_to_all(X, axis_name='x', split_axis=0,
    ↪concat_axis=0, tiled=False)
    recv_labels = jax.lax.all_to_all(X_labels, axis_name='x', split_axis=0,
    ↪concat_axis=0, tiled=False)

    recv_X = recv_X.reshape((num_devices*batch_size, d))
    recv_labels = recv_labels.reshape((num_devices*batch_size,))
    recv_labels = jnp.where(recv_labels == -1, num_local_experts,
    ↪recv_labels - device_num*num_local_experts)

    perm2 = jnp.argsort(recv_labels)
    recv_X_sorted = recv_X[perm2]
    group_sizes = jnp.bincount(recv_labels, length=num_local_experts)

    out = jax.lax.ragged_dot(recv_X_sorted, w, group_sizes)

```

```

unperm2 = jnp.argsort(perm2)
out = out[unperm2] # (S, F)

out = out.reshape((num_devices, batch_size, f))
out = jax.lax.all_to_all(out, axis_name='x', split_axis=0,
↪concat_axis=0, tiled=False)

out_acc = out_acc.at[safe_idx].add(jnp.where(valid[:, :, None], out, 0.
↪0))

prog += batch_size

return out_acc, prog

out_acc, _ = jax.lax.while_loop(cond, body, (out_acc, progress))
out_acc = out_acc[jnp.argsort(perm)]
return out_acc

out = moe_explicit(W, A, B)

```

```

[13]: import numpy as np

def moe_reference(W, A, B):
    """Ground truth: out[i] = A[i] @ W[B[i]]."""
    return jnp.einsum('sd,sdf->sf', A, W[B]) # W[B]: [S,D,F]

def compare(S=32, D=1024, F=4096, E=8, seed=0, balanced=True, atol=1e-2):
    kW, kA, kB = jax.random.split(jax.random.PRNGKey(seed), 3)
    W = jax.random.normal(kW, (E, D, F), jnp.float32) * 0.02
    A = jax.random.normal(kA, (S, D), jnp.float32)
    B = (jnp.arange(S) % E) if balanced else jax.random.randint(kB, (S,), 0, E)

    ref = np.asarray(moe_reference(W, A, B))

    Wsh = jax.device_put(W, jax.NamedSharding(mesh, P('x', None, None)))
    Ash = jax.device_put(A, jax.NamedSharding(mesh, P('x', None)))
    Bsh = jax.device_put(B, jax.NamedSharding(mesh, P('x')))
    got = np.asarray(moe_explicit(Wsh, Ash, Bsh))

    err = np.abs(got - ref)
    ok = np.allclose(got, ref, atol=atol, rtol=atol)
    print(f"balanced={balanced} match={ok} max_abs_err={err.max():.2e}
↪mean={err.mean():.2e}")
    if not ok:
        bad = np.where(~np.isclose(got, ref, atol=atol, rtol=atol).
↪all(axis=1))[0]

```

```

        print(f" {len(bad)} mismatched token rows, first few: {bad[:10]}.
↪tolist()}")
        return ok

compare(balanced=True)      # start here - capacity fits in ~1 round, should be
↪exact
compare(balanced=False)    # then imbalanced - exercises the multi-round
↪while_loop

```

```

balanced=True  match=True  max_abs_err=0.00e+00  mean=0.00e+00
balanced=False match=True  max_abs_err=0.00e+00  mean=0.00e+00

```

[13]: True

```

[14]: mS, mD, mF, mE = 512, 1024, 4096, 8
      _kw, _ka = jax.random.split(jax.random.PRNGKey(20), 2)
      moe_W = jax.device_put(jax.random.normal(_kw, (mE, mD, mF), jnp.float32) * 0.
↪02, NamedSharding(mesh, P("x", None, None)))
      moe_A = jax.device_put(jax.random.normal(_ka, (mS, mD), jnp.float32),
↪ NamedSharding(mesh, P("x", None)))
      moe_B = jax.device_put((jnp.arange(mS) % mE).astype(jnp.int32),
↪ NamedSharding(mesh, P("x")))

      moe_explicit_jit = jax.jit(moe_explicit)

      benchmark(jitted_moe,      moe_W, moe_A, moe_B, name="moe_jit")
      benchmark(moe_explicit_jit, moe_W, moe_A, moe_B, name="moe_explicit")

      profile_and_save(jitted_moe,      moe_W, moe_A, moe_B, name="q2p3_moe_jit")
      profile_and_save(moe_explicit_jit, moe_W, moe_A, moe_B,
↪name="q2p3_moe_explicit")

```

```

            moe_jit:      3.710 ms/iter  (50 iters)
            moe_explicit:  0.436 ms/iter  (50 iters)
trace dir : /kaggle/working/profiles/q2p3_moe_jit
zip saved : /kaggle/working/profiles/q2p3_moe_jit.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q2p3_moe_jit.zip

trace dir : /kaggle/working/profiles/q2p3_moe_explicit
zip saved : /kaggle/working/profiles/q2p3_moe_explicit.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q2p3_moe_explicit.zip

```

[14]: '/kaggle/working/profiles/q2p3_moe_explicit.zip'

1.4 Part 4

Implement top-k and average the results

```
[15]: @jax.shard_map(in_specs=(P('x', None, None), P('x', None), P('x', None)),  
    ↪out_specs=P('x', None))  
def moe_topk_explicit(w, a, b):  
    d = a.shape[1]  
    f = w.shape[-1]  
    k = b.shape[-1]  
    s_x = a.shape[0]  
    num_local_experts = w.shape[0]  
    device_num = jax.lax.axis_index('x')  
    num_devices = jax.lax.axis_size('x')  
    num_total_experts = num_devices*num_local_experts  
  
    b = b.reshape((s_x*k,))  
    expert_allocation = b  
    device_allocation = b // num_local_experts  
  
    perm = jnp.argsort(expert_allocation)  
    a_sorted = a[perm // k]  
  
    expert_allocation_sorted = b[perm]  
    counts = jnp.bincount(device_allocation, length=num_devices)  
    offsets = jnp.cumsum(counts) - counts  
    progress = jnp.zeros(num_devices).astype(jnp.int32)  
    batch_size = a.shape[0] // num_devices  
    out_acc = jnp.zeros((s_x*k, f))  
    out_acc = jax.lax.pcast(out_acc, ('x',), to='varying')  
  
    def cond(val):  
        out_acc, prog = val  
        local_left = jnp.any(prog < counts)  
        global_left = jax.lax.psum(local_left.astype(jnp.int32), axis_name='x')  
        return global_left > 0  
  
    def body(val):  
        out_acc, prog = val  
  
        j = jnp.arange(batch_size)  
        idx = offsets[:, None] + prog[:, None] + j[None, :]  
        valid = (prog[:, None] + j[None, :]) < counts[:, None]  
        safe_idx = jnp.where(valid, idx, 0)  
        X = jnp.where(valid[:, :, None], a_sorted[safe_idx], 0.0)  
        X_labels = jnp.where(valid, expert_allocation_sorted[safe_idx], -1)
```

```

    recv_X = jax.lax.all_to_all(X, axis_name='x', split_axis=0,
    ↪concat_axis=0, tiled=False)
    recv_labels = jax.lax.all_to_all(X_labels, axis_name='x', split_axis=0,
    ↪concat_axis=0, tiled=False)

    recv_X = recv_X.reshape((num_devices*batch_size, d))
    recv_labels = recv_labels.reshape((num_devices*batch_size,))
    recv_labels = jnp.where(recv_labels == -1, num_local_experts,
    ↪recv_labels - device_num*num_local_experts)

    perm2 = jnp.argsort(recv_labels)
    recv_X_sorted = recv_X[perm2]
    group_sizes = jnp.bincount(recv_labels, length=num_local_experts)

    out = jax.lax.ragged_dot(recv_X_sorted, w, group_sizes)

    unperm2 = jnp.argsort(perm2)
    out = out[unperm2] # (S, F)

    out = out.reshape((num_devices, batch_size, f))
    out = jax.lax.all_to_all(out, axis_name='x', split_axis=0,
    ↪concat_axis=0, tiled=False)

    out_acc = out_acc.at[safe_idx].add(jnp.where(valid[:, :, None], out, 0.
    ↪0))

    prog += batch_size

    return out_acc, prog

out_acc, _ = jax.lax.while_loop(cond, body, (out_acc, progress))
out_acc = out_acc[jnp.argsort(perm)]
out_acc = jnp.mean(out_acc.reshape((s_x, k, f)), axis=1)
return out_acc

```

```

[16]: # top k MoE test
import numpy as np
from jax.sharding import NamedSharding, PartitionSpec as P

tk_S, tk_D, tk_F, tk_E, tk_k = 32, 64, 128, 8, 2 # small dims for a fast
    ↪CPU-sim check

_kW, _kA, _kB = jax.random.split(jax.random.PRNGKey(0), 3)
tk_W = jax.random.normal(_kW, (tk_E, tk_D, tk_F), jnp.float32) * 0.02
tk_A = jax.random.normal(_kA, (tk_S, tk_D), jnp.float32)
tk_B = jax.random.randint(_kB, (tk_S, tk_k), 0, tk_E) # [S, k]

```

```

# ---- oracle: out[s] = mean_j A[s] @ W[B[s,j]] ----
tk_ref = jnp.mean(jnp.einsum('sd,skdf->skf', tk_A, tk_W[tk_B]), axis=1) # [S,␣
    ↪F]

# ---- sharded inputs ----
tk_Wsh = jax.device_put(tk_W, NamedSharding(mesh, P('x', None, None)))
tk_Ash = jax.device_put(tk_A, NamedSharding(mesh, P('x', None)))
tk_Bsh = jax.device_put(tk_B, NamedSharding(mesh, P('x', None))) # 2-D␣
    ↪now

tk_got = moe_topk_explicit(tk_Wsh, tk_Ash, tk_Bsh)

tk_ref_np, tk_got_np = np.asarray(tk_ref), np.asarray(tk_got)
tk_err = np.abs(tk_ref_np - tk_got_np)
print("shapes got:", tk_got_np.shape, " ref:", tk_ref_np.shape)
print("match:", np.allclose(tk_ref_np, tk_got_np, atol=1e-3, rtol=1e-3),
      " max_err:", f"{tk_err.max():.2e}", " mean_err:", f"{tk_err.mean():.2e}")
if not np.allclose(tk_ref_np, tk_got_np, atol=1e-3, rtol=1e-3):
    bad = np.where(~np.isclose(tk_ref_np, tk_got_np, atol=1e-3, rtol=1e-3).
    ↪all(1))[0]
    print(" mismatched token rows:", bad[:10].tolist(), f"(total {len(bad)})")

```

```

shapes got: (32, 128) ref: (32, 128)
match: False max_err: 1.25e-03 mean_err: 2.25e-04
mismatched token rows: [10, 13, 15] (total 3)

```

2 Problem 3

2.1 Part 1

```

[17]: @jax.shard_map(in_specs=(P('x', 'y'), P('y', None)), out_specs=P('x', None))
def collective_allreduce(a, w):
    y_size = jax.lax.axis_size('y')
    chunk_size = w.shape[-1] // y_size

    result = []
    for i in range(y_size):
        chunk = w[:, i*chunk_size:(i+1)*chunk_size]
        local_matmul = jnp.einsum('bd,df->bf', a, chunk)
        reduced = jax.lax.psum(local_matmul, axis_name='y')
        result.append(reduced)

    result = jnp.concatenate(result, axis=1)

    return result

```

```
[18]: B, D, F = 8, 16, 32
kA, kW = jax.random.split(jax.random.PRNGKey(0), 2)
A = jax.random.normal(kA, (B, D), jnp.float32)      # [B, D]
W = jax.random.normal(kW, (D, F), jnp.float32)      # [D, F]

# ---- shard: A[Bx, Dy], W[Dy, F] ----
A_sh = jax.device_put(A, NamedSharding(mesh, P('x', 'y'))))
W_sh = jax.device_put(W, NamedSharding(mesh, P('y', None)))

# ---- unsharded reference: Out[B, F] = A @ W ----
ref = A @ W

# ---- call your impl (rename / add args as needed) ----
out = collective_allreduce(A_sh, W_sh)              # e.g. allreduce_matmul(A_sh, W_sh, n_tiles=4)

out_np, ref_np = np.asarray(out), np.asarray(ref)
print("out:", out_np.shape, " ref:", ref_np.shape,
      " max_err:", f"{np.abs(out_np - ref_np).max():.2e}")
```

out: (8, 32) ref: (8, 32) max_err: 4.77e-07

```
[19]: pB, pD, pF = 4096, 8192, 8192
_ka, _kw = jax.random.split(jax.random.PRNGKey(10), 2)
ar_a = jax.device_put(jax.random.normal(_ka, (pB, pD), jnp.float32),
    ↪NamedSharding(mesh, P('x', 'y'))))
ar_w = jax.device_put(jax.random.normal(_kw, (pD, pF), jnp.float32),
    ↪NamedSharding(mesh, P('y', None)))

# baseline: plain matmul, let XLA insert the all-reduce
allreduce_jit = jax.jit(jax.sharding.auto_axes(
    lambda a, w: jnp.einsum('bd,df->bf', a, w), out_sharding=P('x', None)))
allreduce_collective = jax.jit(collective_allreduce)

benchmark(allreduce_jit, ar_a, ar_w, name='allreduce_jit')
benchmark(allreduce_collective, ar_a, ar_w, name='allreduce_collective')

profile_and_save(allreduce_jit, ar_a, ar_w, name='q3p1_allreduce_jit')
profile_and_save(allreduce_collective, ar_a, ar_w,
    ↪name='q3p1_allreduce_collective')
```

```
allreduce_jit: 1.565 ms/iter (50 iters)
allreduce_collective: 1.646 ms/iter (50 iters)
trace dir : /kaggle/working/profiles/q3p1_allreduce_jit
zip saved : /kaggle/working/profiles/q3p1_allreduce_jit.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q3p1_allreduce_jit.zip
```

```

trace dir : /kaggle/working/profiles/q3p1_allreduce_collective
zip saved : /kaggle/working/profiles/q3p1_allreduce_collective.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q3p1_allreduce_collective.zip

```

```
[19]: '/kaggle/working/profiles/q3p1_allreduce_collective.zip'
```

2.2 Part 2

```

[20]: @jax.shard_map(in_specs=(P('x', 'y'), P('y', None)), out_specs=P('x', 'y'))
def collective_reduce_scatter(b, w):
    y_size = jax.lax.axis_size('y')
    chunk_size = w.shape[-1] // y_size
    idx = jax.lax.axis_index('y')

    init_val = jnp.zeros((b.shape[0], chunk_size))
    init_val = jax.lax.pcast(init_val, ('x', 'y'), to='varying')

    def body(i, acc):
        chunk_idx = (idx - 1 - i) % y_size
        w_chunk = jax.lax.dynamic_slice_in_dim(w, chunk_idx*chunk_size,
        ↪ chunk_size, axis=1)
        local_matmul = jnp.einsum('bf,fd->bd', b, w_chunk)
        ret = acc + local_matmul
        perm = [(j, (j+1)%y_size) for j in range(y_size)] # send to the right
        return jax.lax.ppermute(ret, axis_name='y', perm=perm)

    out = jax.lax.fori_loop(0, y_size-1, body, init_val, unroll=True)

    last_chunk = jax.lax.dynamic_slice_in_dim(w, idx*chunk_size, chunk_size,
    ↪ axis=1)

    return out + jnp.einsum('bf,fd->bd', b, last_chunk)

```

```

[21]: import numpy as np
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

B, F, D = 8, 32, 16
kT, kW = jax.random.split(jax.random.PRNGKey(1), 2)
Tmp = jax.random.normal(kT, (B, F), jnp.float32) # [B, F]
W2 = jax.random.normal(kW, (F, D), jnp.float32) # [F, D]

Tmp_sh = jax.device_put(Tmp, NamedSharding(mesh, P('x', 'y')))
W2_sh = jax.device_put(W2, NamedSharding(mesh, P('y', None)))

```

```

ref = Tmp @ W2

out = collective_reduce_scatter(Tmp_sh, W2_sh)

out_np, ref_np = np.asarray(out), np.asarray(ref)
print("out:", out_np.shape, " ref:", ref_np.shape,
      " sharding:", out.sharding.spec,
      " max_err:", f"{np.abs(out_np - ref_np).max():.2e}")

```

out: (8, 16) ref: (8, 16) sharding: P('x', 'y') max_err: 9.54e-07

2.3 Part 3

```

[22]: # chapter's all gather collective matmul
def collective_matmul_allgather_lhs_contracting(lhs, rhs):
    # lhs is the looped operand; rhs is the local operand
    axis_size = jax.lax.axis_size('y') # axis_size = 4 for this example
    idx = jax.lax.axis_index('y')

    chunk_size = lhs.shape[1]
    assert rhs.shape[0] % chunk_size == 0

    def f(i, carries):
        accum, lhs = carries
        rhs_chunk = jax.lax.dynamic_slice_in_dim(rhs, (idx + i) % axis_size *
        ↪ chunk_size, chunk_size)
        # Matmul for a chunk
        update = lhs @ rhs_chunk
        # Circular shift to the left
        lhs = jax.lax.ppermute(
            lhs,
            axis_name='y',
            perm=[(j, (j - 1) % axis_size) for j in range(axis_size)]
        )
        return accum + update, lhs

    accum = jnp.zeros((lhs.shape[0], rhs.shape[1]), dtype=lhs.dtype)
    # accum = jax.lax.pcast(accum, ('x', 'y'), to='varying')
    accum = jax.lax.pcast(accum, ('x', 'y'), to='varying')

    accum, lhs = jax.lax.fori_loop(0, axis_size - 1, f, (accum, lhs),
    ↪ unroll=True)

    # Compute the last chunk after the final permute to leave lhs in the state
    ↪ we found it
    i = axis_size - 1

```

```

    rhs_chunk = jax.lax.dynamic_slice_in_dim(rhs, (idx + i) % axis_size * ␣
↪ chunk_size, chunk_size)
    update = lhs @ rhs_chunk
    return accum + update

# my (un-shard_mapped) reduce-scatter

def collective_reduce_scatter(b, w):
    y_size = jax.lax.axis_size('y')
    chunk_size = w.shape[-1] // y_size
    idx = jax.lax.axis_index('y')

    init_val = jnp.zeros((b.shape[0], chunk_size))
    init_val = jax.lax.pcast(init_val, ('x', 'y'), to='varying')

    def body(i, acc):
        chunk_idx = (idx - 1 - i) % y_size
        w_chunk = jax.lax.dynamic_slice_in_dim(w, chunk_idx*chunk_size, ␣
↪ chunk_size, axis=1)
        local_matmul = jnp.einsum('bf,fd->bd', b, w_chunk)
        ret = acc + local_matmul
        perm = [(j, (j+1)%y_size) for j in range(y_size)] # send to the right
        return jax.lax.ppermute(ret, axis_name='y', perm=perm)

    out = jax.lax.fori_loop(0, y_size-1, body, init_val, unroll=True)

    last_chunk = jax.lax.dynamic_slice_in_dim(w, idx*chunk_size, chunk_size, ␣
↪ axis=1)

    return out + jnp.einsum('bf,fd->bd', b, last_chunk)

@jax.shard_map(in_specs=(P('x', 'y'), P(None, 'y'), P('y', None)), ␣
↪ out_specs=P('x', 'y'))
def ffn_block(x, w_up, w_down):
    tmp = collective_matmul_allgather_lhs_contracting(x, w_up)
    return collective_reduce_scatter(tmp, w_down)

```

```

[23]: import numpy as np
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

B, D, F = 8, 16, 32
kI, kU, kD = jax.random.split(jax.random.PRNGKey(2), 3)
In  = jax.random.normal(kI, (B, D), jnp.float32)      # [B, D]
Win = jax.random.normal(kU, (D, F), jnp.float32)      # [D, F]
Wout = jax.random.normal(kD, (F, D), jnp.float32)     # [F, D]

```

```

# In[Bx, Dy], Win[D, Fy], Wout[Fy, D]
In_sh = jax.device_put(In, NamedSharding(mesh, P('x', 'y')))
Win_sh = jax.device_put(Win, NamedSharding(mesh, P(None, 'y')))
Wout_sh = jax.device_put(Wout, NamedSharding(mesh, P('y', None)))

# unsharded reference
ref = (In @ Win) @ Wout

out = ffn_block(In_sh, Win_sh, Wout_sh)

out_np, ref_np = np.asarray(out), np.asarray(ref)
print("out:", out_np.shape, " ref:", ref_np.shape,
      " sharding:", out.sharding.spec,
      " max_err:", f"{np.abs(out_np - ref_np).max():.2e}")

```

out: (8, 16) ref: (8, 16) sharding: P('x', 'y') max_err: 3.81e-06

```

[24]: pB, pD, pF = 4096, 8192, 8192
_ki, _ku, _kd = jax.random.split(jax.random.PRNGKey(12), 3)
ffn_x = jax.device_put(jax.random.normal(_ki, (pB, pD), jnp.float32),
    ↪NamedSharding(mesh, P('x', 'y')))
ffn_wu = jax.device_put(jax.random.normal(_ku, (pD, pF), jnp.float32),
    ↪NamedSharding(mesh, P(None, 'y')))
ffn_wd = jax.device_put(jax.random.normal(_kd, (pF, pD), jnp.float32),
    ↪NamedSharding(mesh, P('y', None)))

# baseline
def _ffn_ref(x, w_up, w_down):
    return jnp.einsum('bf,fd->bd', jnp.einsum('bd,df->bf', x, w_up), w_down)
ffn_jit = jax.jit(jax.sharding.auto_axes(_ffn_ref, out_sharding=P('x',
    ↪'y'))))
ffn_collective = jax.jit(ffn_block)

benchmark(ffn_jit, ffn_x, ffn_wu, ffn_wd, name='ffn_jit')
benchmark(ffn_collective, ffn_x, ffn_wu, ffn_wd, name='ffn_collective')

profile_and_save(ffn_jit, ffn_x, ffn_wu, ffn_wd, name='q3p3_ffn_jit')
profile_and_save(ffn_collective, ffn_x, ffn_wu, ffn_wd,
    ↪name='q3p3_ffn_collective')

```

```

ffn_jit: 2.255 ms/iter (50 iters)
ffn_collective: 2.002 ms/iter (50 iters)
trace dir : /kaggle/working/profiles/q3p3_ffn_jit
zip saved : /kaggle/working/profiles/q3p3_ffn_jit.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.
/kaggle/working/profiles/q3p3_ffn_jit.zip

```



```

trace dir : /kaggle/working/profiles/q3p3_ffn_collective
zip saved : /kaggle/working/profiles/q3p3_ffn_collective.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q3p3_ffn_collective.zip

```

```
[24]: '/kaggle/working/profiles/q3p3_ffn_collective.zip'
```

2.4 Part 4

```

[25]: @jax.shard_map(in_specs=(P('x', 'y'), P('y', None)), out_specs=P('x', 'y'))
def collective_reduce_scatter_bidi(b, w):
    y_size = jax.lax.axis_size('y')
    chunk_size = w.shape[-1] // y_size
    idx = jax.lax.axis_index('y')

    assert y_size % 2 == 0 # this impl only works for even axis size y

    def get_dest_idx(i):
        from_left = ((y_size // 2) + idx - i) % y_size
        from_right = ((y_size // 2) + idx + i) % y_size
        return from_left, from_right

    def move_right_perm():
        return [(j, (j+1)%y_size) for j in range(y_size)]

    def move_left_perm():
        return [(j, (j-1)%y_size) for j in range(y_size)]

    def body(i, acc):
        left_acc, right_acc = acc
        left_dest_idx, right_dest_idx = get_dest_idx(i)
        left_dest_chunk = jax.lax.dynamic_slice_in_dim(w,
        ↪left_dest_idx*chunk_size, chunk_size, axis=1)
        right_dest_chunk = jax.lax.dynamic_slice_in_dim(w,
        ↪right_dest_idx*chunk_size, chunk_size, axis=1)
        left_acc += jnp.einsum('bf,fd->bd', b, left_dest_chunk)
        right_acc += jnp.einsum('bf,fd->bd', b, right_dest_chunk)
        return (
            jax.lax.ppermute(left_acc, axis_name='y', perm=move_right_perm()),
            jax.lax.ppermute(right_acc, axis_name='y', perm=move_left_perm()),
        )

    opp_idx = ((y_size // 2) + idx) % y_size
    opp_chunk = jax.lax.dynamic_slice_in_dim(w, opp_idx*chunk_size, chunk_size,
    ↪axis=1)
    left_init = jnp.einsum('bf,fd->bd', b, opp_chunk)
    perm = [(j, (j+1)%y_size) for j in range(y_size)]

```

```

left_init = jax.lax.ppermute(left_init, axis_name='y', perm=perm)
right_init = jnp.zeros((b.shape[0], chunk_size))
right_init = jax.lax.pcast(right_init, ('x', 'y'), to='varying')

left_acc, right_acc = jax.lax.fori_loop(1, (y_size // 2), body, (left_init,
↪right_init), unroll=True)

local_chunk = jax.lax.dynamic_slice_in_dim(w, idx*chunk_size, chunk_size,
↪axis=1)

return left_acc + right_acc + jnp.einsum('bf,fd->bd', b, local_chunk)

```

```

[26]: import numpy as np
import jax, jax.numpy as jnp
from jax.sharding import NamedSharding, PartitionSpec as P

B, F, D = 8, 32, 16
kT, kW = jax.random.split(jax.random.PRNGKey(1), 2)
Tmp = jax.random.normal(kT, (B, F), jnp.float32)      # [B, F]
W2 = jax.random.normal(kW, (F, D), jnp.float32)      # [F, D]

Tmp_sh = jax.device_put(Tmp, NamedSharding(mesh, P('x', 'y')))
W2_sh = jax.device_put(W2, NamedSharding(mesh, P('y', None)))

# unsharded ref
ref = Tmp @ W2

out = collective_reduce_scatter_bidi(Tmp_sh, W2_sh)

out_np, ref_np = np.asarray(out), np.asarray(ref)
print("out:", out_np.shape, " ref:", ref_np.shape,
      " sharding:", out.sharding.spec,
      " max_err:", f"{np.abs(out_np - ref_np).max():.2e}")

```

out: (8, 16) ref: (8, 16) sharding: P('x', 'y') max_err: 9.54e-07

```

[27]: pB, pF, pD = 4096, 8192, 8192
_kt, _kw = jax.random.split(jax.random.PRNGKey(13), 2)
bd_t = jax.device_put(jax.random.normal(_kt, (pB, pF), jnp.float32),
↪NamedSharding(mesh, P('x', 'y')))
bd_w = jax.device_put(jax.random.normal(_kw, (pF, pD), jnp.float32),
↪NamedSharding(mesh, P('y', None)))

rs_jit = jax.jit(jax.sharding.auto_axes(
    lambda t, w: jnp.einsum('bf,fd->bd', t, w), out_sharding=P('x', 'y')))
rs_uni = jax.jit(

```

```

    jax.shard_map(in_specs=(P('x', 'y'), P('y', None)), out_specs=P('x',
↪ 'y'))(collective_reduce_scatter)
)
rs_bidi = jax.jit(collective_reduce_scatter_bidi)

benchmark(rs_jit, bd_t, bd_w, name='reducescatter_jit')
benchmark(rs_uni, bd_t, bd_w, name='reducescatter_uni')
benchmark(rs_bidi, bd_t, bd_w, name='reducescatter_bidi')

profile_and_save(rs_uni, bd_t, bd_w, name='q3p4_reducescatter_uni')
profile_and_save(rs_bidi, bd_t, bd_w, name='q3p4_reducescatter_bidi')

```

```

    reducescatter_jit:    1.595 ms/iter   (50 iters)
    reducescatter_uni:    1.319 ms/iter   (50 iters)
    reducescatter_bidi:    1.227 ms/iter   (50 iters)
trace dir : /kaggle/working/profiles/q3p4_reducescatter_uni
zip saved : /kaggle/working/profiles/q3p4_reducescatter_uni.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q3p4_reducescatter_uni.zip

trace dir : /kaggle/working/profiles/q3p4_reducescatter_bidi
zip saved : /kaggle/working/profiles/q3p4_reducescatter_bidi.zip
-> grab it from the Kaggle 'Output' panel (right side), then open locally.

/kaggle/working/profiles/q3p4_reducescatter_bidi.zip

```

[27]: '/kaggle/working/profiles/q3p4_reducescatter_bidi.zip'

[]:

References

- [1] Jacob Austin et al. *How to Scale Your Model: A Systems View of LLMs on TPUs*. <https://jax-ml.github.io/scaling-book/>. 2025.